

Redes neuronales

Tema 3.1

Contenido

1. Introducción
2. Arquitectura
3. Entrenamiento
4. Técnicas
 - Evitar el sobreajuste
 - Acelerar el entrenamiento
5. Resumen

Contenido

1. Introducción

2. Arquitectura

3. Entrenamiento

4. Técnicas

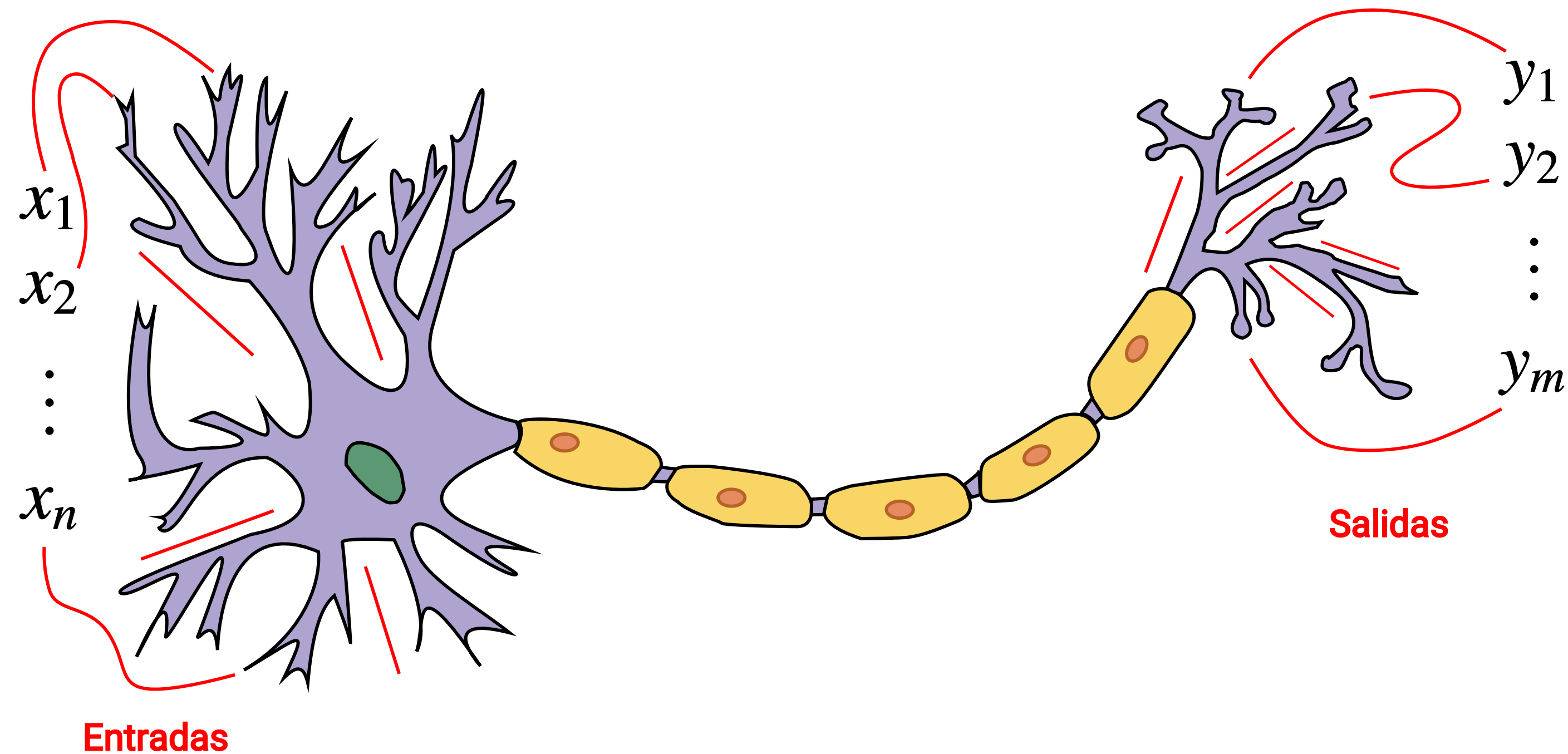
- Evitar el sobreajuste
- Acelerar el entrenamiento

5. Resumen

Introducción

■ Origen

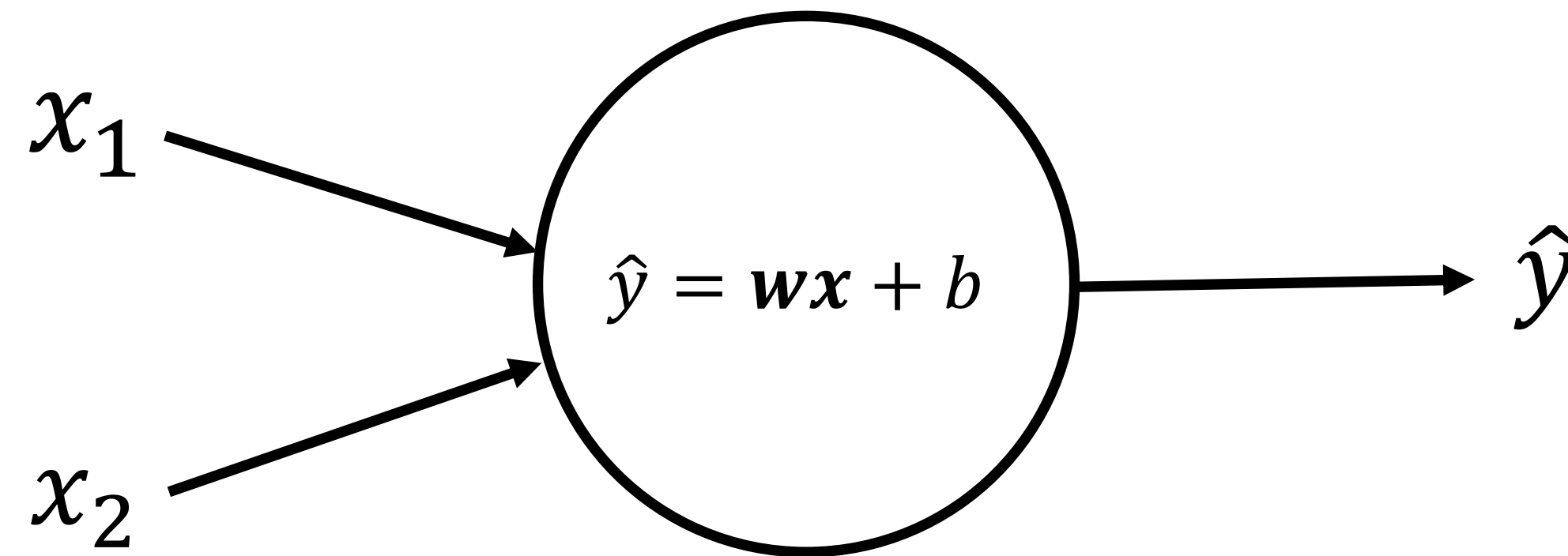
- El **perceptrón** es la pieza clave de cualquier red neuronal.
- Introducido en 1957 por Frank Rosenblatt.
- Es un modelo motivado por la biología.



Introducción

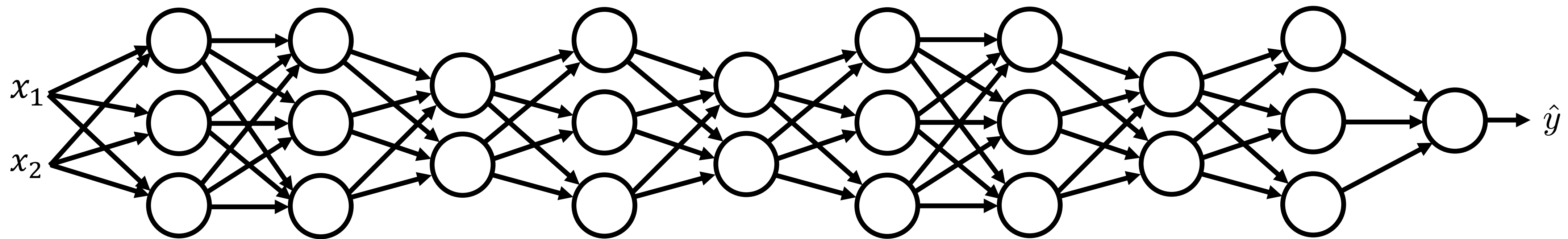
■ Origen

- El **perceptrón** es la pieza clave de cualquier red neuronal.
- Introducido en 1957 por Frank Rosenblatt.
- Es un modelo motivado por la biología.



Introducción

- **Redes totalmente conectadas (fully connected)**
 - Un solo perceptrón está muy limitado a funciones simples y lineales.
 - Las redes neuronales:
 - Unen varios perceptrones en una misma red.



Red neuronal de 10 capas (red profunda o deep neural network)

Introducción

■ Notación:

- Un ejemplo se representa mediante n variables: $x \in \mathbb{R}^n$
- El valor real para un ejemplo dado se representa: $y \in \mathbb{R}$
- El conjunto de datos se compone de m ejemplos del tipo: (x, y)

A table representing a dataset with 4 columns and 4 rows. The columns are labeled x1, x2, x3, and y. The first three columns (x1, x2, x3) are highlighted in orange, and the last column (y) is highlighted in green. A horizontal double-headed arrow above the table is labeled n , indicating the number of features. A vertical double-headed arrow to the left of the table is labeled m , indicating the number of examples. The first row of data is highlighted with a yellow border, showing the values -3.9, 12.1, 0.34, and 2.1.

x1	x2	x3	y
-3.9	12.1	0.34	2.1
7.01	-2	-4.2	-3.21
-9.2	3.12	-1.98	-8.3
0.01	7.23	-1.21	1.67

Mathematical representation of the dataset. The feature matrix X is a 4x4 matrix with rows corresponding to the first three columns of the table. The target vector y is a 1x4 row vector corresponding to the y column. A yellow box highlights the first column of X and the first element of y . A vertical double-headed arrow to the right of X is labeled n , and a horizontal double-headed arrow below y is labeled m .

$$X = \begin{bmatrix} -3,9 & 7,01 & -9,2 & 0,01 \\ 12,1 & -2 & 3,12 & 7,23 \\ 0,34 & -4,2 & -1,98 & -1,21 \end{bmatrix}$$
$$y = [2,1 \quad -3,2 \quad -8,3 \quad 1,67]$$

Introducción

- **Objetivo:**

- Buscar la función: $f: \mathbb{R}^n \rightarrow \mathbb{R}, f(x, \theta) = f_{\theta}(x)$
- Que sea más **coherente** con los datos disponibles: $D = \{(x, y): x \rightarrow y\}$

x_1	x_2	x_3	y
-3.9	12.1	0.34	2.1
7.01	-2	-4.2	-3.21
-9.2	3.12	-1.98	-8.3
0.01	7.23	-1.21	1.67

Introducción

- ¿Qué significa coherente?:
 - Que la predicción $\hat{y}$ se aproxime al valor real y .
 - Haciendo mínima una función de pérdida.

x_1	x_2	x_3	y	$\hat{y}$
-3.9	12.1	0.34	2.1	1.8
7.01	-2	-4.2	-3.21	0.1
-9.2	3.12	-1.98	-8.3	-3.03
0.01	7.23	-1.21	1.67	-0.81

Introducción

■ Objetivo:

- Tendremos que resolver una ecuación del tipo:

$$\theta \leftarrow \underset{\theta}{\operatorname{argmin}} \frac{1}{|D|} \sum_{(x,y) \in D} \operatorname{loss}(y, f_{\theta}(x))$$

- Ejemplo de regresión:

- **Loss:** Error cuadrático

x_1	x_2	x_3	y	$\hat{y}$	loss
-3.9	12.1	0.34	2.1	1.8	0.09
7.01	-2	-4.2	-3.21	0.1	10.96
-9.2	3.12	-1.98	-8.3	-3.03	27.77
0.01	7.23	-1.21	1.67	-0.81	6.15

$$f_w(x) = w \cdot x$$
$$\operatorname{loss}(y, f_w(x)) = (y - f_{\theta}(x))^2$$

Introducción

- **Objetivo:**

- En resumen, dado un conjunto:

$$D = \{(x, y) : x \rightarrow y\}$$

- Diseñaremos una red neuronal con unos parámetros θ :

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(x, \theta) = f_{\theta}(x)$$

- Que se optimizarán para el problema a resolver minimizando la loss:

$$\theta \leftarrow \underset{\theta}{\operatorname{argmin}} \frac{1}{|D|} \sum_{(x, y) \in D} \operatorname{loss}(y, f_{\theta}(x))$$

- Permitiéndonos realizar predicciones:

$$y \approx \hat{y}, \quad \hat{y} = f(x, \theta)$$

Introducción

■ Tipos de problemas:

- Con datos estructurados:
 - Regresión
 - Clasificación
- Con datos no estructurados:
 - Regresión
 - Clasificación
 - Localización
 - Segmentación
 - Etc..

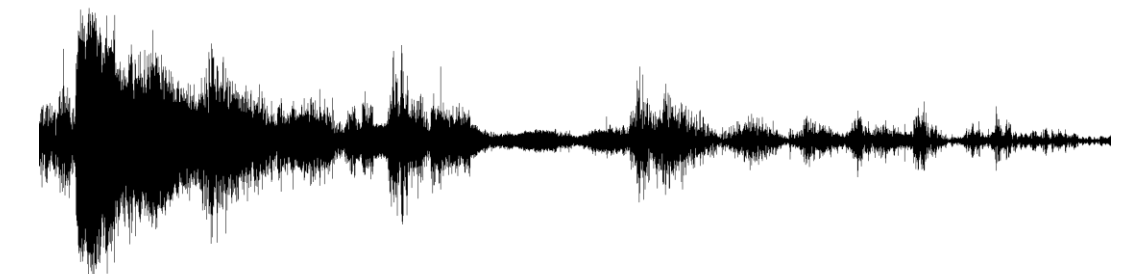
Datos tabulares

x1	x2	x3	y
-3.9	12.1	0.34	2.1
7.01	-2	-4.2	-3.21
-9.2	3.12	-1.98	-8.3
0.01	7.23	-1.21	1.67

Imagen



Sonido



Texto

**La película tiene momentos interesantes,
pero los actores no logran...**

Introducción

Procedimiento a seguir: KDD

- Etapas y pasos necesarios para realizar una tarea de minería de datos.

Fases:

1. Selección
2. Preprocesamiento
3. Codificación
4. Aprendizaje
5. Interpretación y validación

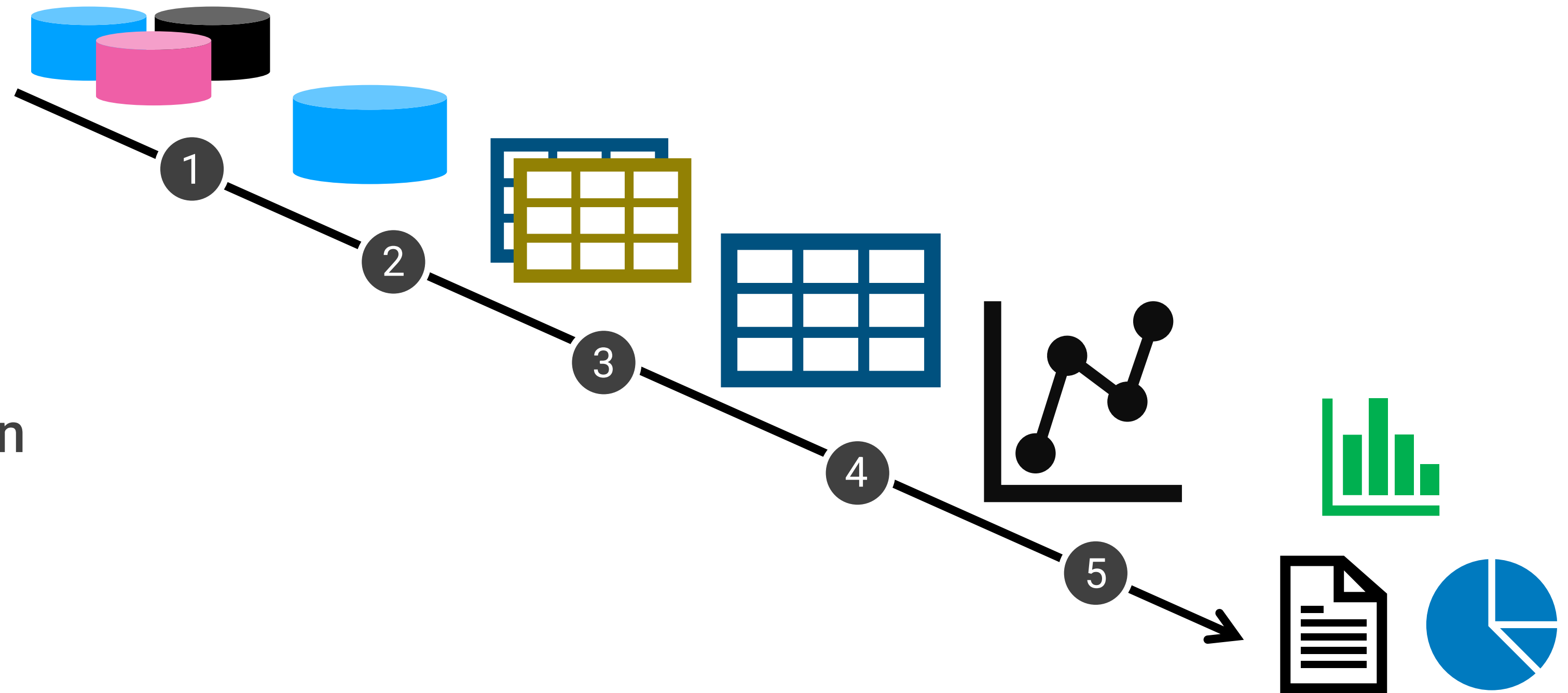


Figura 1: Detalle de etapas y pasos del proceso KDD.

Introducción

Procedimiento a seguir: KDD

- Etapas y pasos necesarios para realizar una tarea de minería de datos.

Fases:

1. Selección
2. Preprocesamiento
3. Codificación
4. Aprendizaje
5. Interpretación y validación

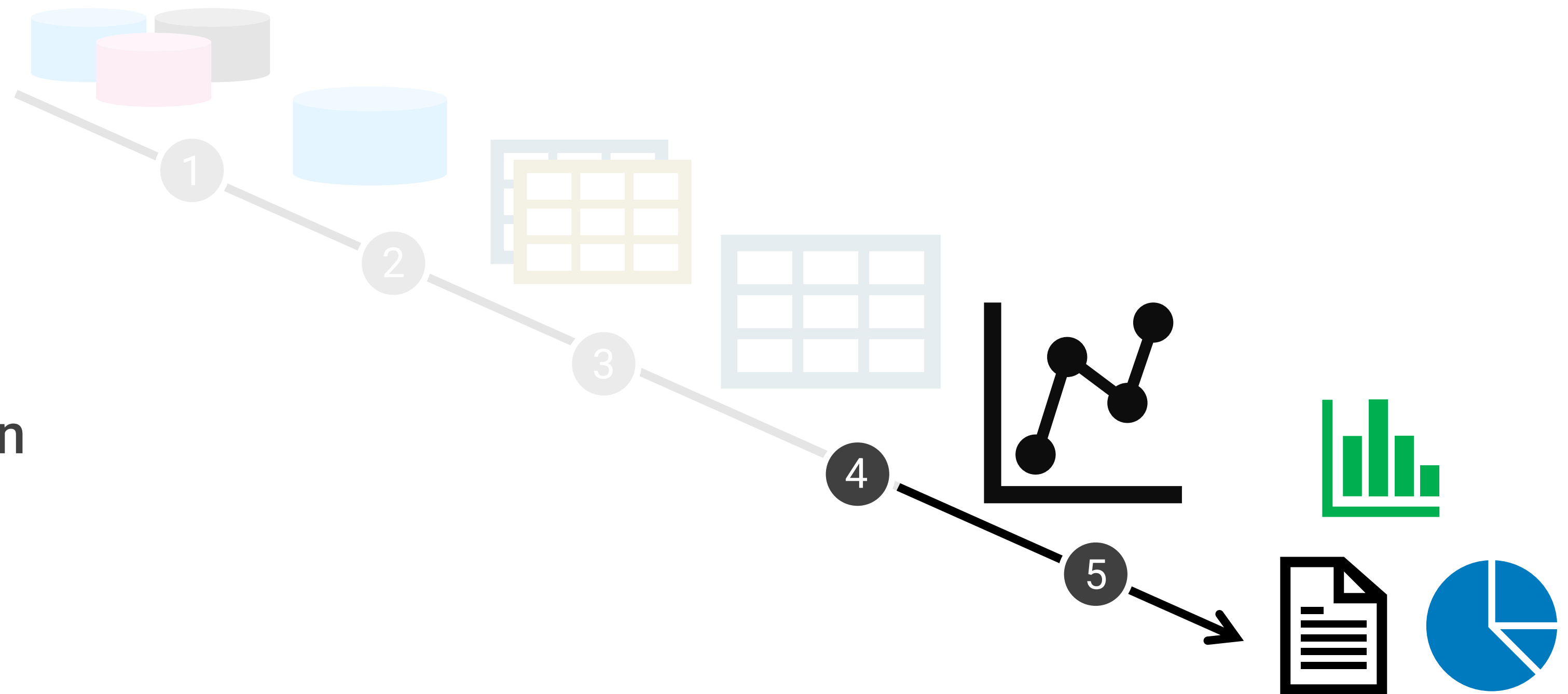


Figura 1: Detalle de etapas y pasos del proceso KDD.

Contenido

1. Introducción

2. Arquitectura

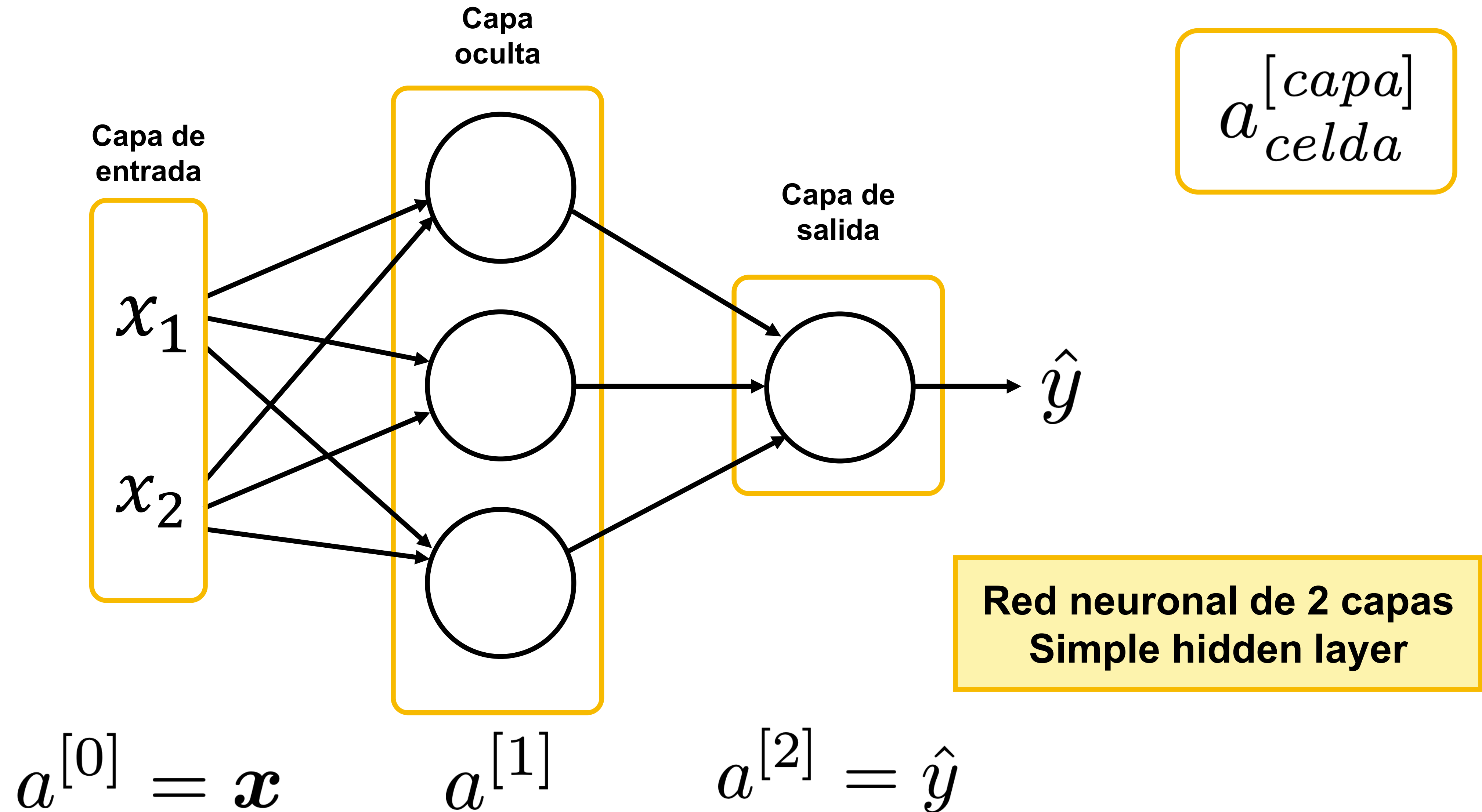
3. Entrenamiento

4. Técnicas

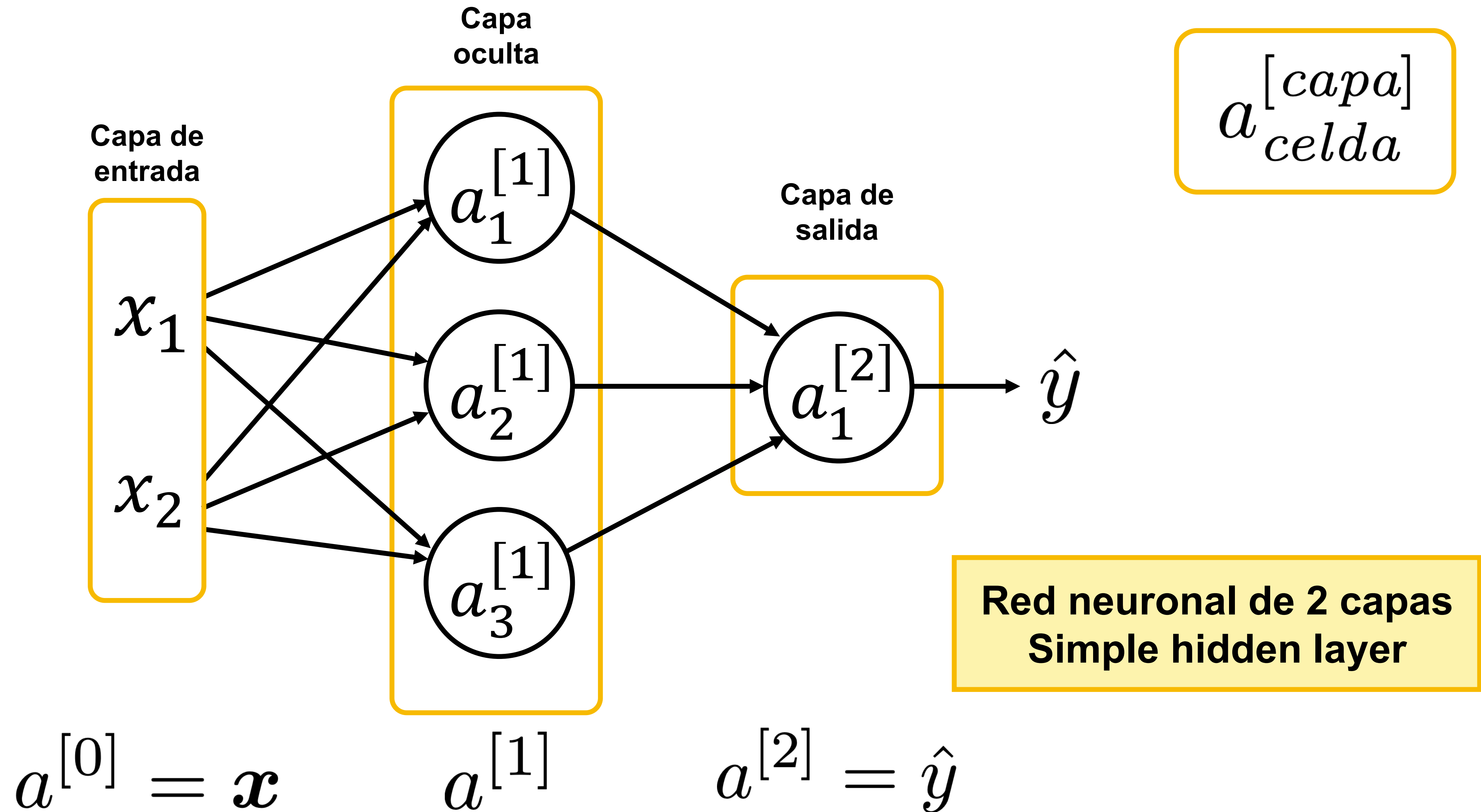
- Evitar el sobreajuste
- Acelerar el entrenamiento

5. Resumen

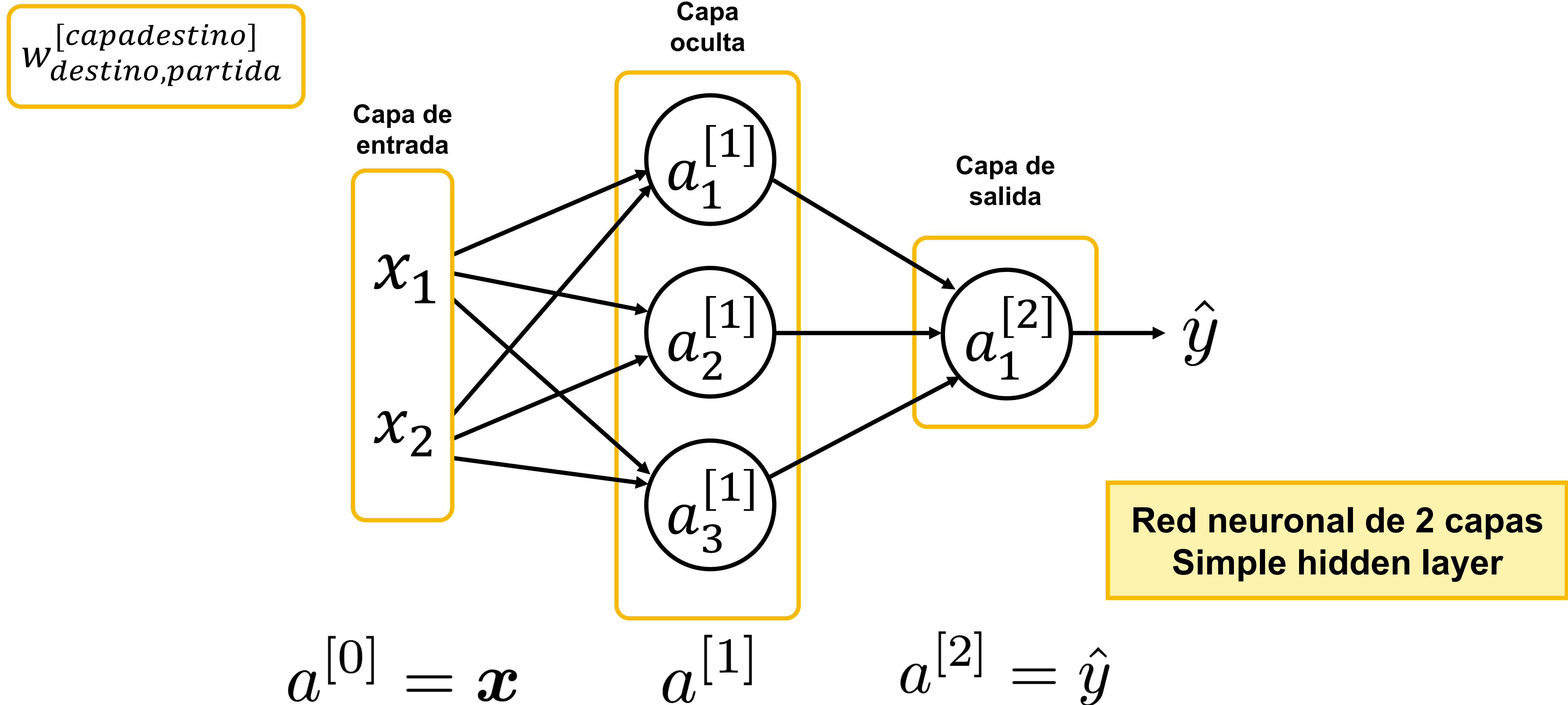
Arquitectura de una red neuronal



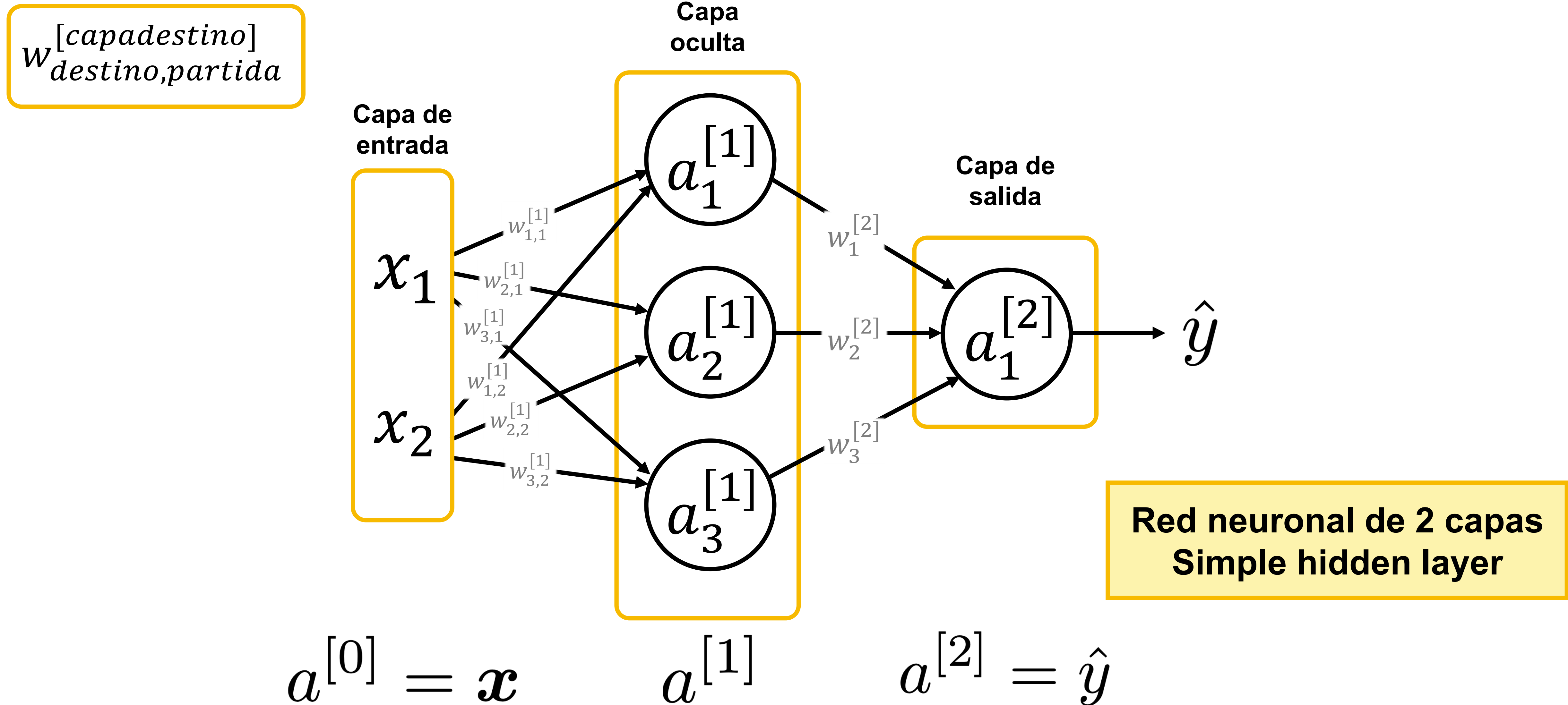
Arquitectura de una red neuronal



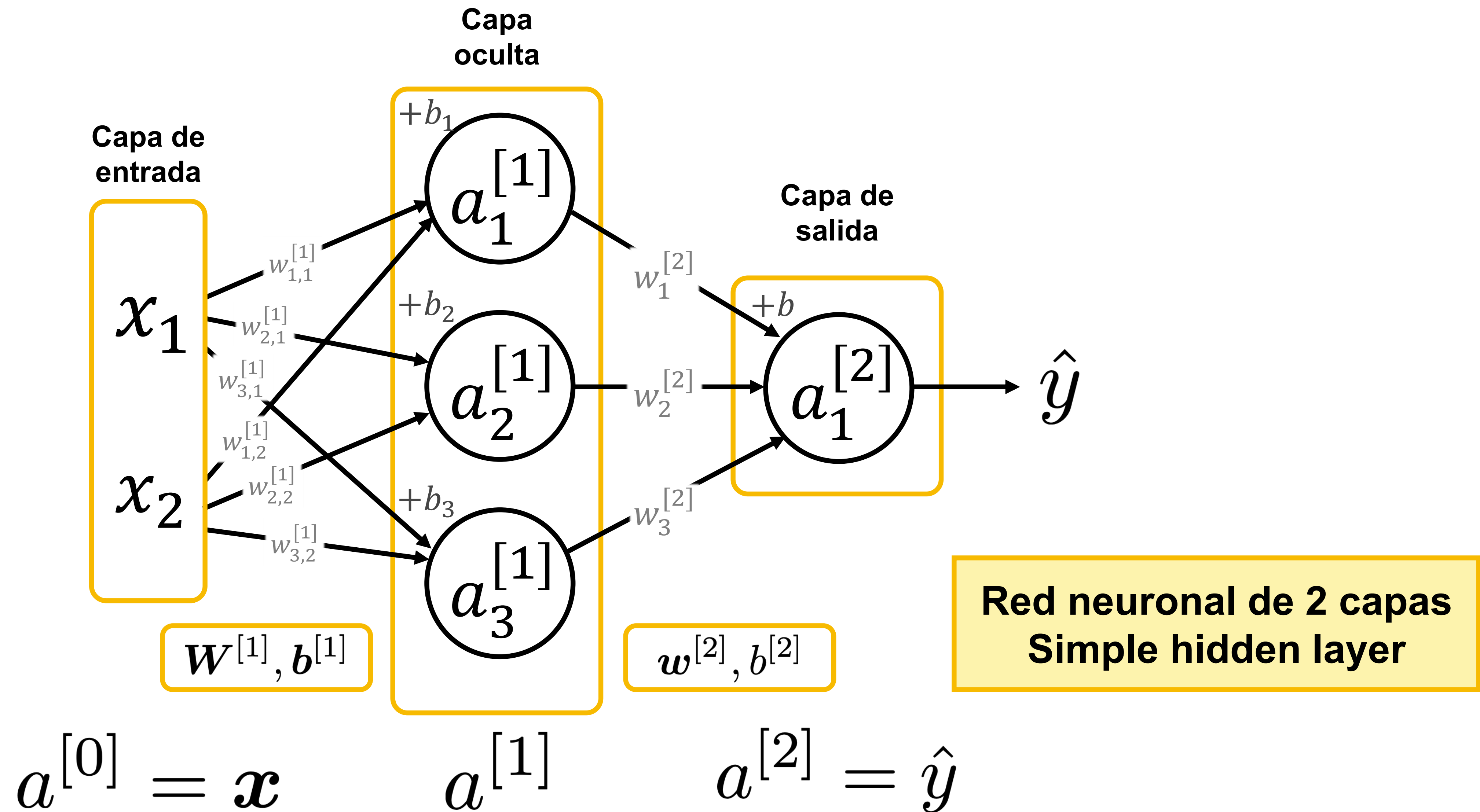
Arquitectura de una red neuronal



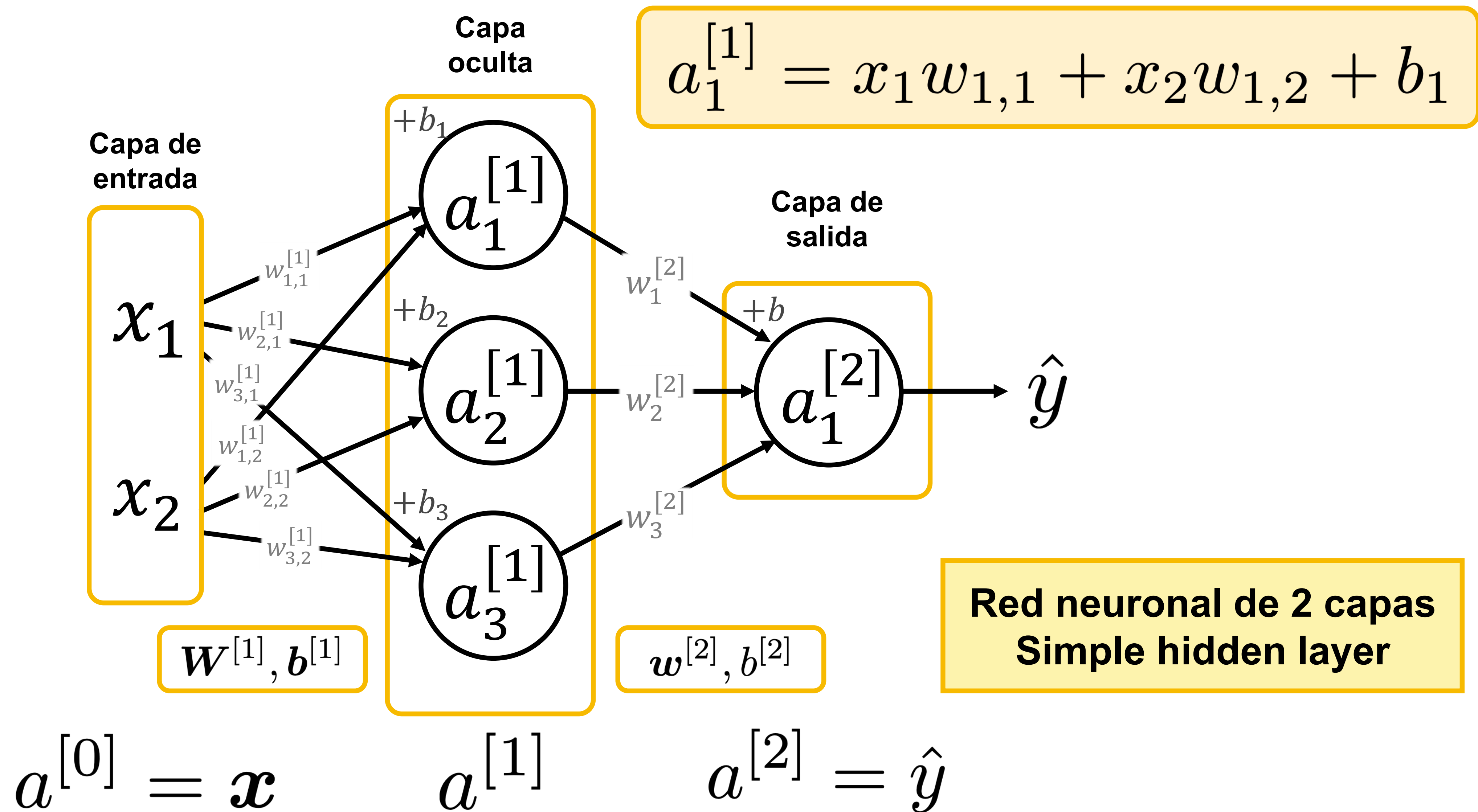
Arquitectura de una red neuronal



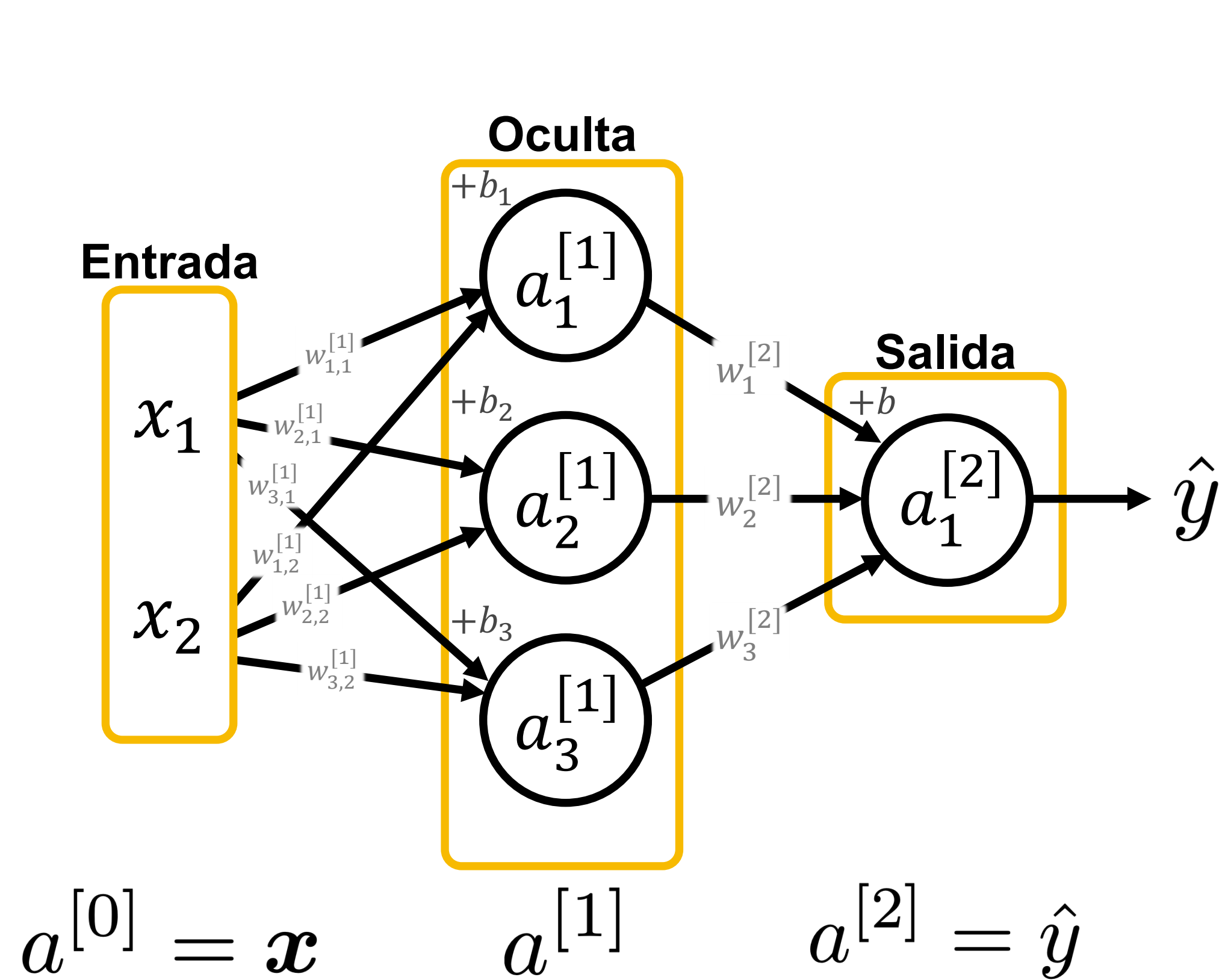
Arquitectura de una red neuronal



Arquitectura de una red neuronal



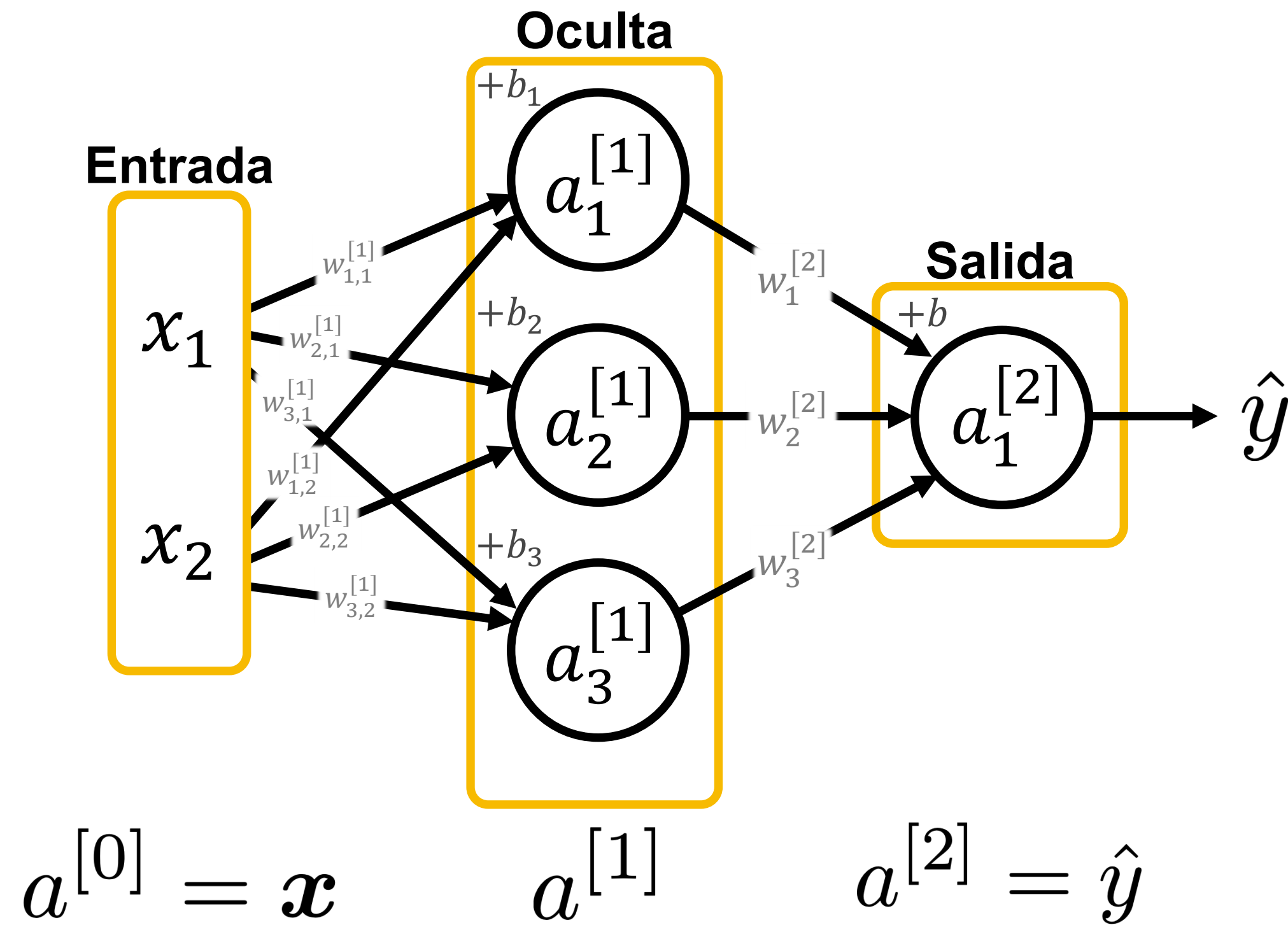
Arquitectura de una red neuronal



$$x = a^{[0]}_{(2,1)} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Red neuronal de 2 capas

Arquitectura de una red neuronal



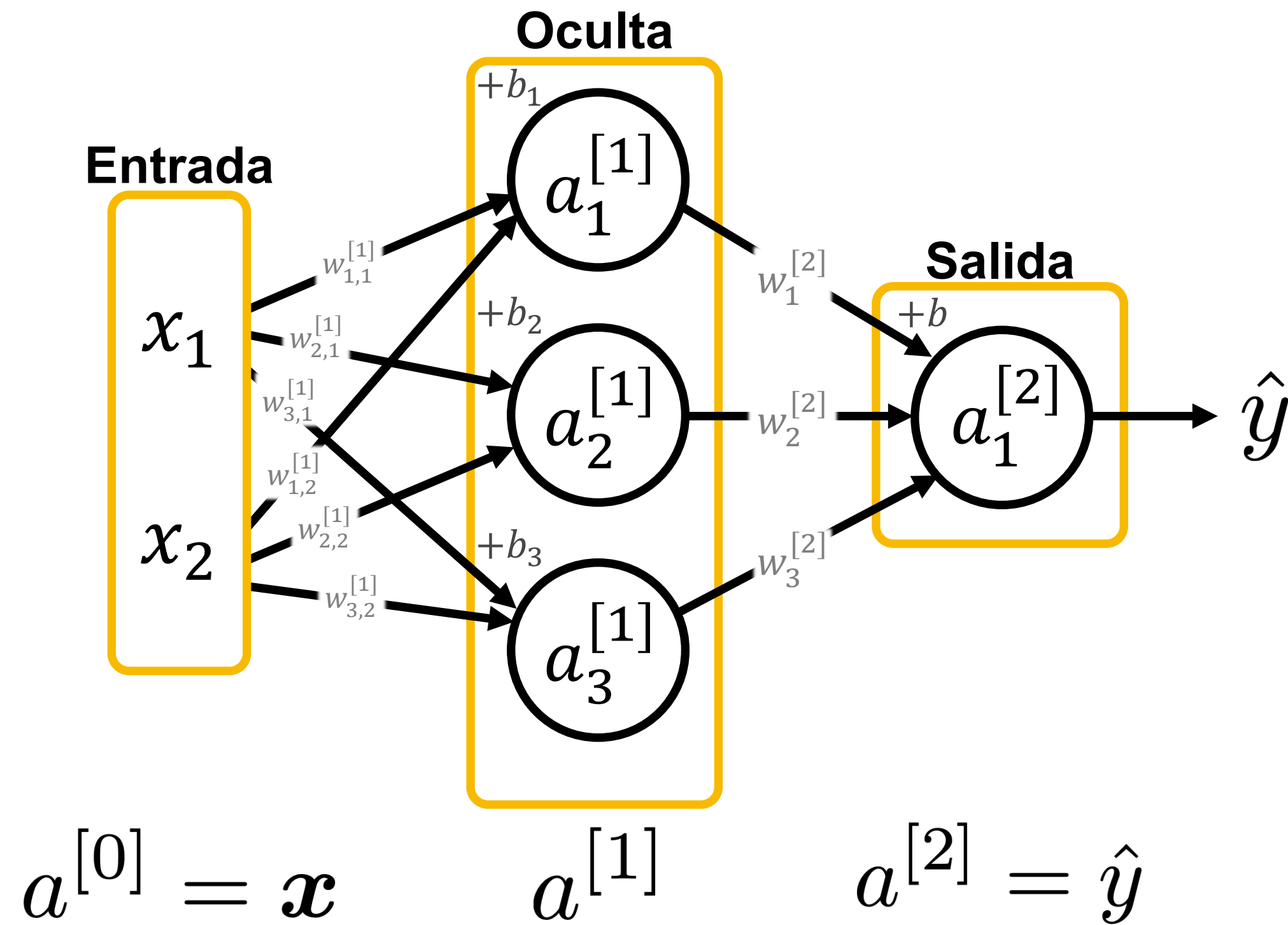
Red neuronal de 2 capas

$$x = a^{[0]}_{(2,1)} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$W^{[1]}_{(3,2)} = \begin{bmatrix} w_{1,1}^{[1]} & w_{1,2}^{[1]} \\ w_{2,1}^{[1]} & w_{2,2}^{[1]} \\ w_{3,1}^{[1]} & w_{3,2}^{[1]} \end{bmatrix} \quad b^{[1]}_{(3,1)} = \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \end{bmatrix} \quad a^{[1]}_{(3,1)} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \end{bmatrix}$$

$$a^{[1]} = W^{[1]} a^{[0]} + b^{[1]} = W^{[1]} x + b^{[1]}$$

Arquitectura de una red neuronal



Red neuronal de 2 capas

$$x = a^{[0]}_{(2,1)} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

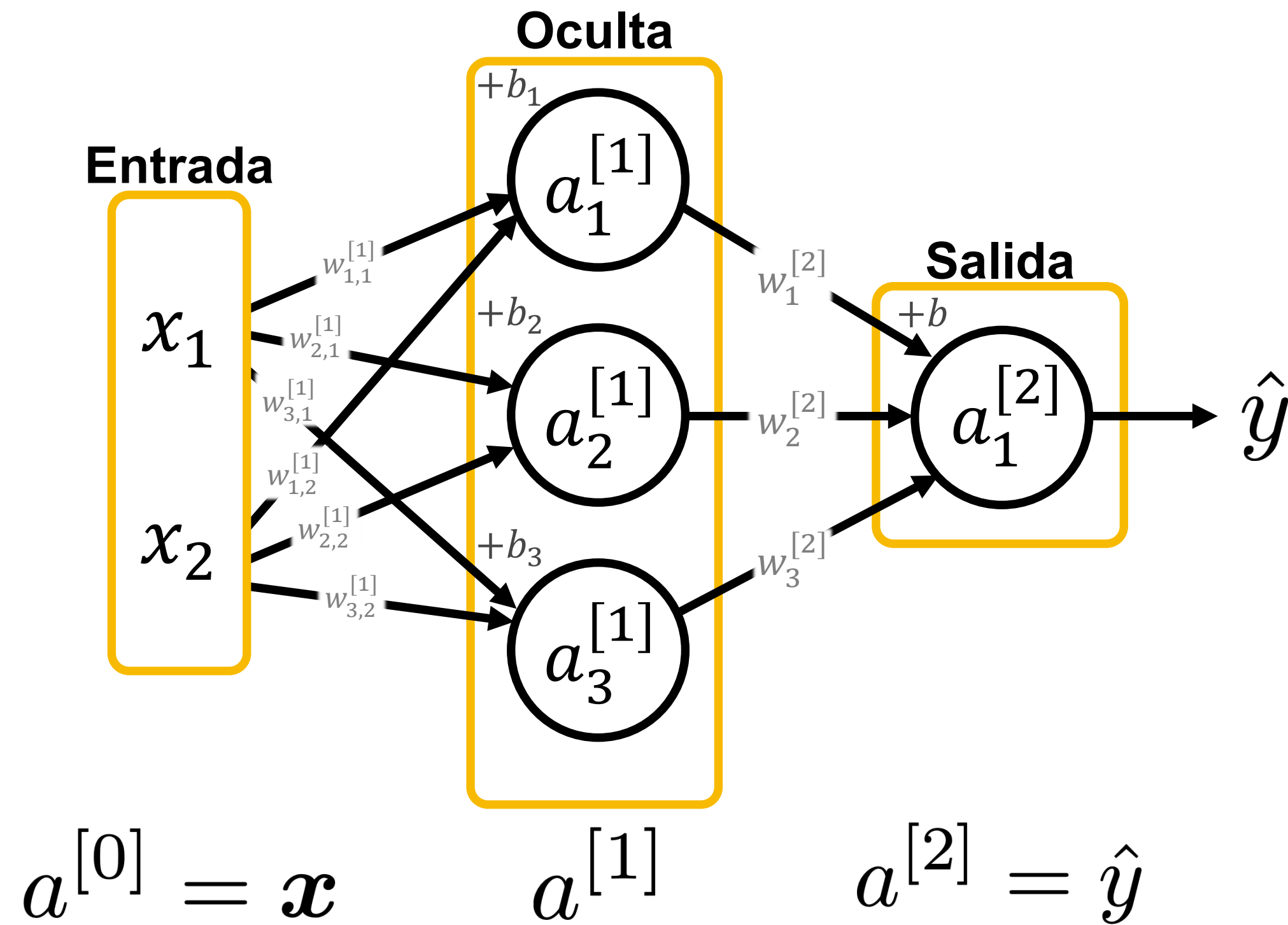
$$W^{[1]}_{(3,2)} = \begin{bmatrix} w_{1,1}^{[1]} & w_{1,2}^{[1]} \\ w_{2,1}^{[1]} & w_{2,2}^{[1]} \\ w_{3,1}^{[1]} & w_{3,2}^{[1]} \end{bmatrix} \quad b^{[1]}_{(3,1)} = \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \end{bmatrix} \quad a^{[1]}_{(3,1)} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \end{bmatrix}$$

$$a^{[1]} = W^{[1]} a^{[0]} + b^{[1]} = W^{[1]} x + b^{[1]}$$

$$w^{[2]}_{(1,3)} = \begin{bmatrix} w_1^{[2]} & w_2^{[2]} & w_3^{[2]} \end{bmatrix}$$

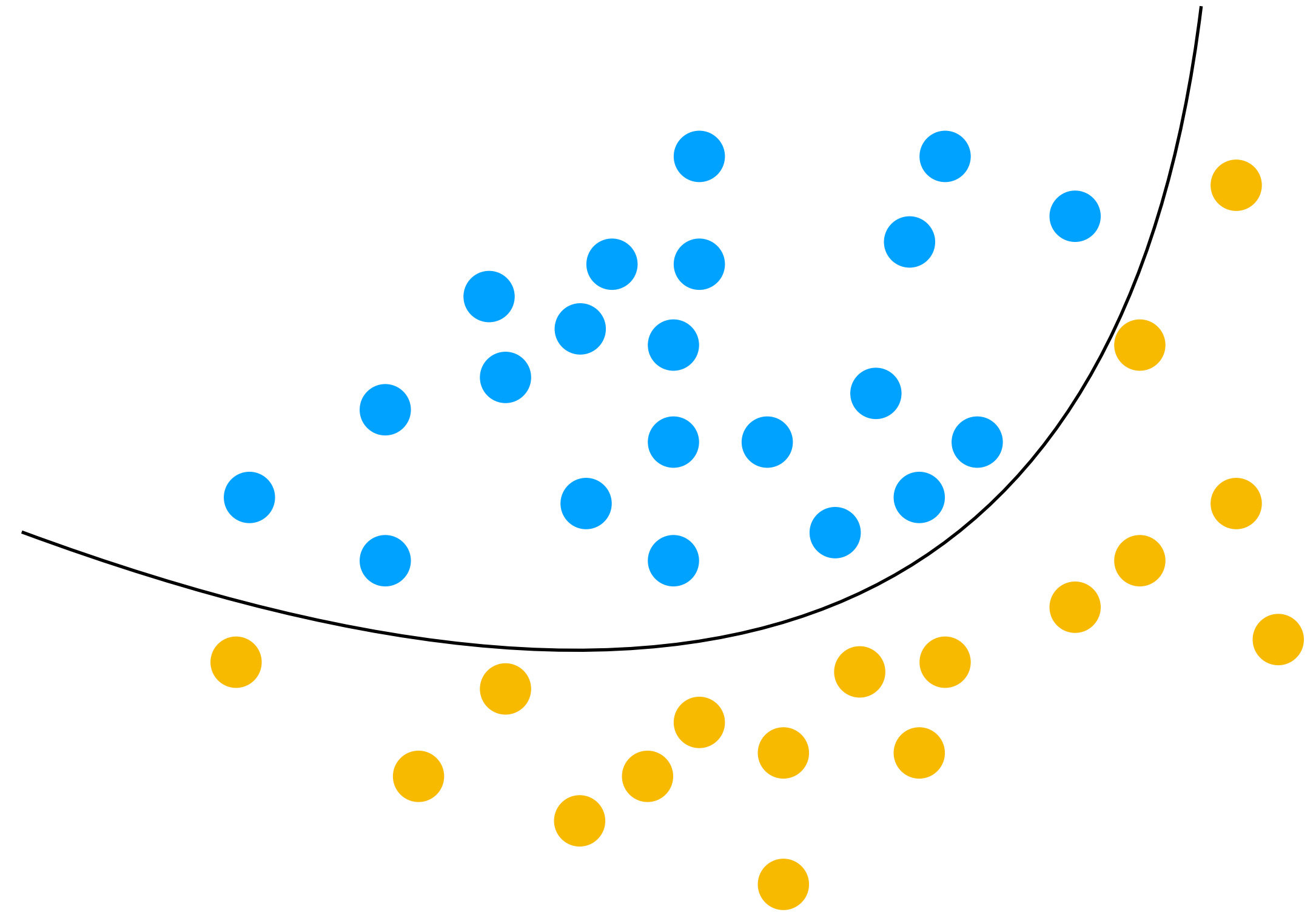
$$a^{[2]} = w^{[2]} a^{[1]} + b^{[2]} = \hat{y}$$

Arquitectura de una red neuronal



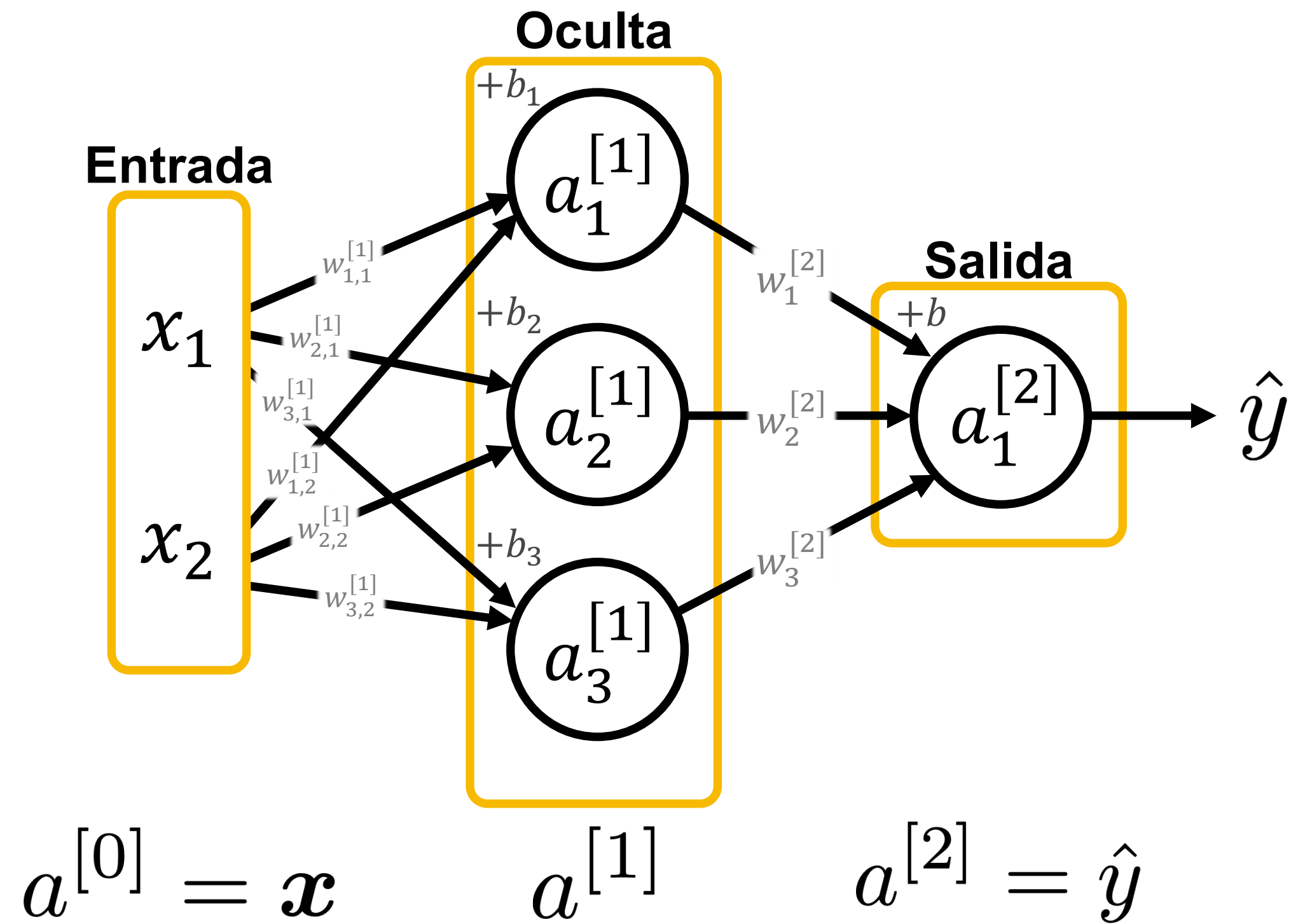
Red neuronal de 2 capas

La composición de funciones lineales da lugar a una **función lineal**

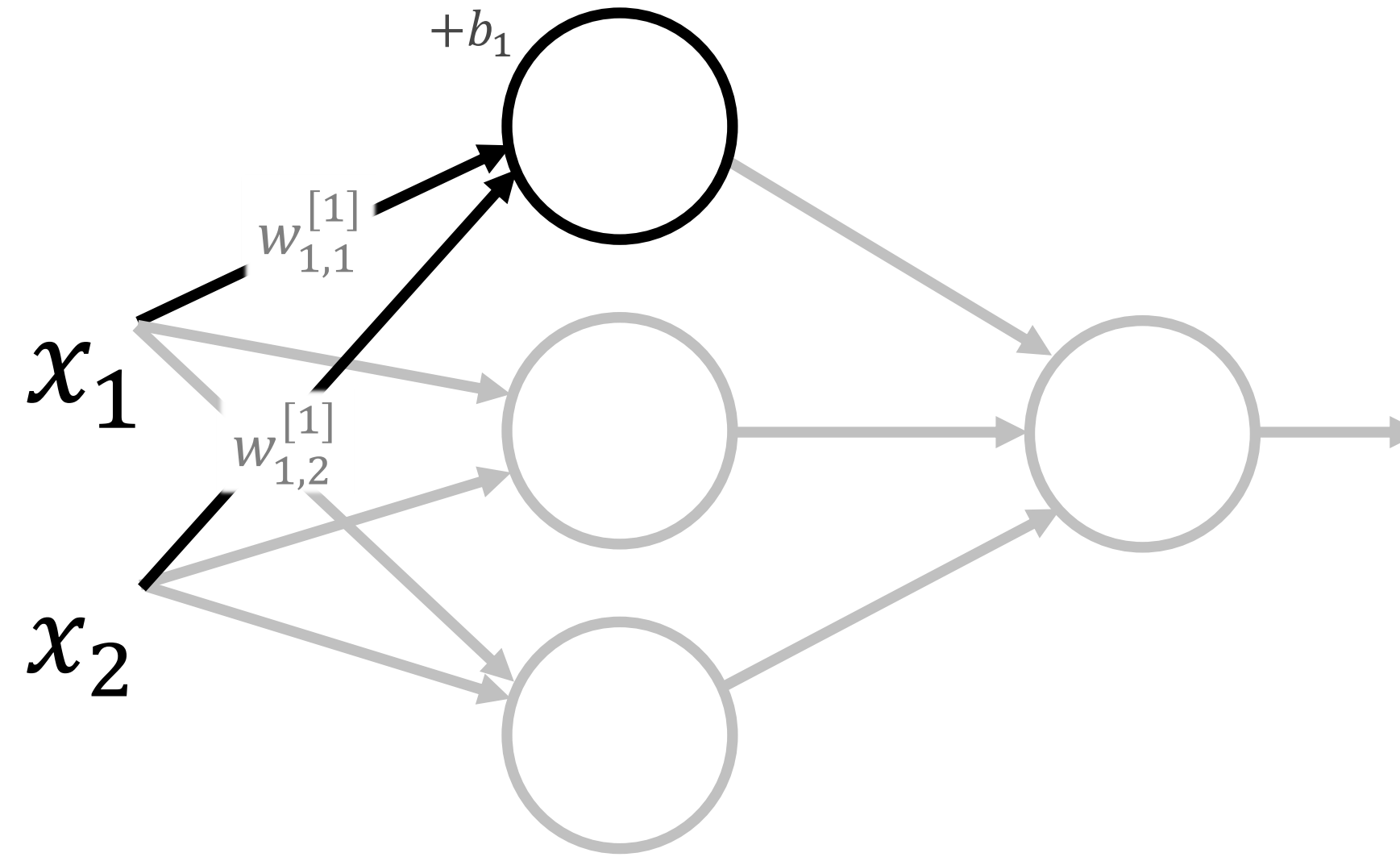


No todos los problemas se pueden resolver de manera lineal

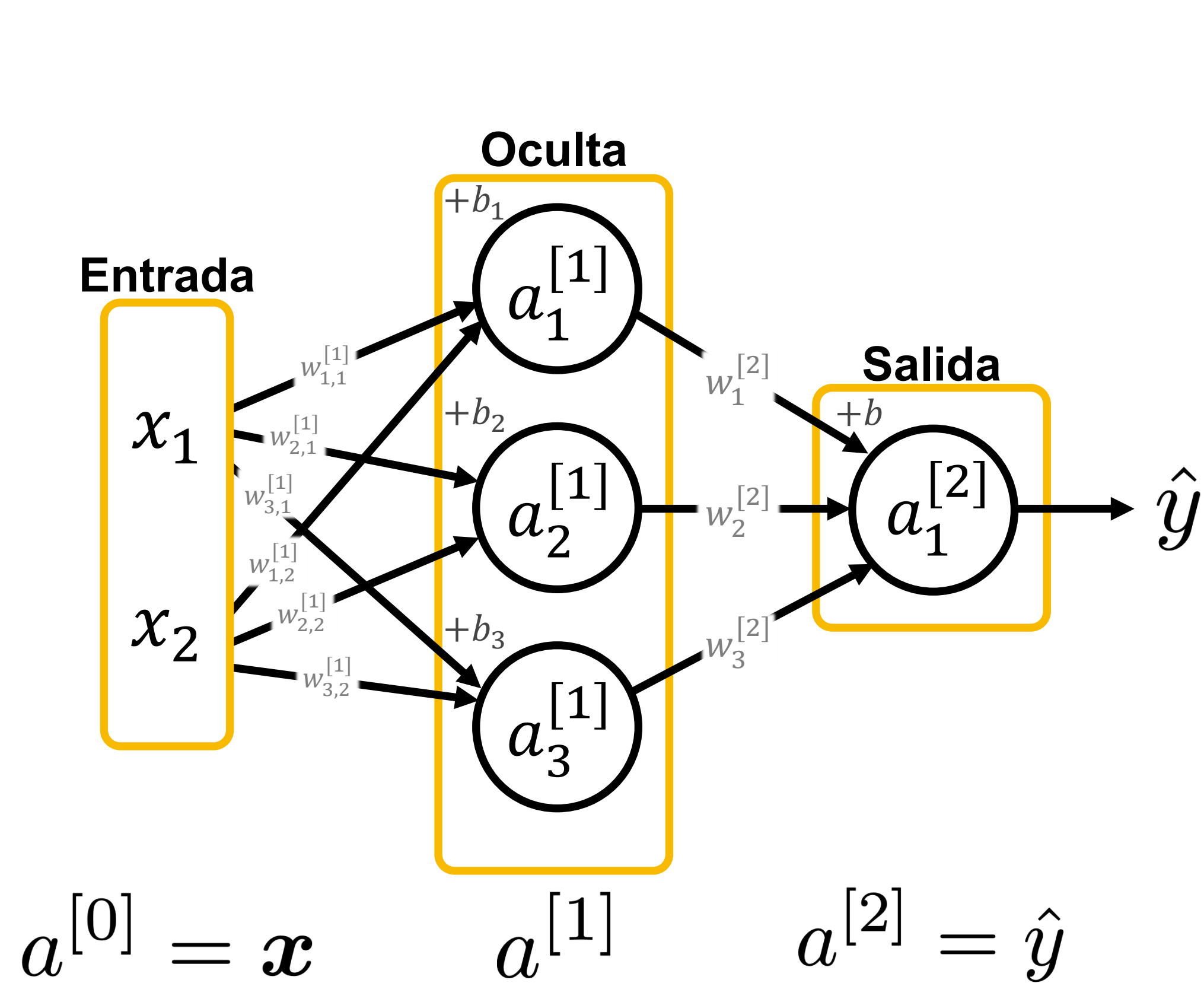
Arquitectura de una red neuronal



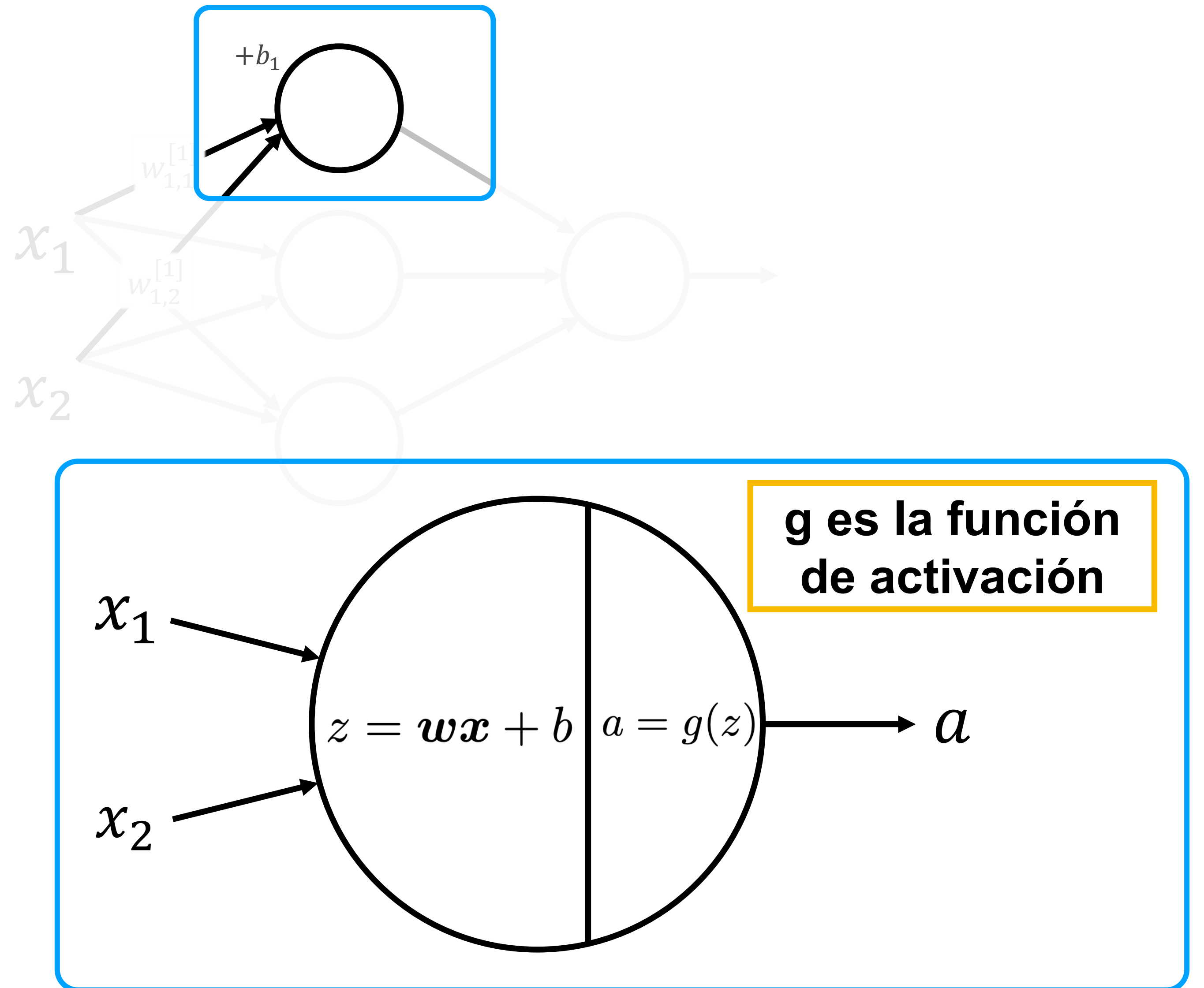
Red neuronal de 2 capas



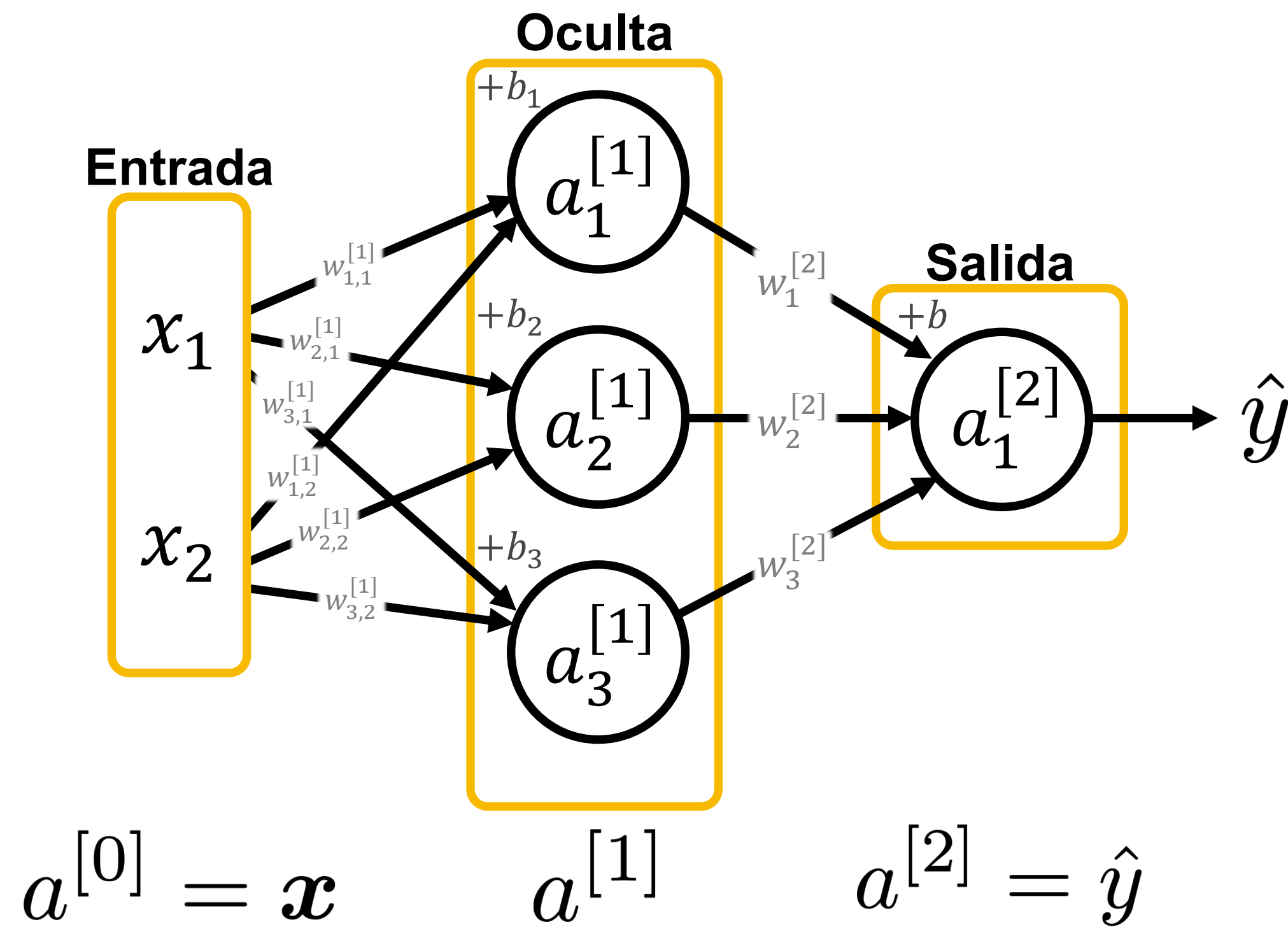
Arquitectura de una red neuronal



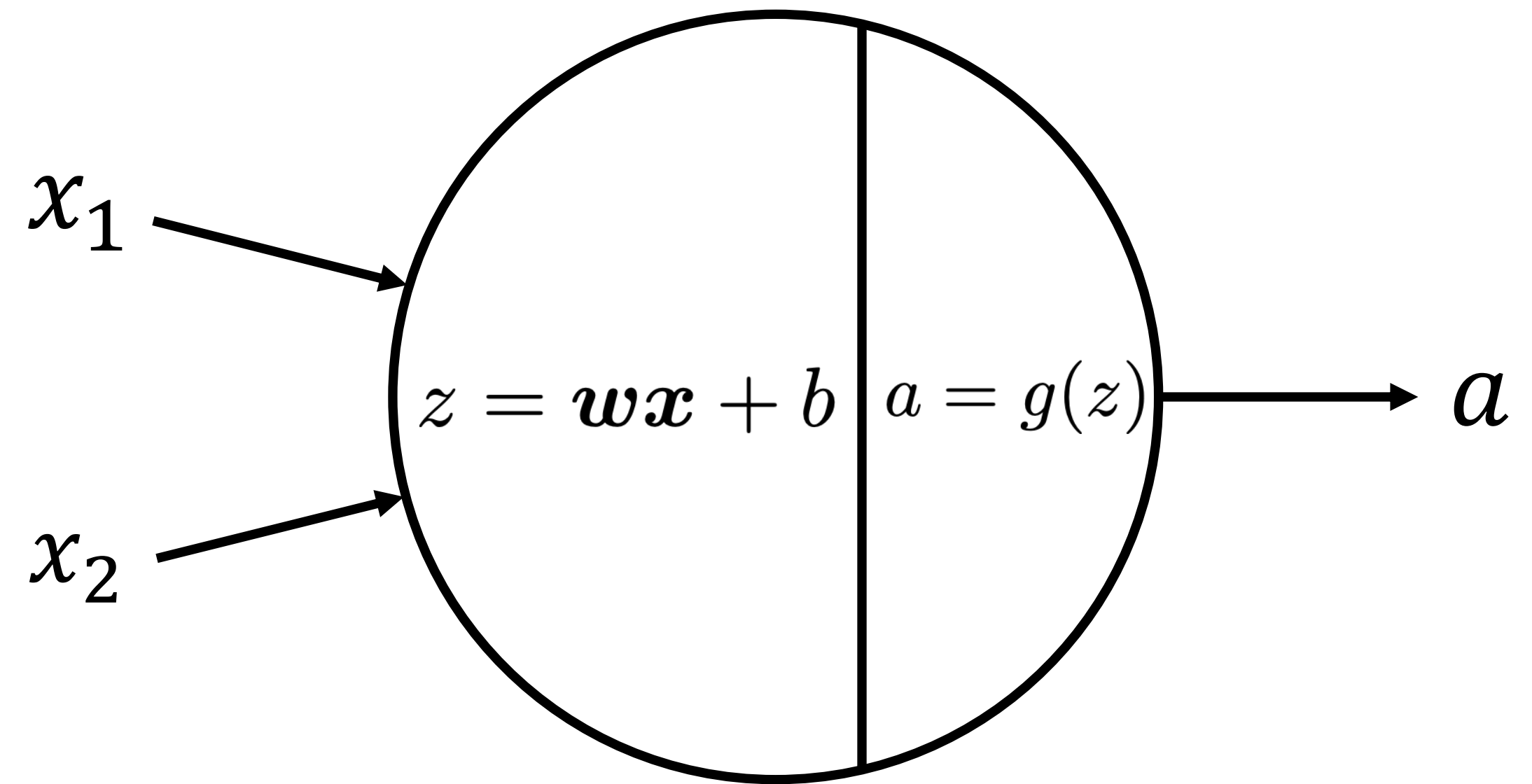
Red neuronal de 2 capas



Funciones de activación



Red neuronal de 2 capas



$$z^{[1]} = \mathbf{W}^{[1]}x + b^{[1]} \qquad z^{[2]} = \mathbf{w}^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[1]} = g(z^{[1]}) \qquad a^{[2]} = g(z^{[2]})$$

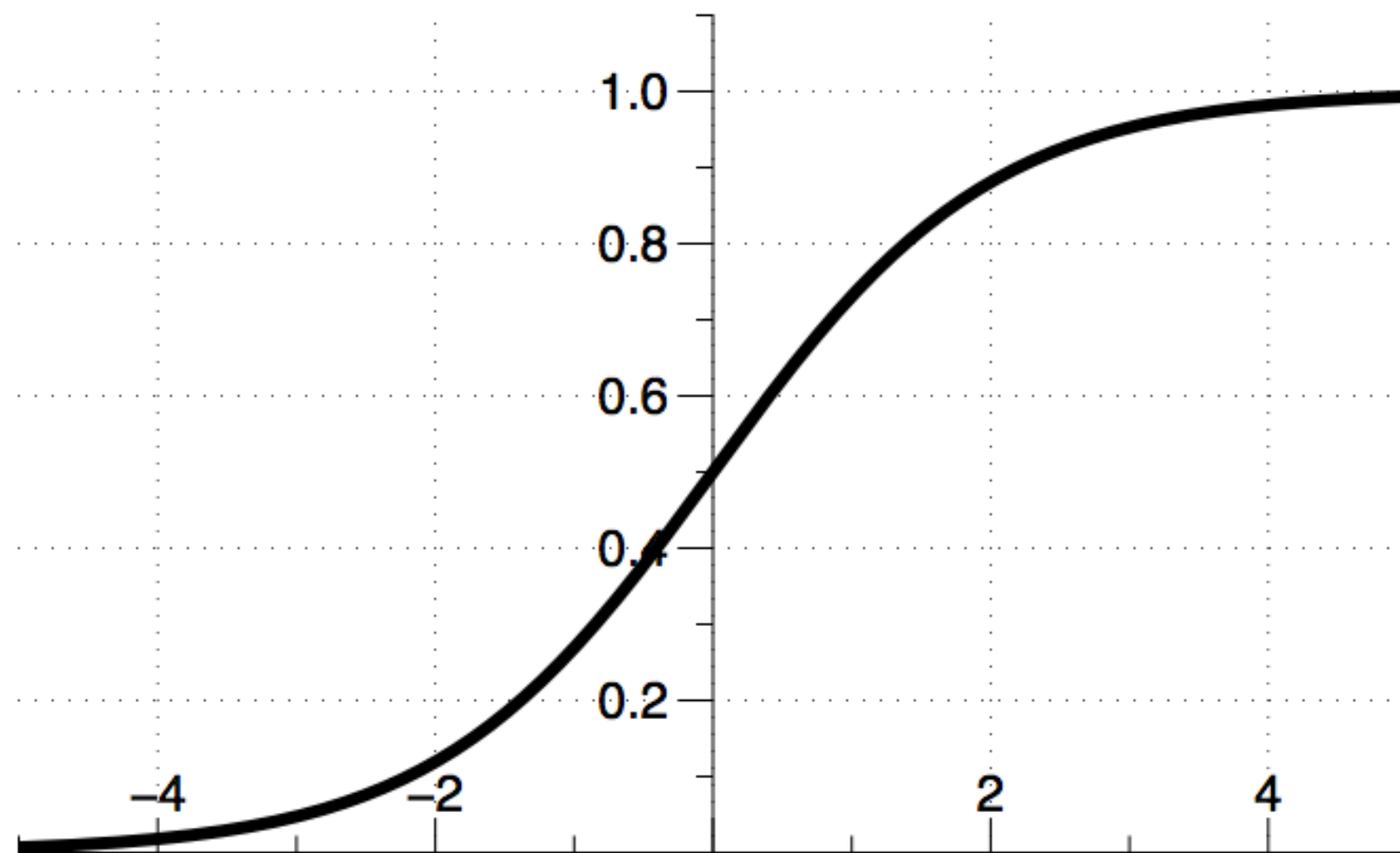
g será una función no lineal

Arquitectura de una red neuronal

■ Funciones de activación

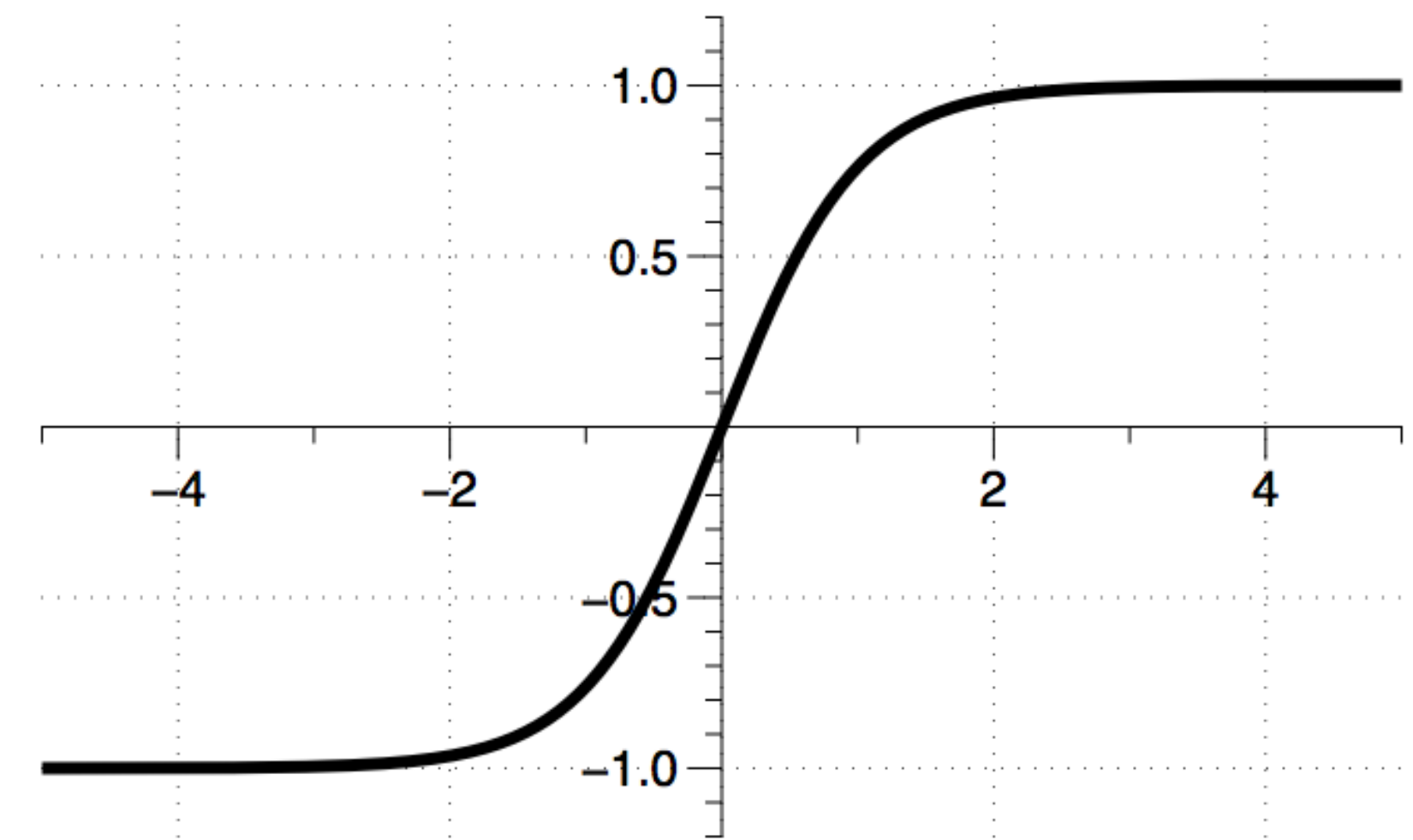
Sigmoide

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Tangente hiperbólica

$$a = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

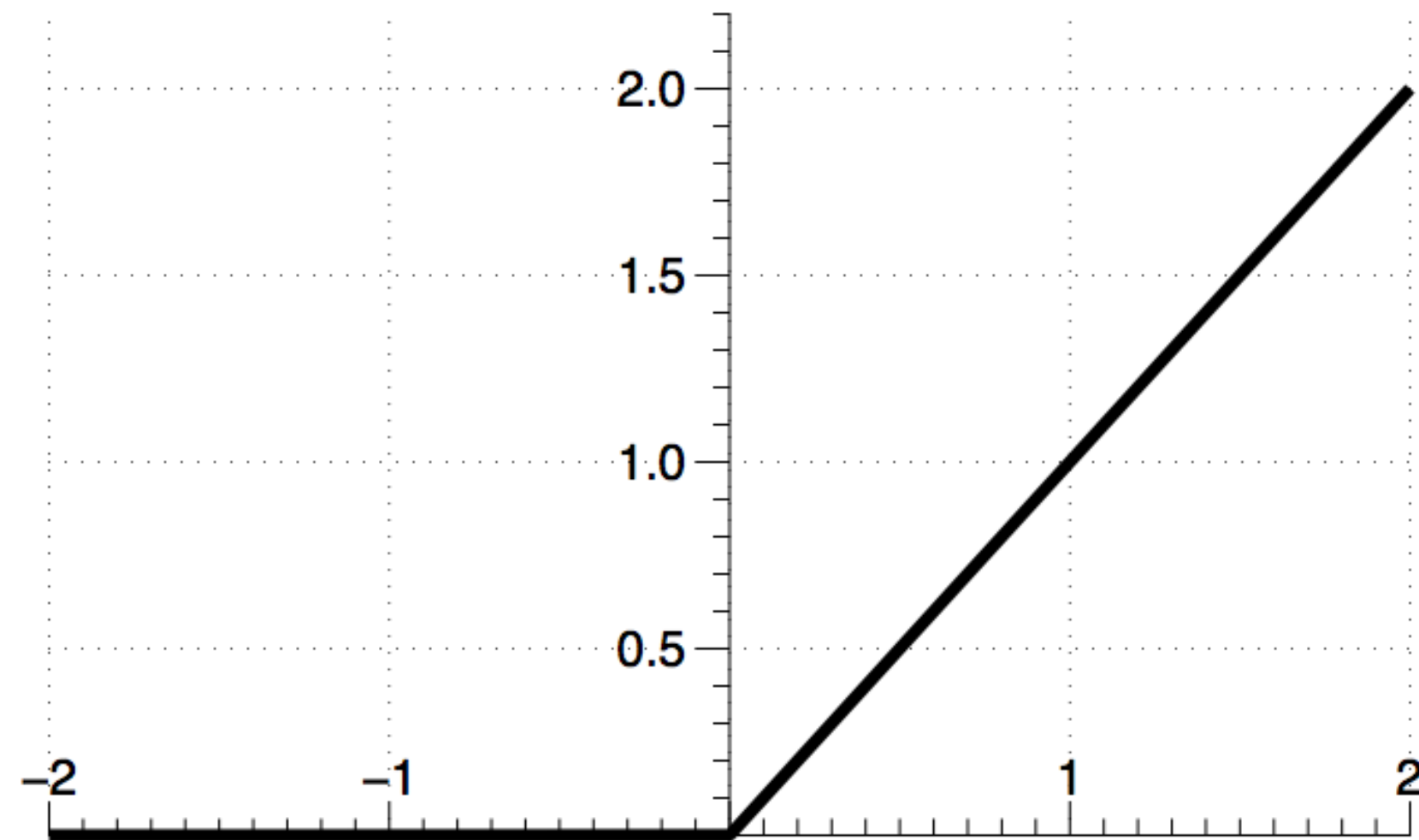


Arquitectura de una red neuronal

- Funciones de activación

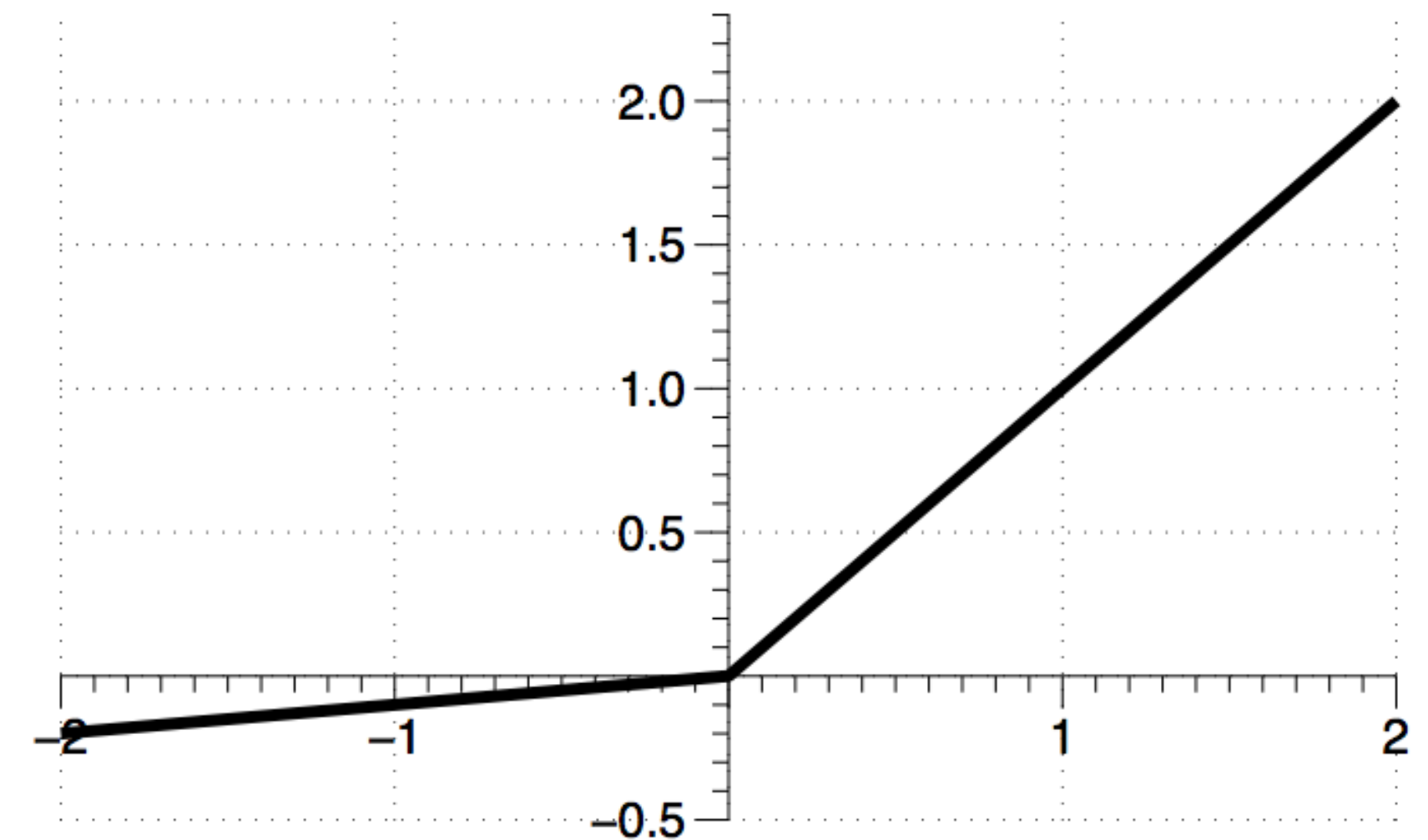
ReLU

$$a = \max(0, z)$$



Leaky ReLU

$$a = \max(0.01z, z)$$



Arquitectura de una red neuronal

- El tipo de problema también condiciona la arquitectura:

Regresión

$$y \in \mathbb{R}; D = \{(x, y), \dots\}$$

x1	x2	x3	y
-3.9	12.1	0.34	2.1
7.01	-2	-4.2	-3.21
-9.2	3.12	-1.98	-8.3
0.01	7.23	-1.21	1.67

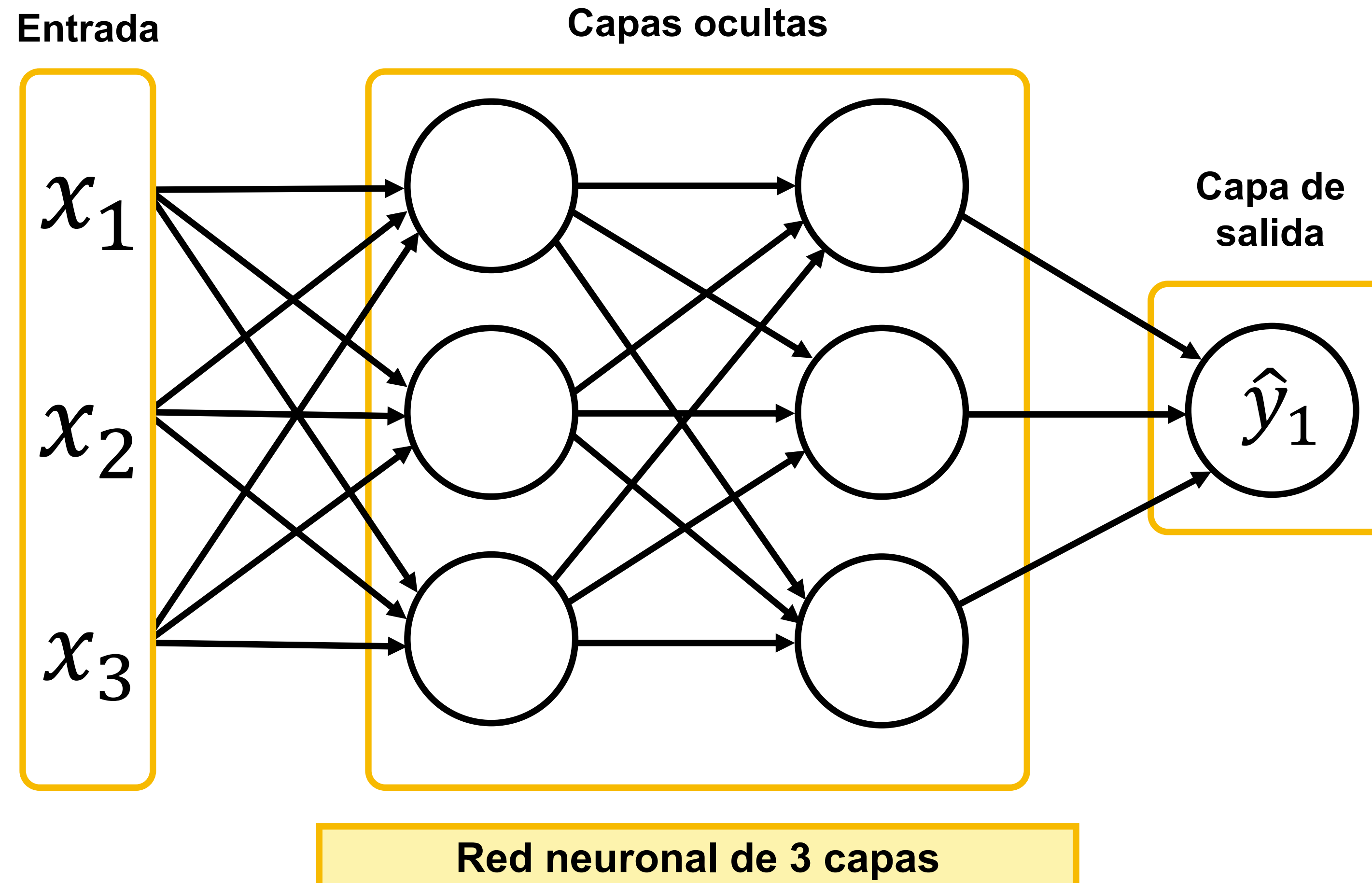
Clasificación Binaria

$$y \in \{0, 1\}; D = \{(x, y), \dots\}$$

x1	x2	x3	y
-3.9	12.1	0.34	1
7.01	-2	-4.2	0
-9.2	3.12	-1.98	0
0.01	7.23	-1.21	1

Arquitectura de una red neuronal

- El tipo de problema también condiciona la arquitectura:



Arquitectura de una red neuronal

- El tipo de problema también condiciona la arquitectura:

Clasificación multi-clase

$$y \in \{1, 2, \dots, n\}; D = \{(x, y), \dots\}$$

x1	x2	x3	y
-3.9	12.1	0.34	3
7.01	-2	-4.2	1
-9.2	3.12	-1.98	2
0.01	7.23	-1.21	1
8.3	-2	-1.3	4

codificación one-hot

x1	x2	x3	y1	y2	y3	y4
-3.9	12.1	0.34	0	0	1	0
7.01	-2	-4.2	1	0	0	0
-9.2	3.12	-1.98	0	1	0	0
0.01	7.23	-1.21	1	0	0	0
8.3	-2	1.3	0	0	0	1

Clasificación multi-etiqueta

$$y \in \{0, 1\}^n; D = \{(x, y), \dots\}$$

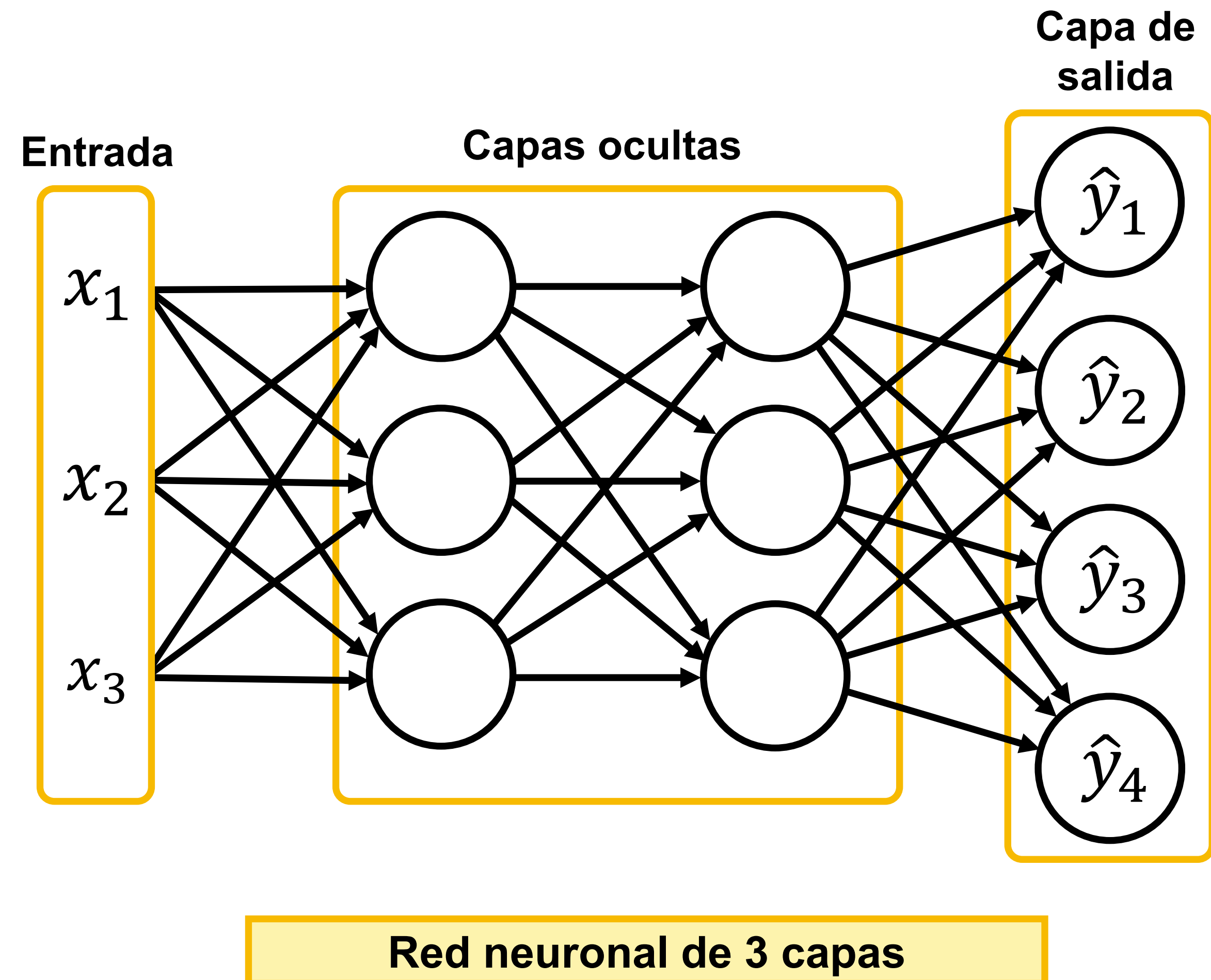
x1	x2	x3	y
-3.9	12.1	0.34	1,3
7.01	-2	-4.2	4
-9.2	3.12	-1.98	1,2,3
0.01	7.23	-1.21	1
8.3	-2	-1.3	3,4

codificación k-hot

x1	x2	x3	y1	y2	y3	y4
-3.9	12.1	0.34	1	0	1	0
7.01	-2	-4.2	0	0	0	1
-9.2	3.12	-1.98	1	1	1	0
0.01	7.23	-1.21	1	0	0	0
8.3	-2	1.3	0	0	1	1

Arquitectura de una red neuronal

- El tipo de problema también condiciona la arquitectura:



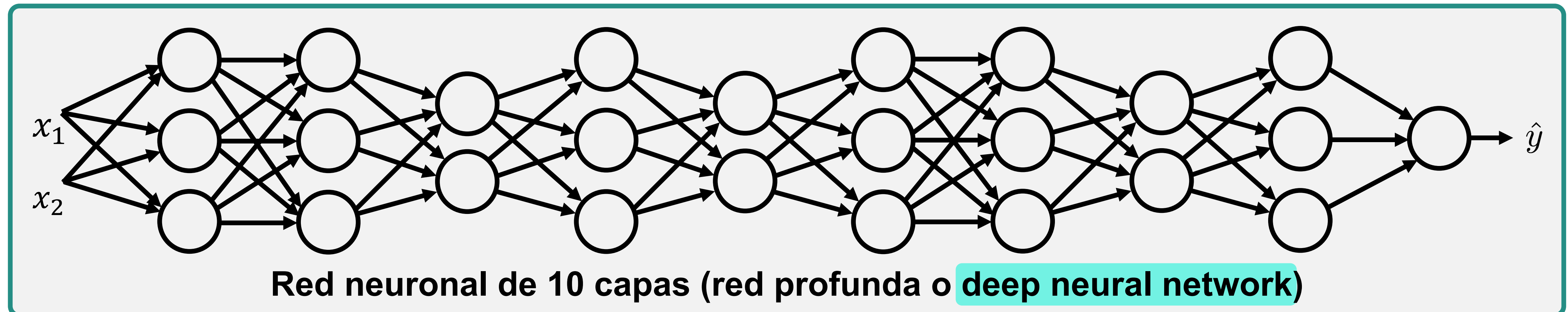
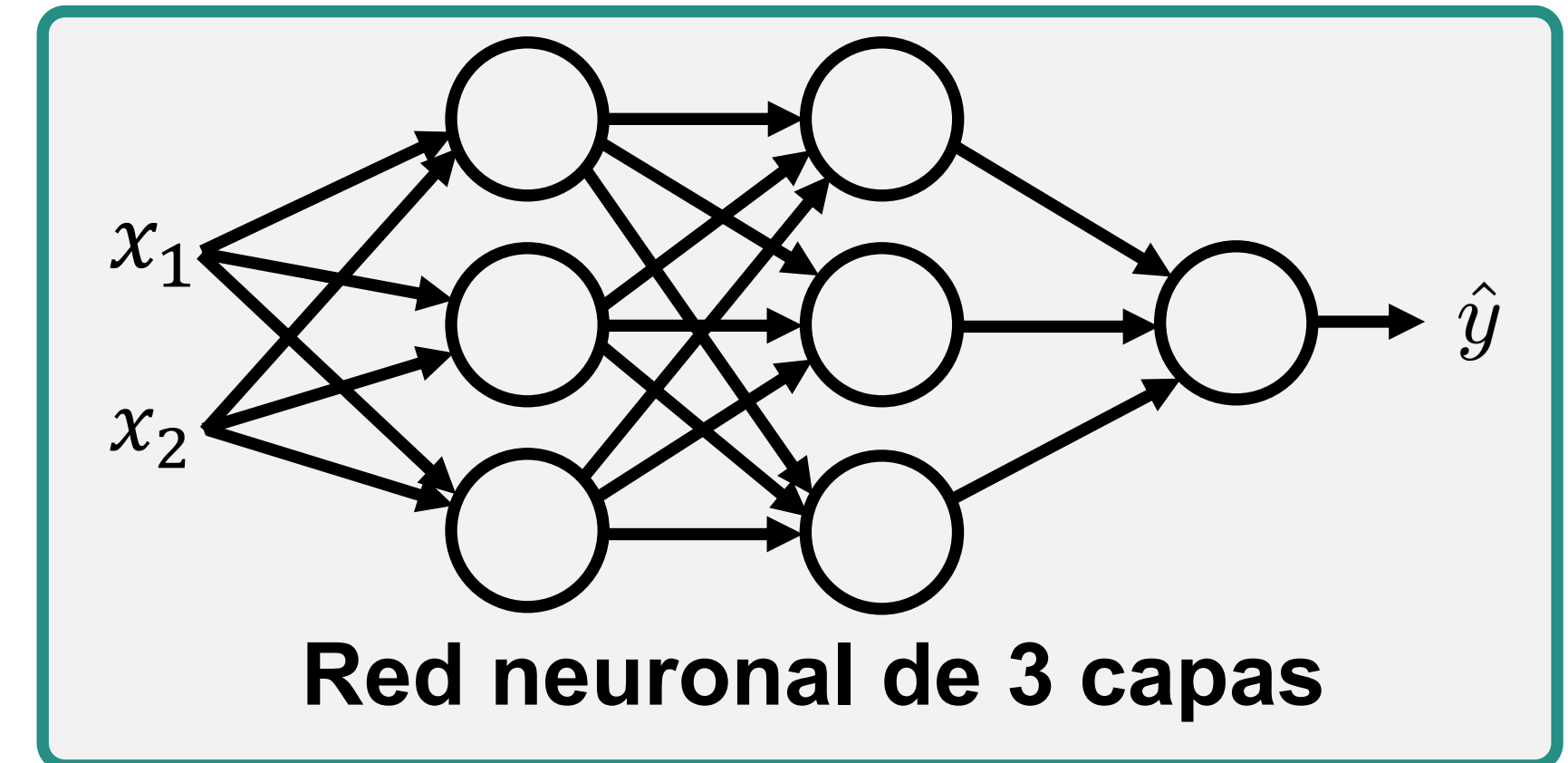
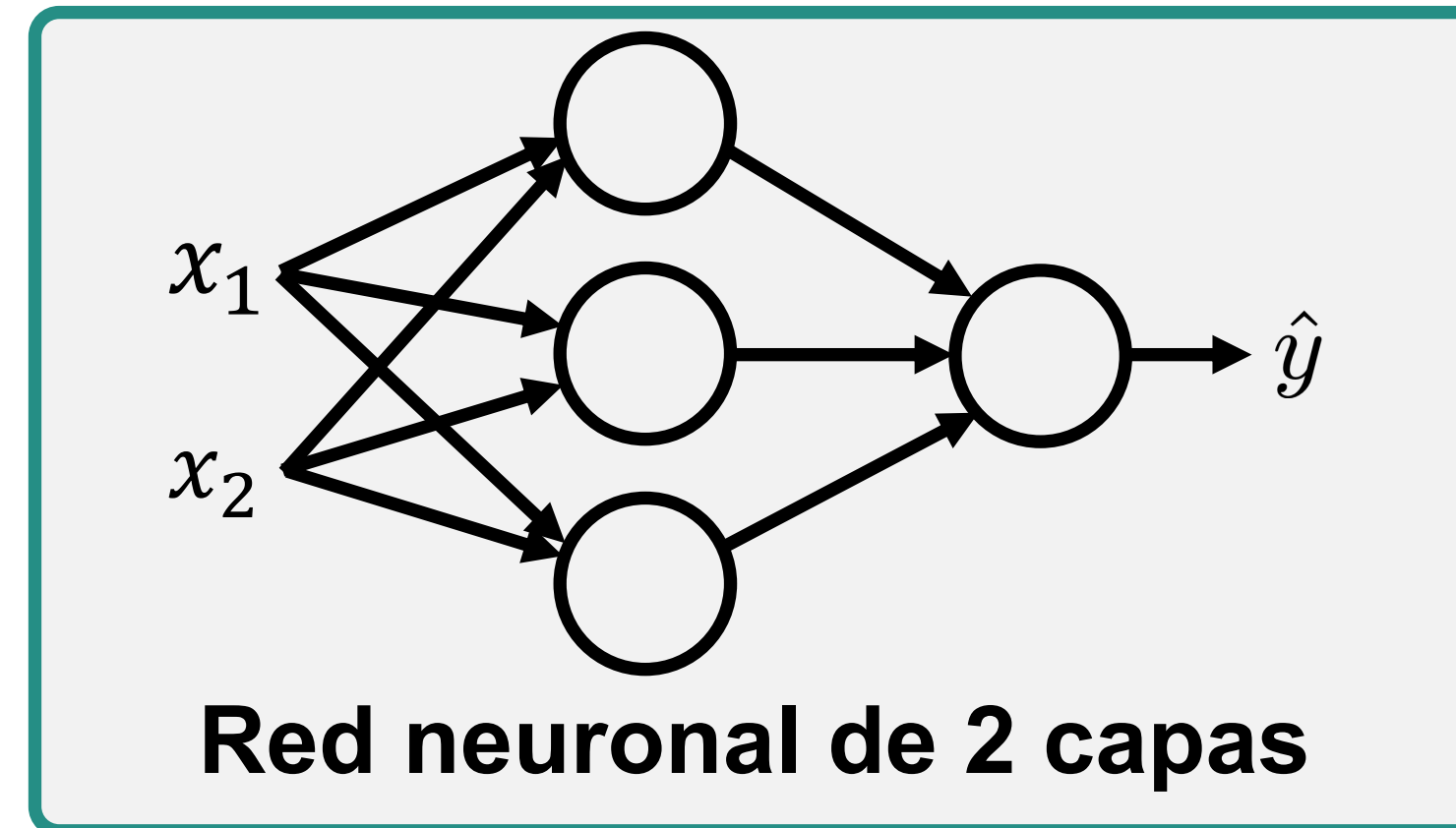
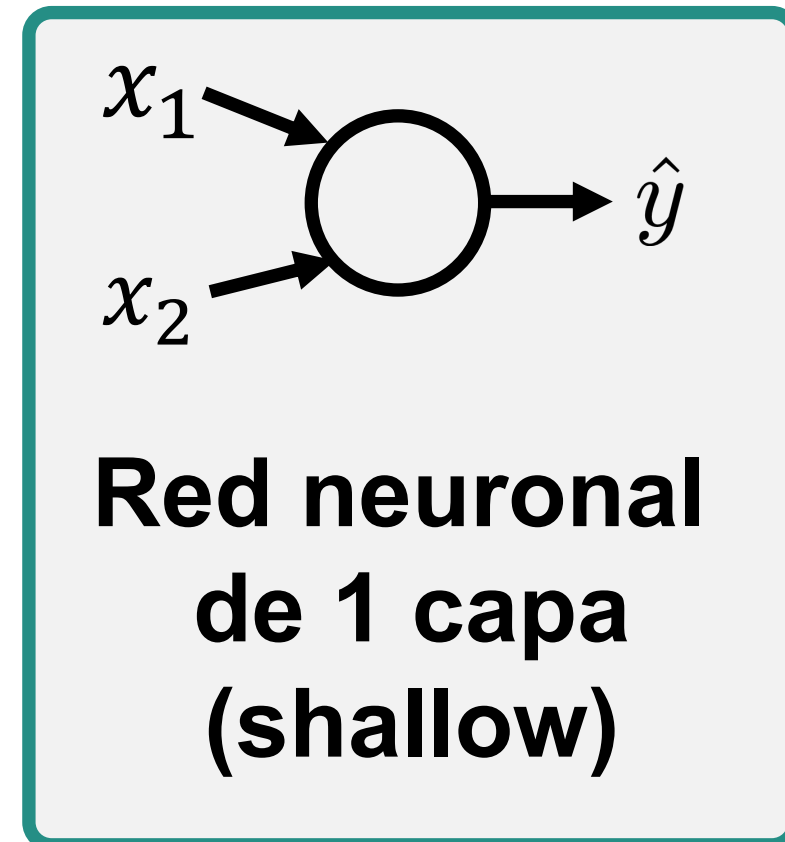
Arquitectura de una red neuronal

- El tipo de problema también condiciona la arquitectura:

Tipo de Problema	Función de activación de la última capa
<i>Regresión</i>	Ninguna, o ReLU para asegurar no negatividad
<i>Clasificación Binaria</i>	Sigmoide
<i>Clasificación Multiclase</i>	Softmax
<i>Clasificación Multietiqueta</i>	Sigmoide

Arquitectura de una red neuronal

- ¿Cuándo consideramos profunda una red neuronal?



Contenido

1. Introducción

2. Arquitectura

3. Entrenamiento

4. Técnicas

- Evitar el sobreajuste
- Acelerar el entrenamiento

5. Resumen

Entrenamiento

- **Objetivo**

- Obtener parámetros θ del modelo que lo hagan óptimo para resolver su tarea predictiva.

- **Parámetros:**

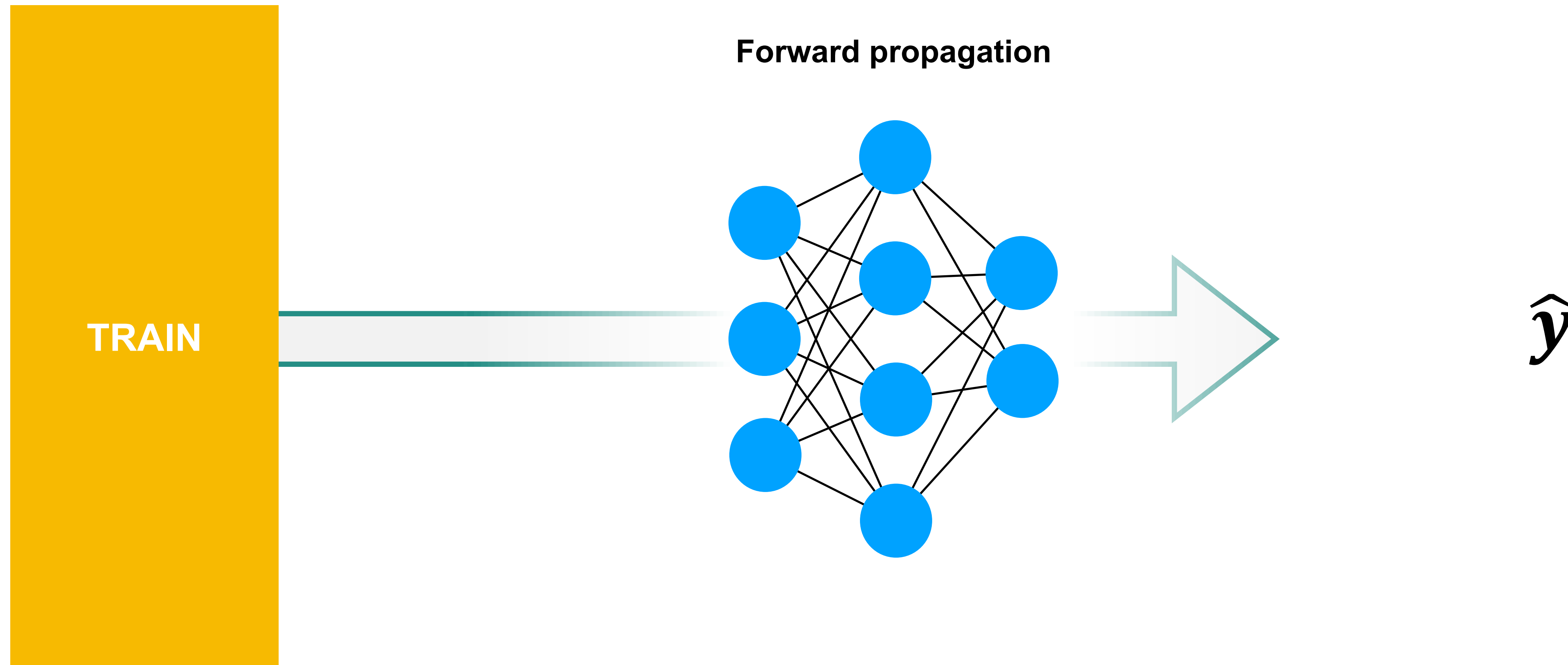
- Pesos de las conexiones entre neuronas y términos independientes.

- **Hiperparámetros:**

- **Learning rate:** Tasa (ganancia) de aprendizaje.
- **Epochs:** Número de iteraciones en el entrenamiento (epochs).
- **Arquitectura:** Número de neuronas, capas, funciones de activación...
 - Afecta al número de parámetros del modelo.
 - A mayor complejidad del modelo, mayor riesgo de sobreajuste.
- **Etc.**

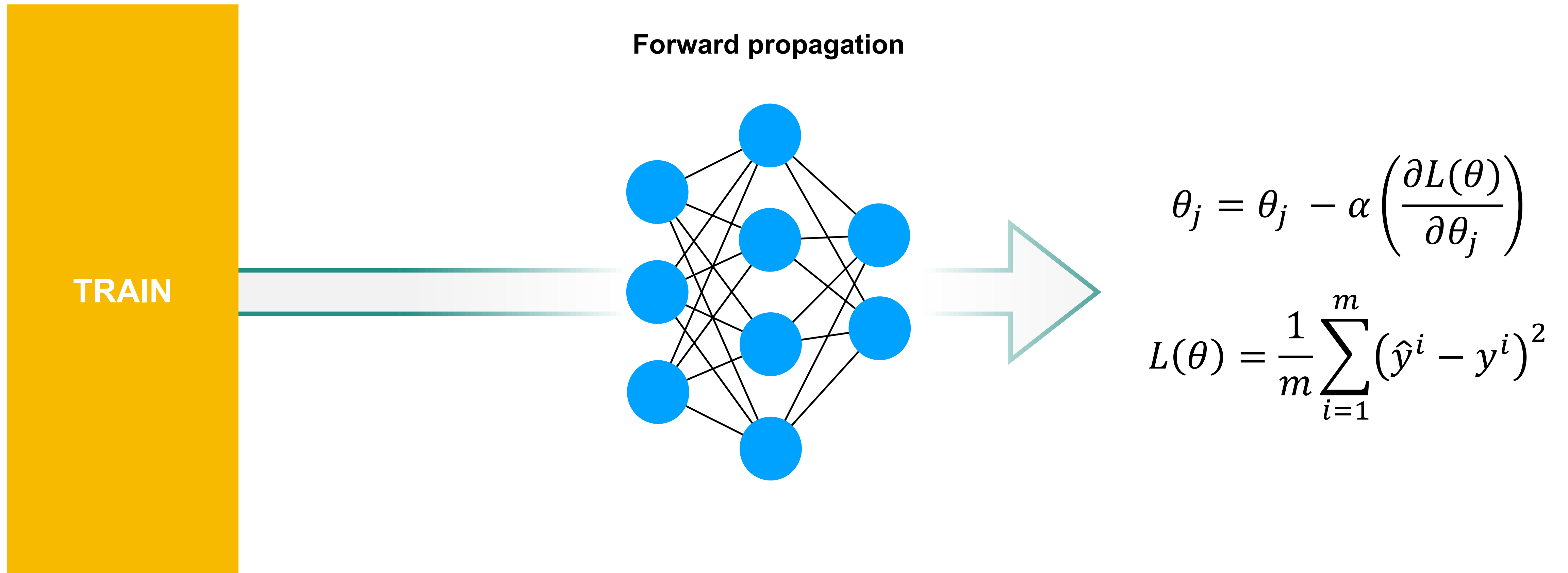
Entrenamiento

- Gradient descent



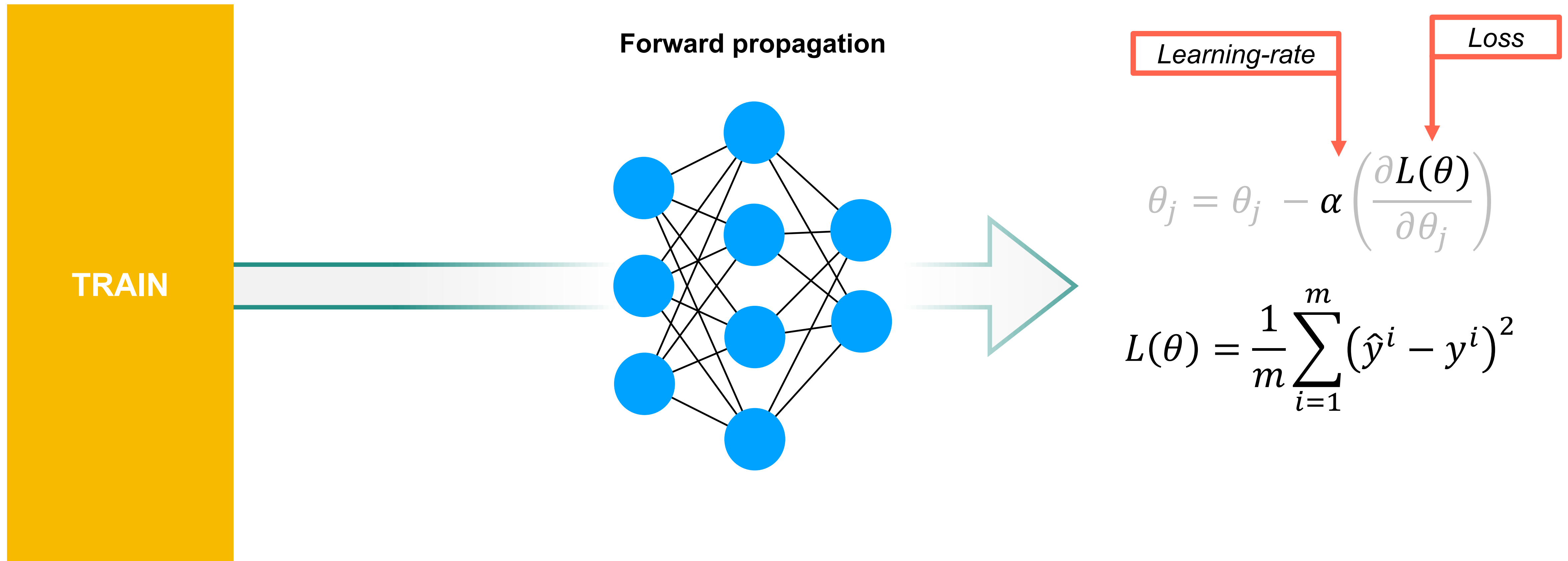
Entrenamiento

- Gradient descent



Entrenamiento

- Gradient descent



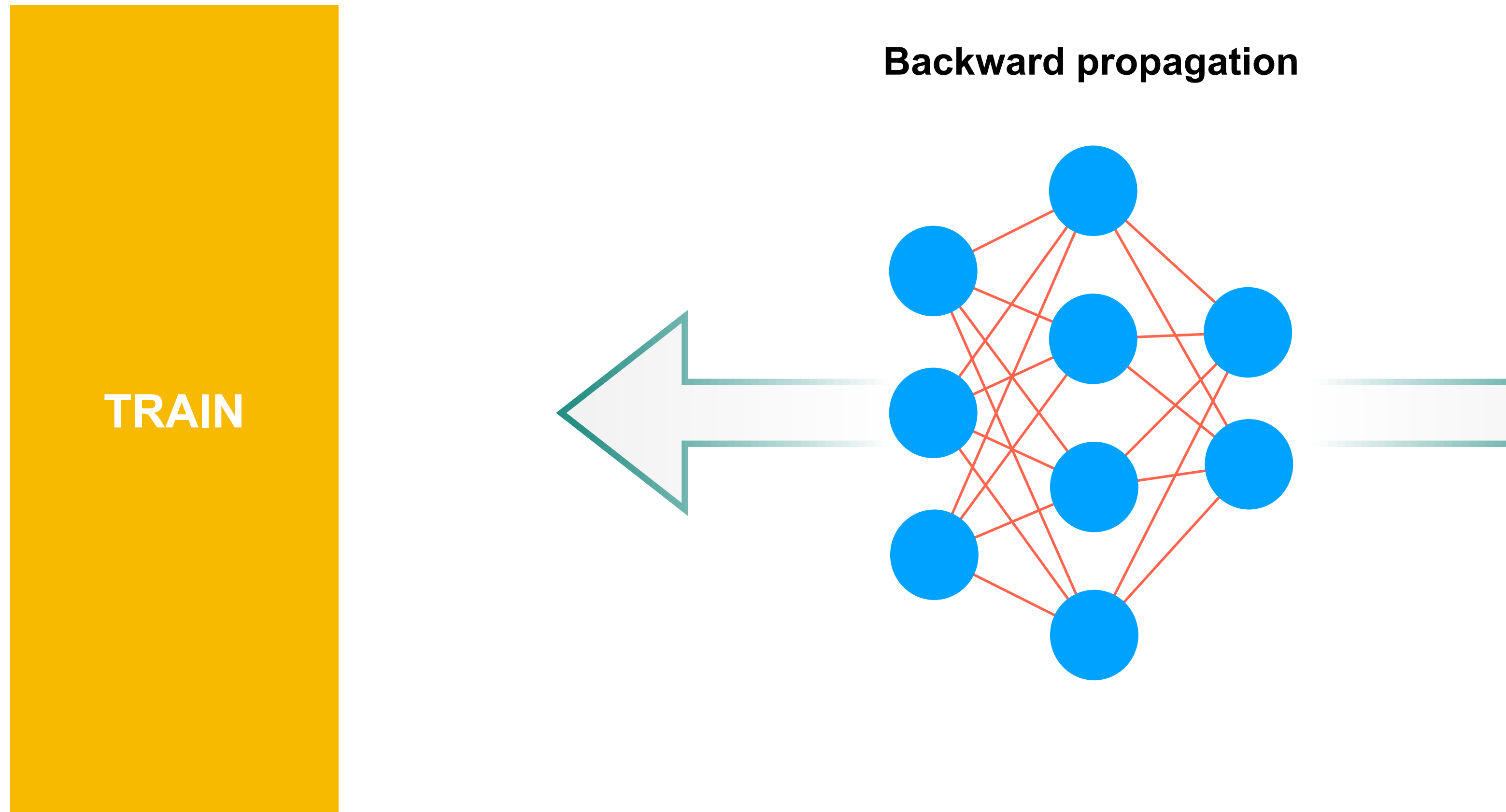
Entrenamiento

- **Losses y funciones de activación (última capa):**

Tipo de Problema	Función de activación de la última capa	Función de pérdida
<i>Regresión</i>	Ninguna, o ReLU para asegurar no negatividad	Mean Squared Error (MSE)
<i>Clasificación Binaria</i>	Sigmoide	Binary Cross-Entropy
<i>Clasificación Multiclase</i>	Softmax	Categorical Cross-Entropy
<i>Clasificación Multietiqueta</i>	Sigmoide	Binary Cross-Entropy

Entrenamiento

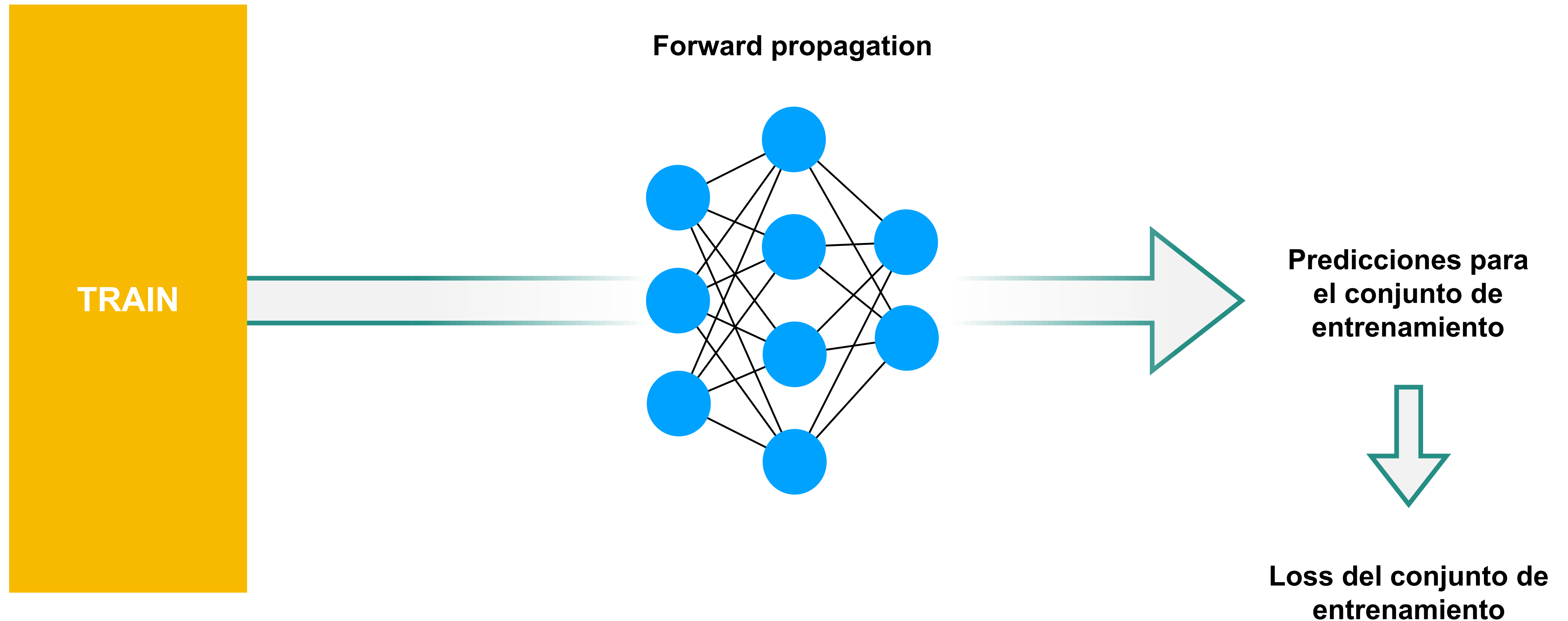
- Gradient descent



$$\theta_j = \theta_j - \alpha \left(\frac{\partial L(\theta)}{\partial \theta_j} \right)$$
$$L(\theta) = \frac{1}{m} \sum_{i=1}^m (\hat{y}^i - y^i)^2$$

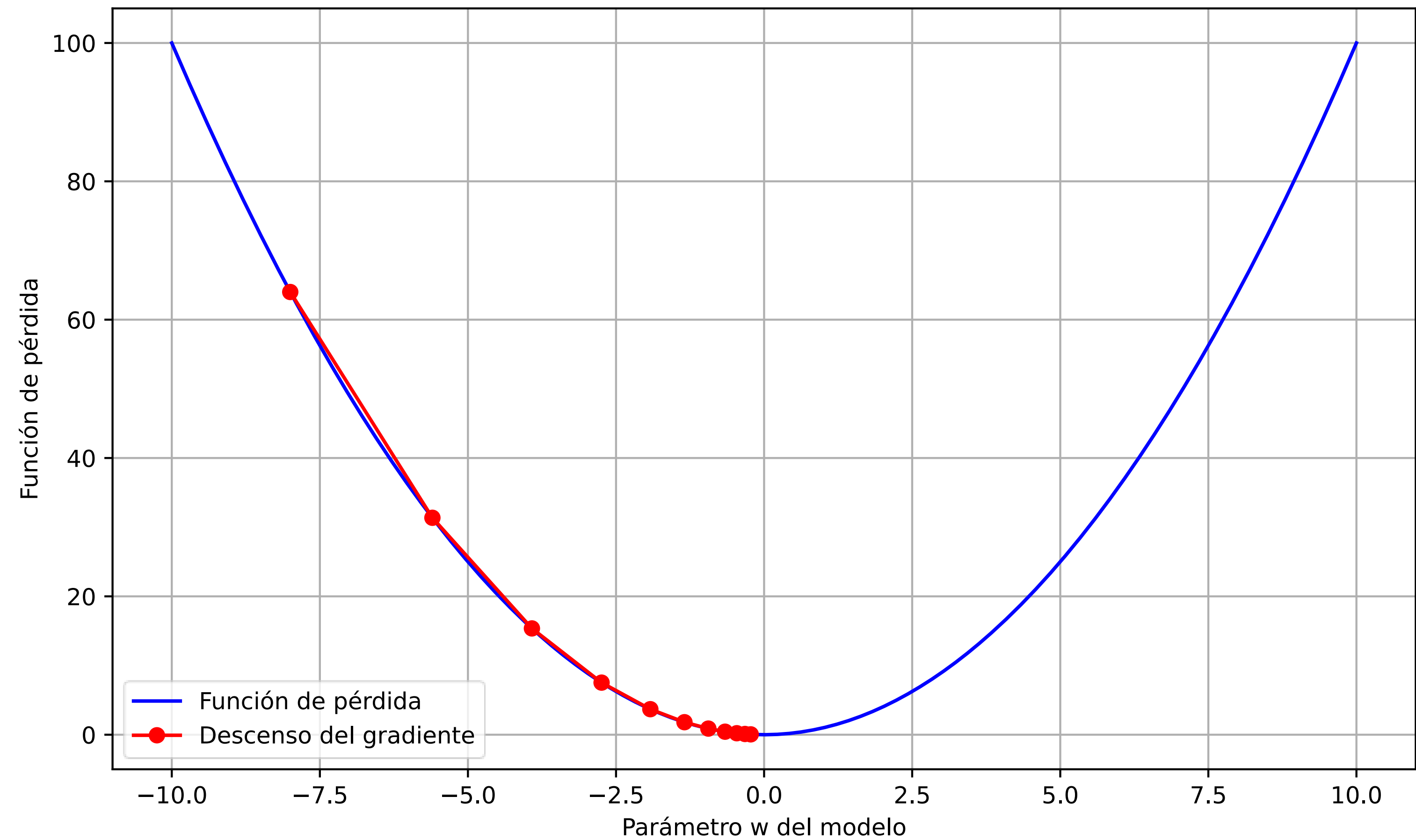
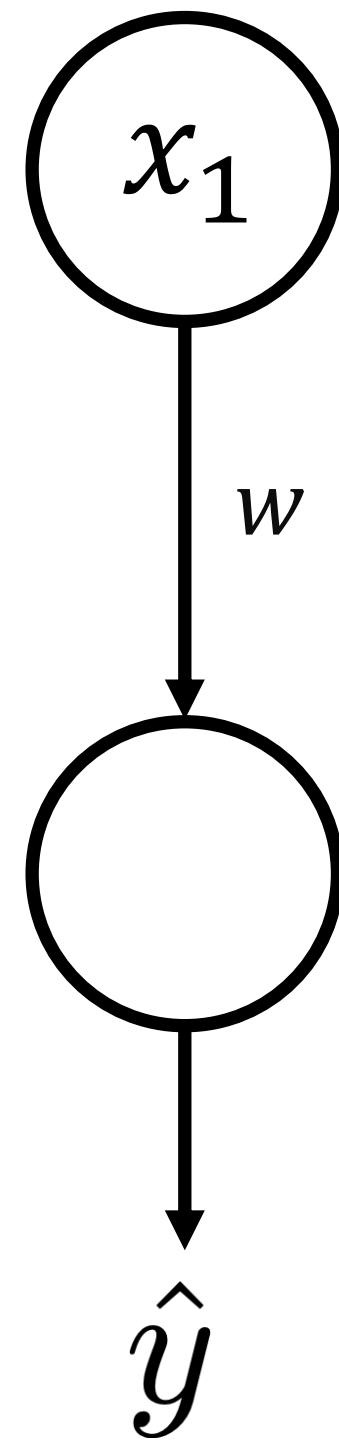
Entrenamiento

- Gradient descent



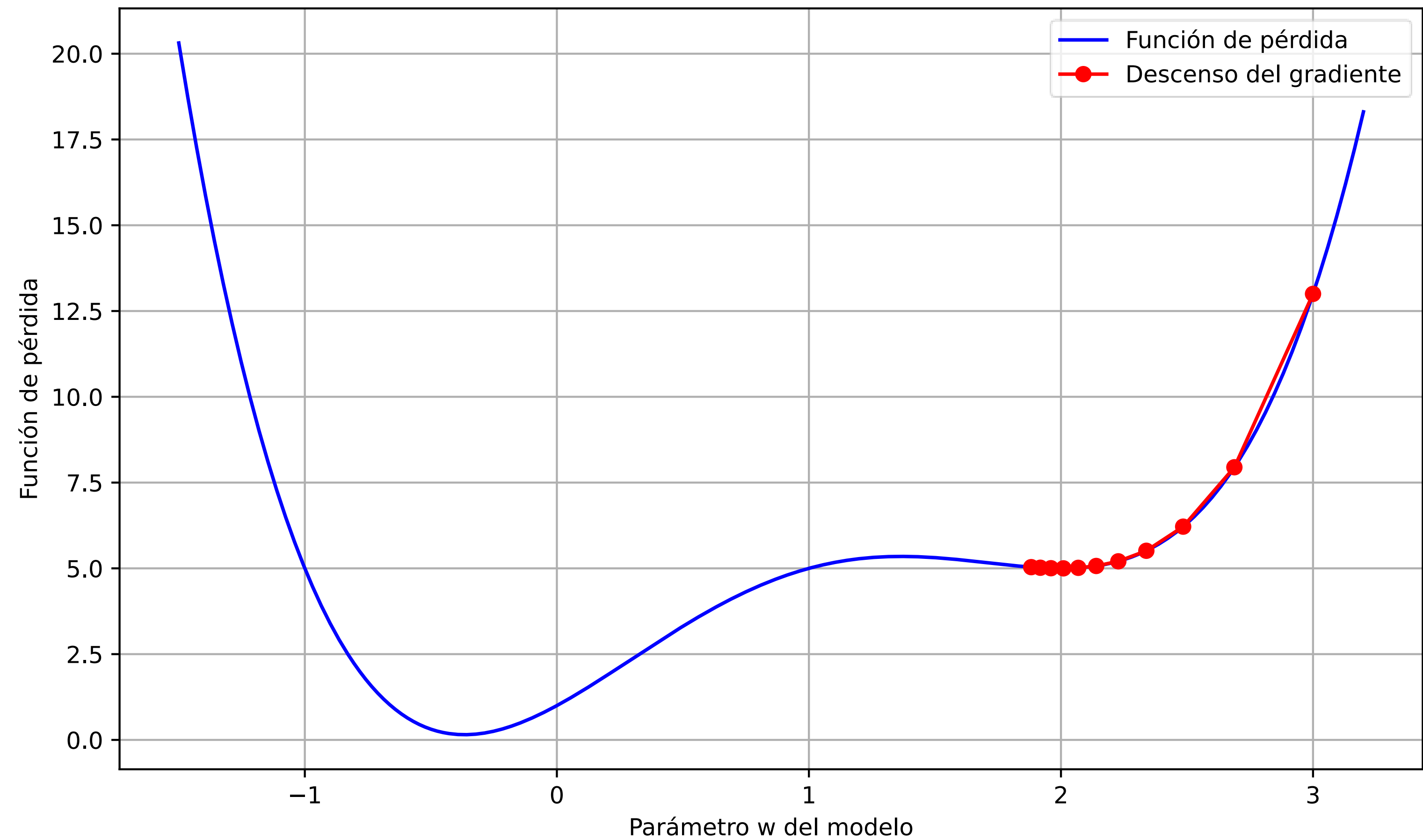
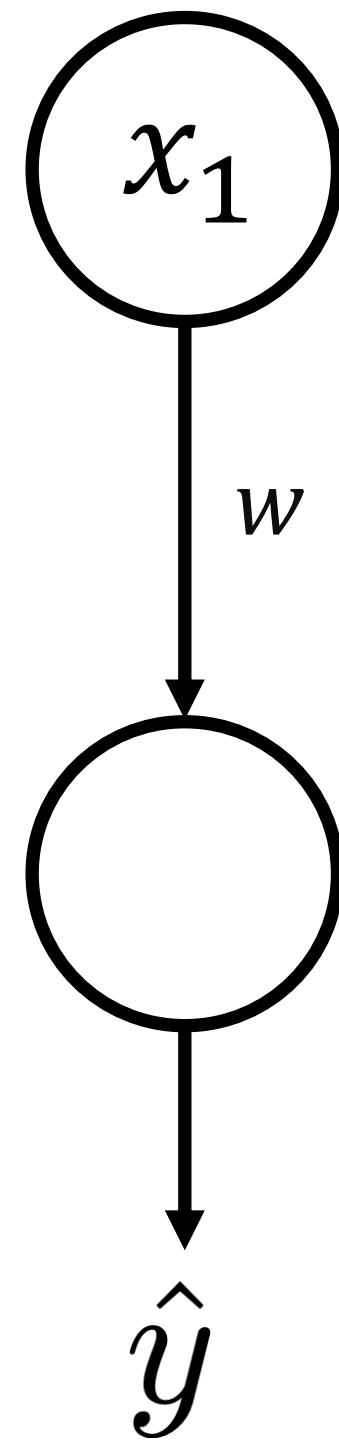
Entrenamiento

- Gradient descent: Loss convexa



Entrenamiento

- Gradient descent: Loss no convexa



Entrenamiento

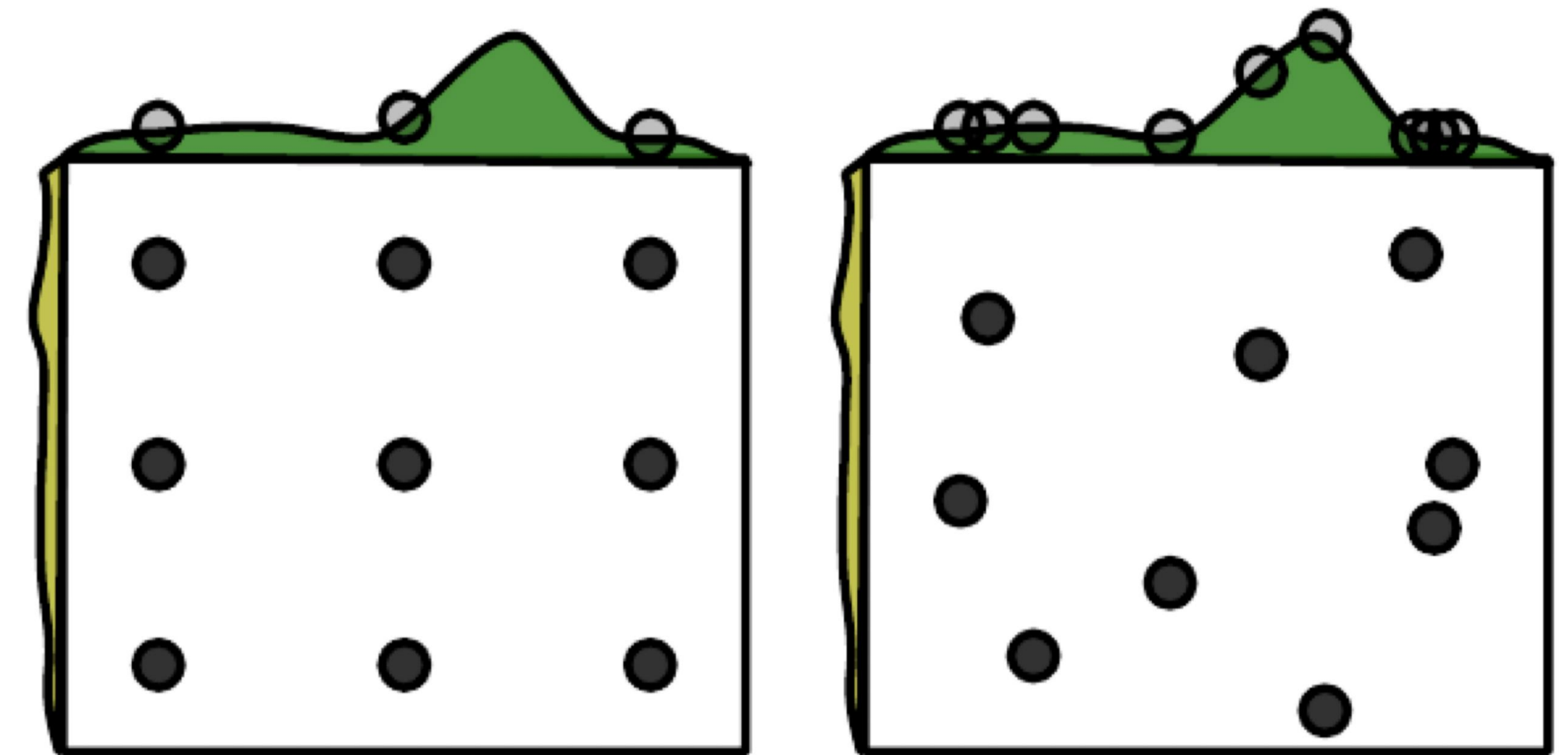
▪ Búsqueda de hiperparámetros

- Grid Search:

- Se escoge un subconjunto de valores para cada hiperparámetro y se prueba con todas las combinaciones posibles.

- Random Search

- Se decide el número de pruebas a efectuar y en cada una de ellas se escoge aleatoriamente un valor para cada hiperparámetro.



Grid search

Random search

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281-305.

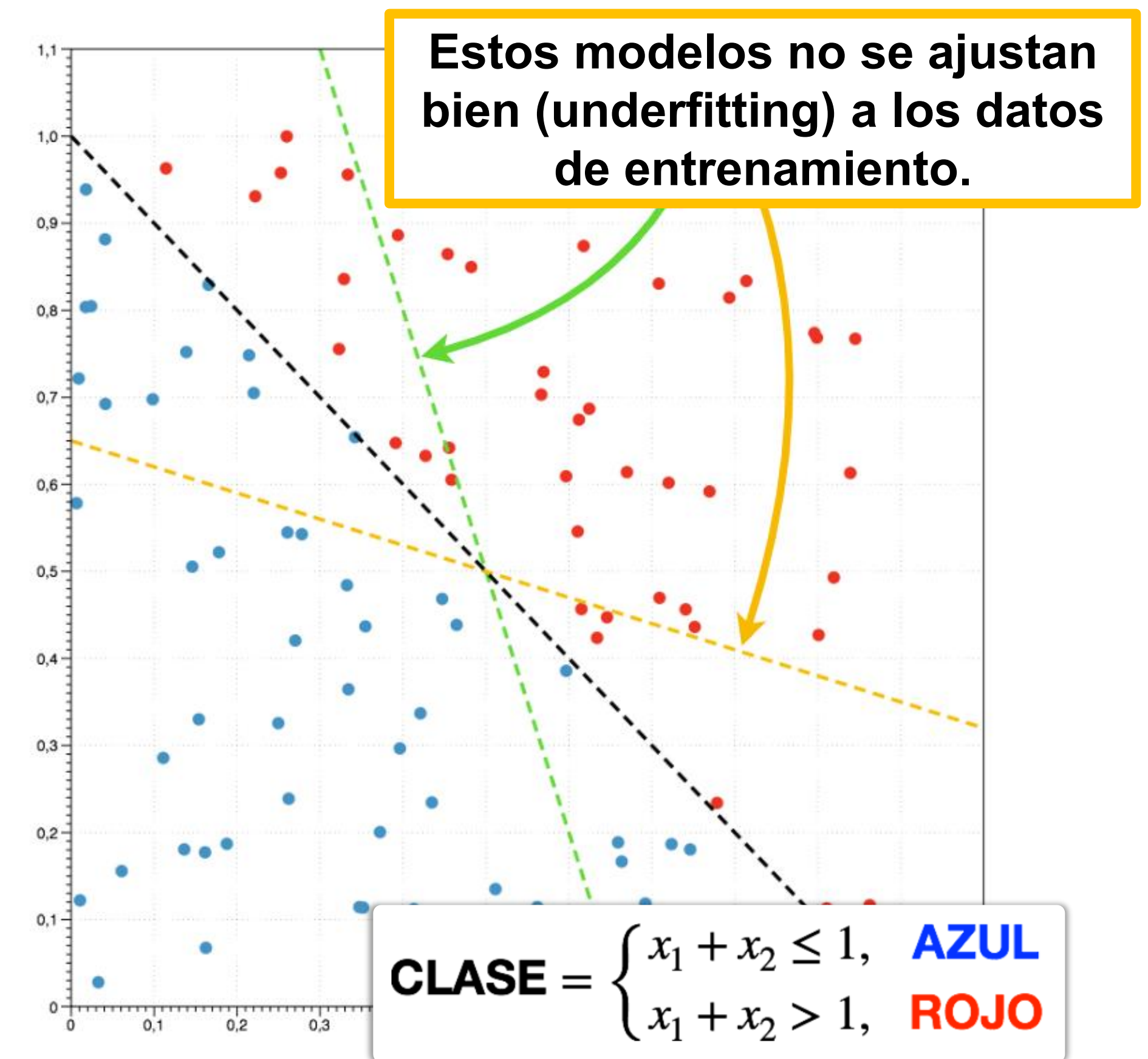
Entrenamiento

- **¿Cómo sería un buen modelo?**
 - Aquel que tenga un **equilibrio** entre:
 - **Underfitting**
 - No es capaz de aprender de los datos de entrenamiento.
 - **Overfitting**
 - Solo funciona con los datos de entrenamiento.

Entrenamiento

■ Underfitting

- El modelo no es capaz de aprender.
- El error en el conjunto de entrenamiento es claramente superior al que cabría esperar
- Posibles soluciones:
 - Más iteraciones.
 - Más complejidad del modelo:
 - Más neuronas/capas.
 - Otra arquitectura.
 - Etc

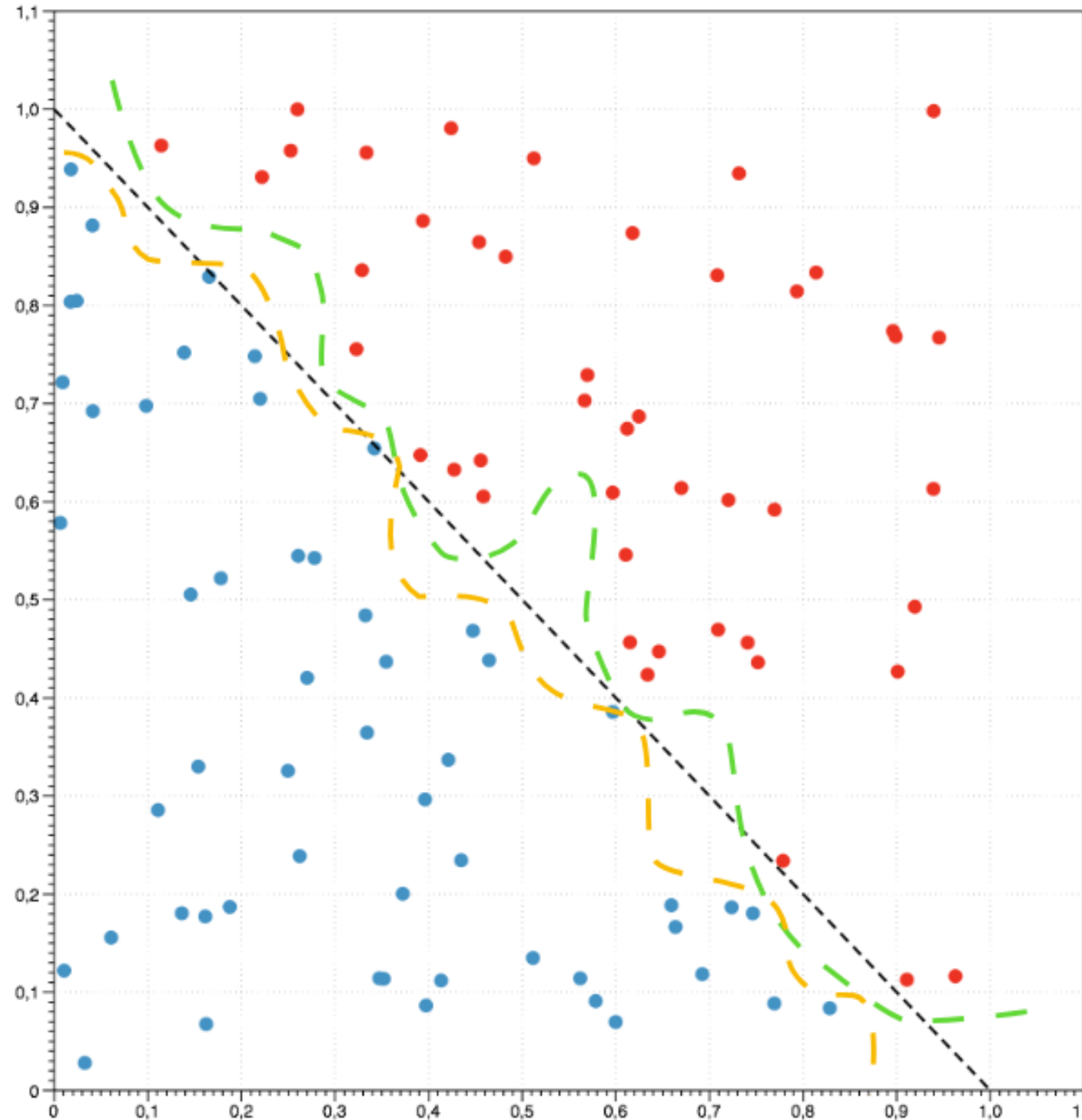


Entrenamiento

■ Overfitting

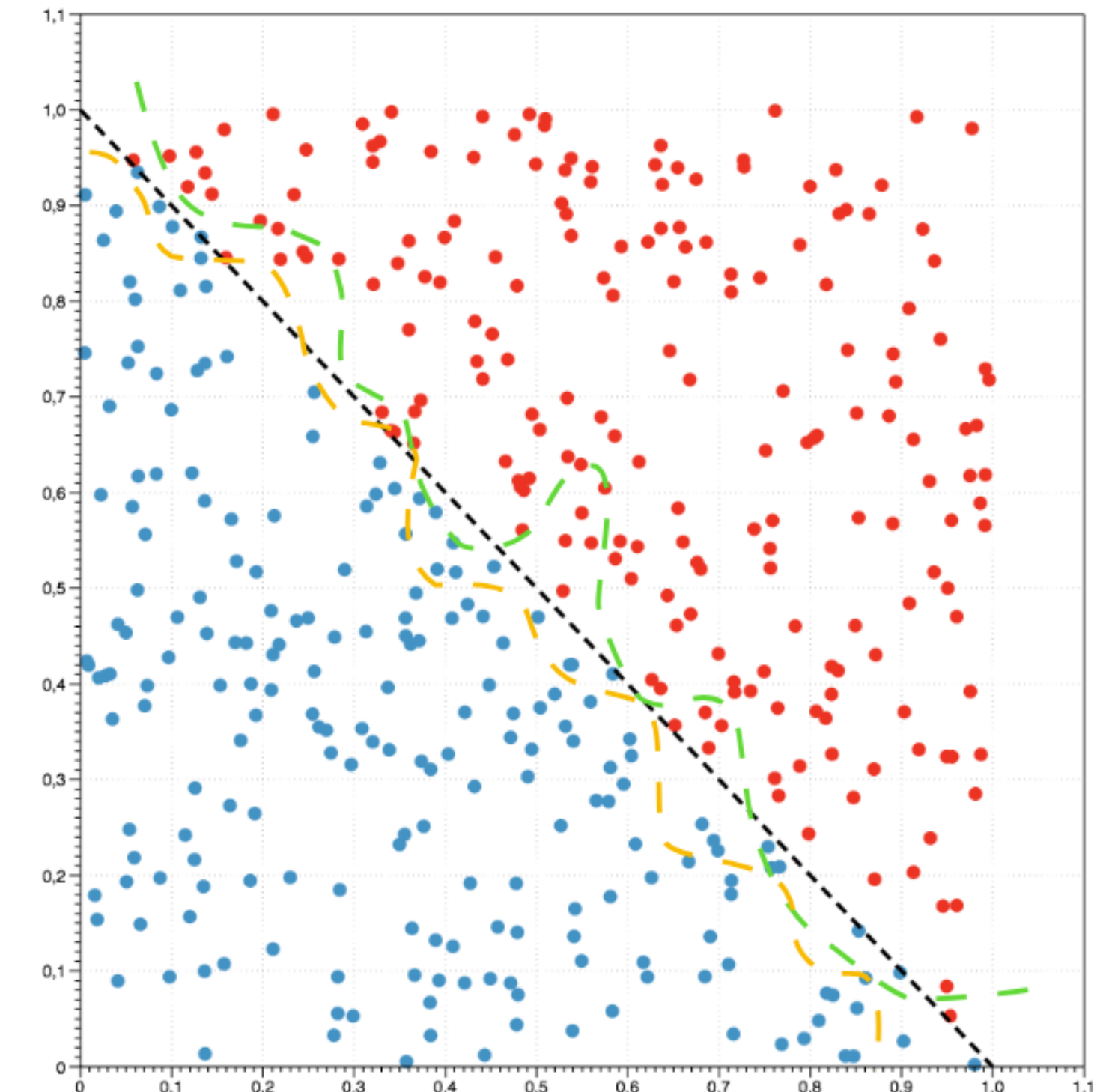
- El modelo se **sobreajusta** a los datos de entrenamiento.

Datos de entrenamiento



$$\text{CLASE} = \begin{cases} x_1 + x_2 \leq 1, & \text{AZUL} \\ x_1 + x_2 > 1, & \text{ROJO} \end{cases}$$

Datos de test



Contenido

1. Introducción

2. Arquitectura

3. Entrenamiento

4. Técnicas

- Evitar el sobreajuste
- Acelerar el entrenamiento

5. Resumen

Evitar el sobreajuste

- **Técnicas:**

- 1. DropOut**

- Eliminación aleatoria de pesos/unidades.

- 2. Data Augmentation**

- Aumentar el conjunto de entrenamiento.

- 3. Early stopping**

- Parada temprana.

Evitar el sobreajuste

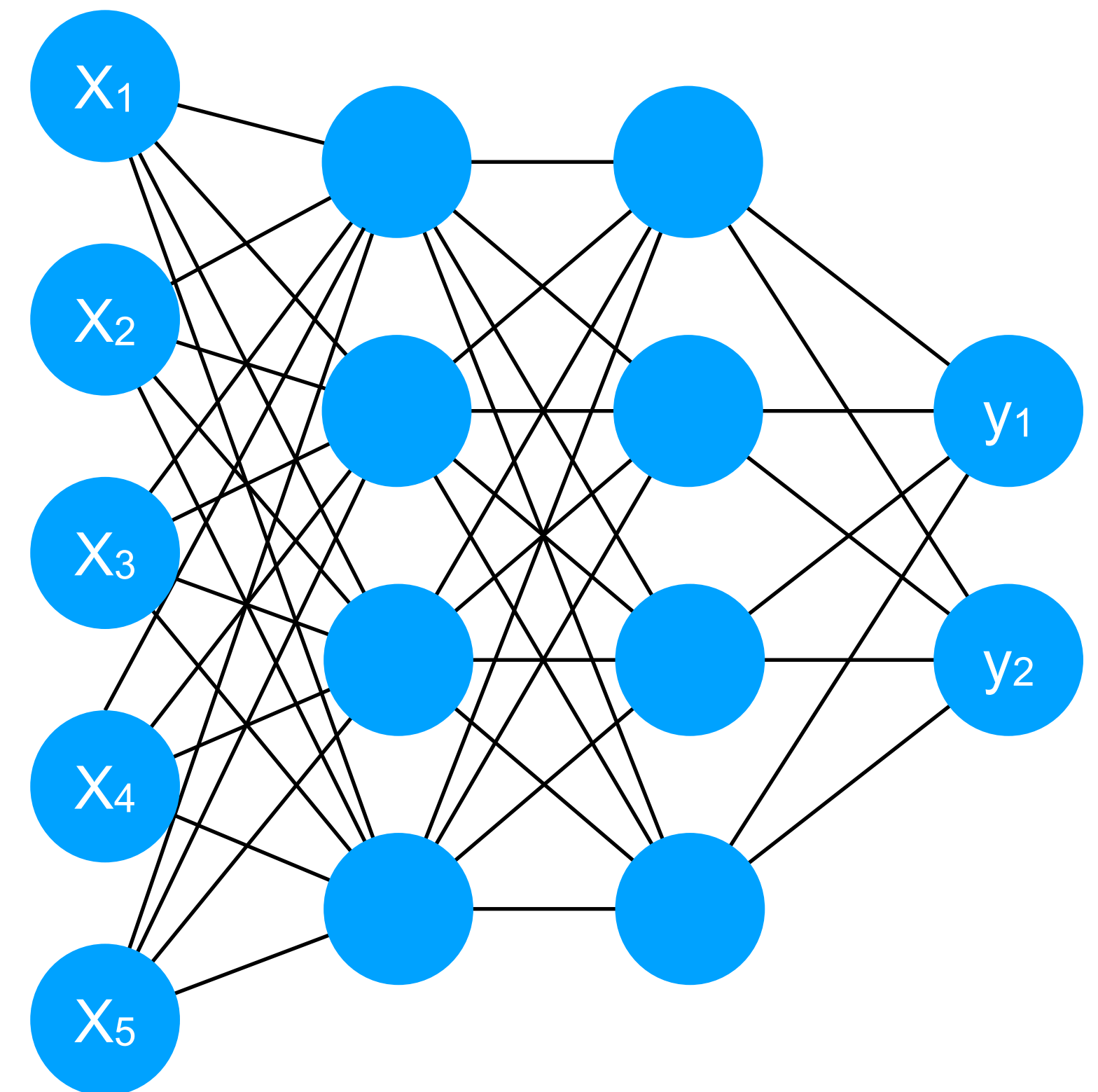
■ 1 - DropOut

- **Funcionamiento:**

- Deshabilitar un porcentaje de unidades, elegidas aleatoriamente, en cada iteración del entrenamiento.

- **Detalles:**

- Puede usarse en la capa de entrada.
- Distintas capas pueden tener distintos valores.
- No se utiliza en la capa de salida.

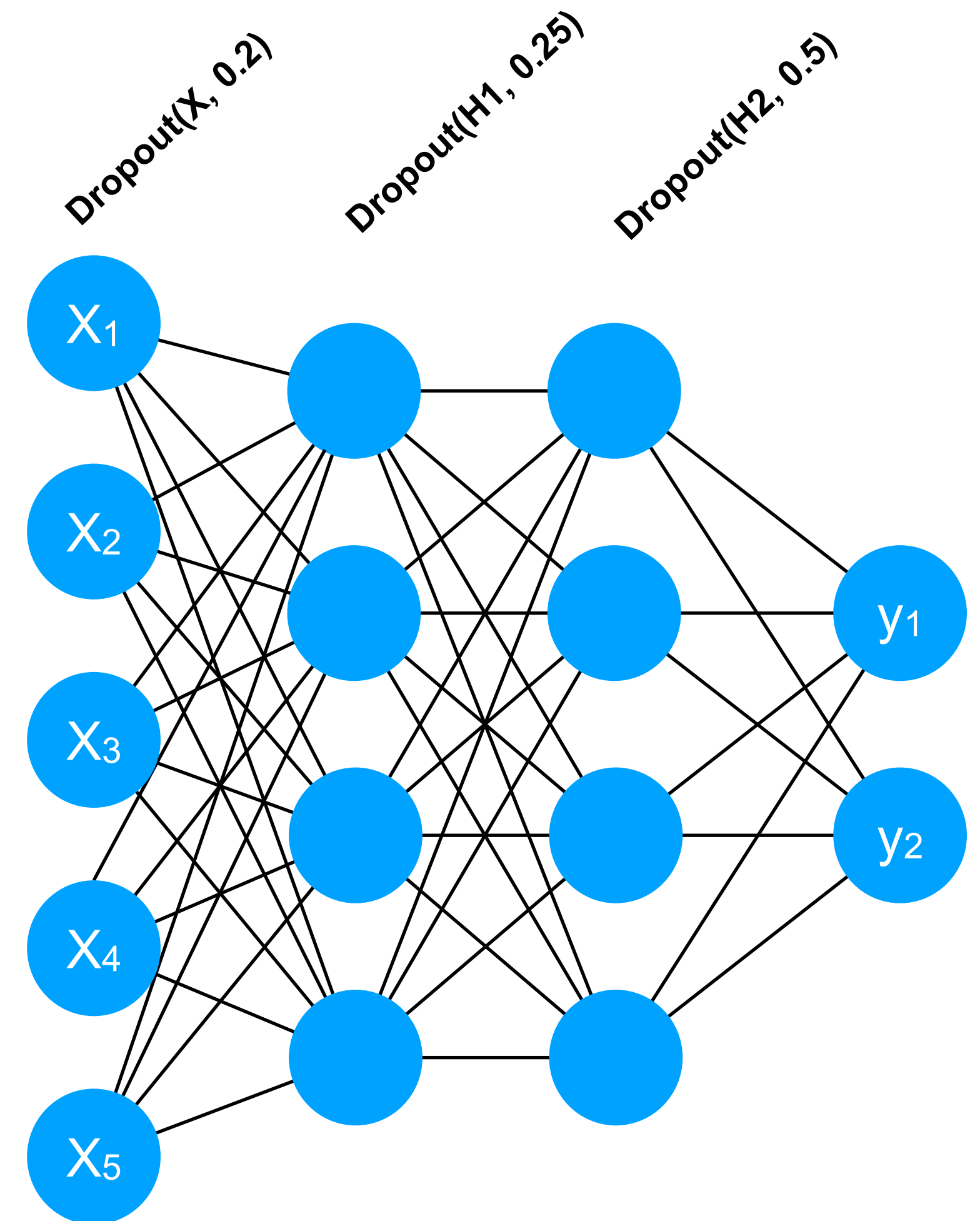


Evitar el sobreajuste

■ 1 - DropOut

• Ejemplo:

- Capa de entrada: 20%
- Capa oculta 1: 25%
- Capa oculta 2: 50%

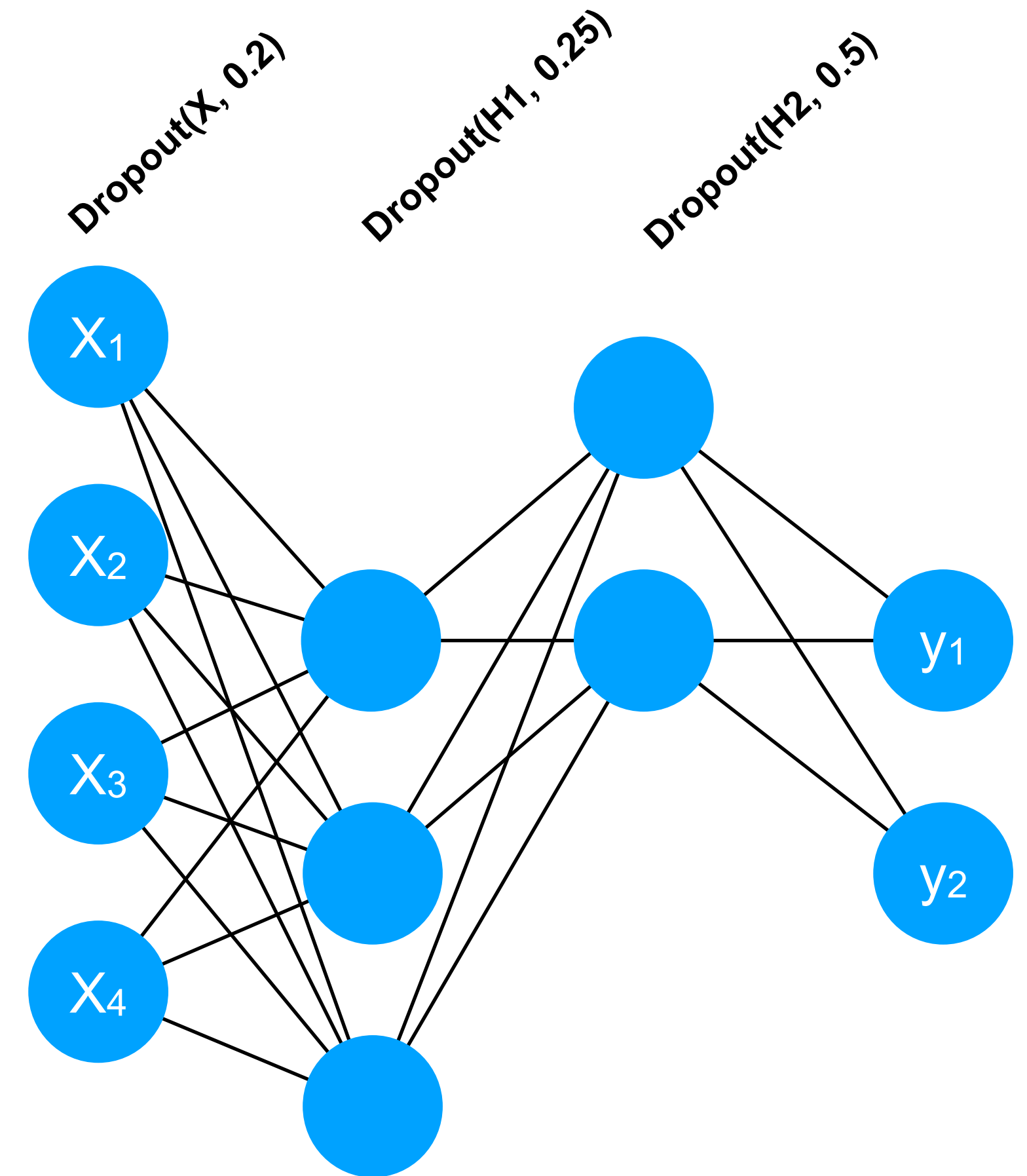


Evitar el sobreajuste

■ 1 - DropOut

• Ejemplo:

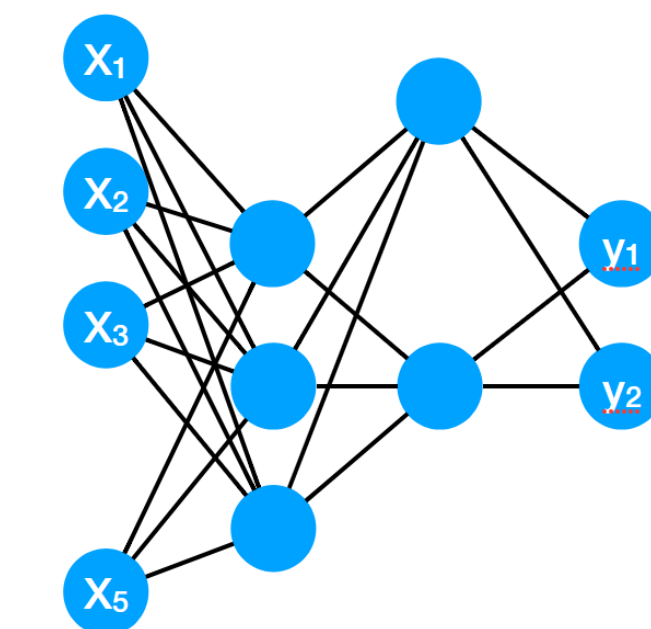
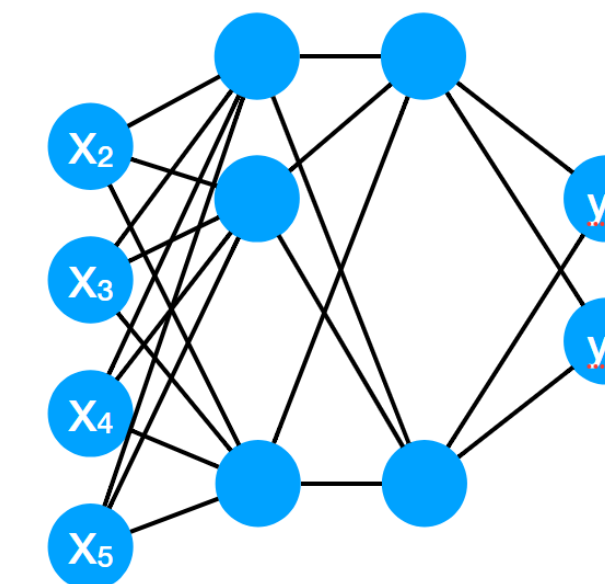
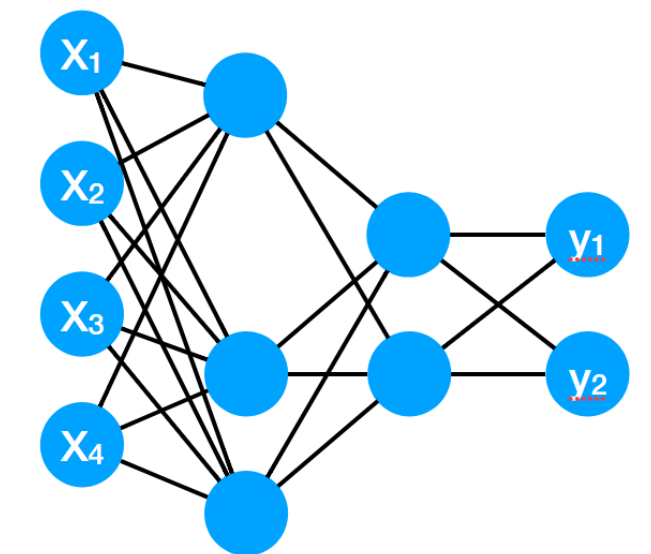
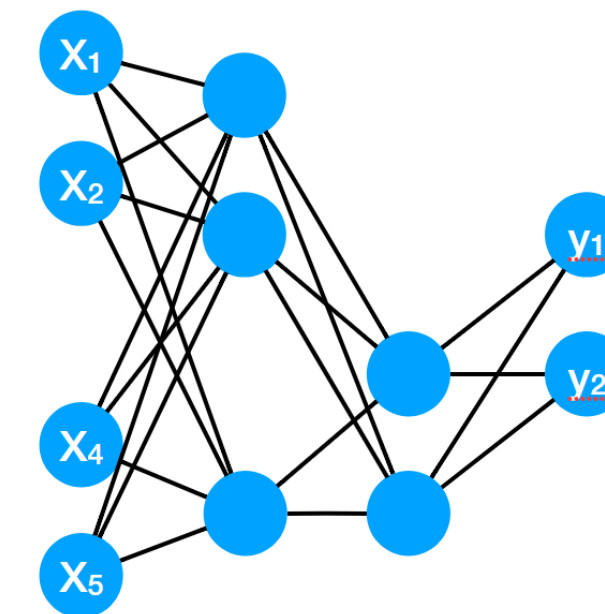
- Capa de entrada: 20%
- Capa oculta 1: 25%
- Capa oculta 2: 50%



Evitar el sobreajuste

■ 1 - DropOut

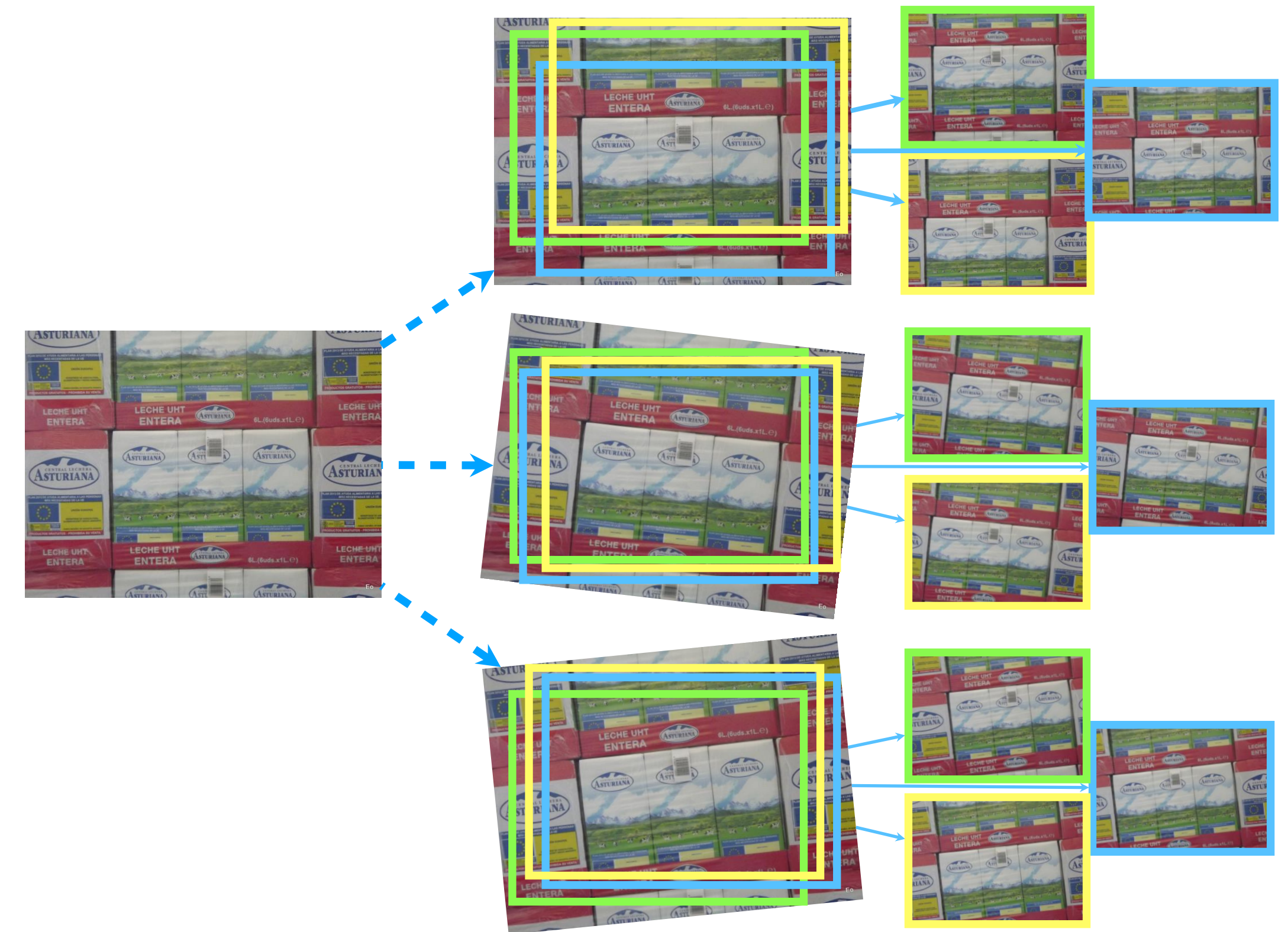
- Similar a entrenar un *ensemble* de modelos.
- ¿Por qué actúa como regularizador?
 - En cada iteración se trabaja con una red más pequeña que la original.
 - Los modelos más sencillos tienen más dificultades para sobreajustarse.



Evitar el sobreajuste

■ 2 - Data Augmentation

- Incrementar el número de ejemplos de entrenamiento de forma “artificial”.
- Muy usado en problemas de vision por computador.



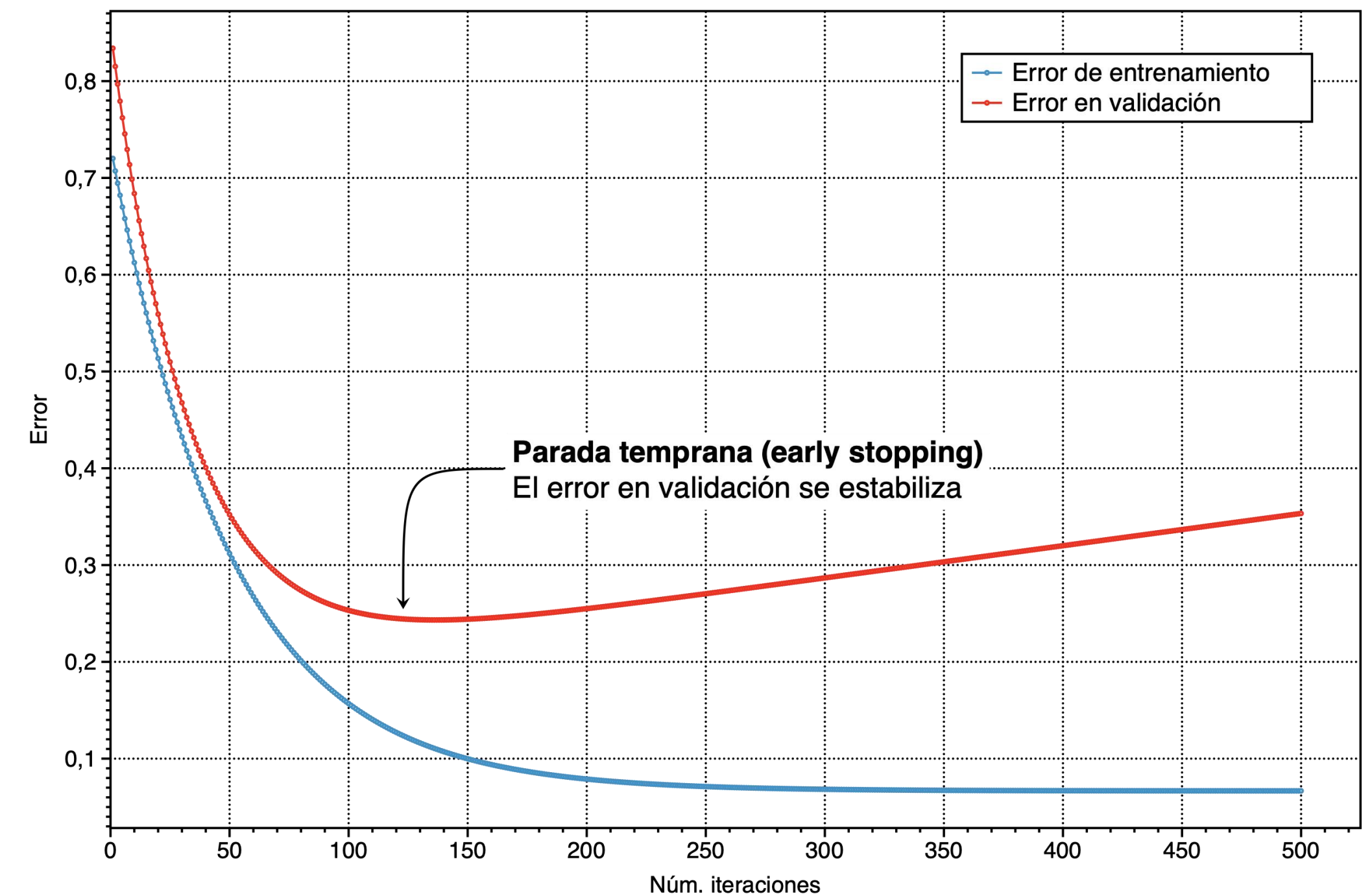
Ejemplo de «data augmentation»

Un ejemplo original da lugar a 9 ejemplos de entrenamiento distintos

Evitar el sobreajuste

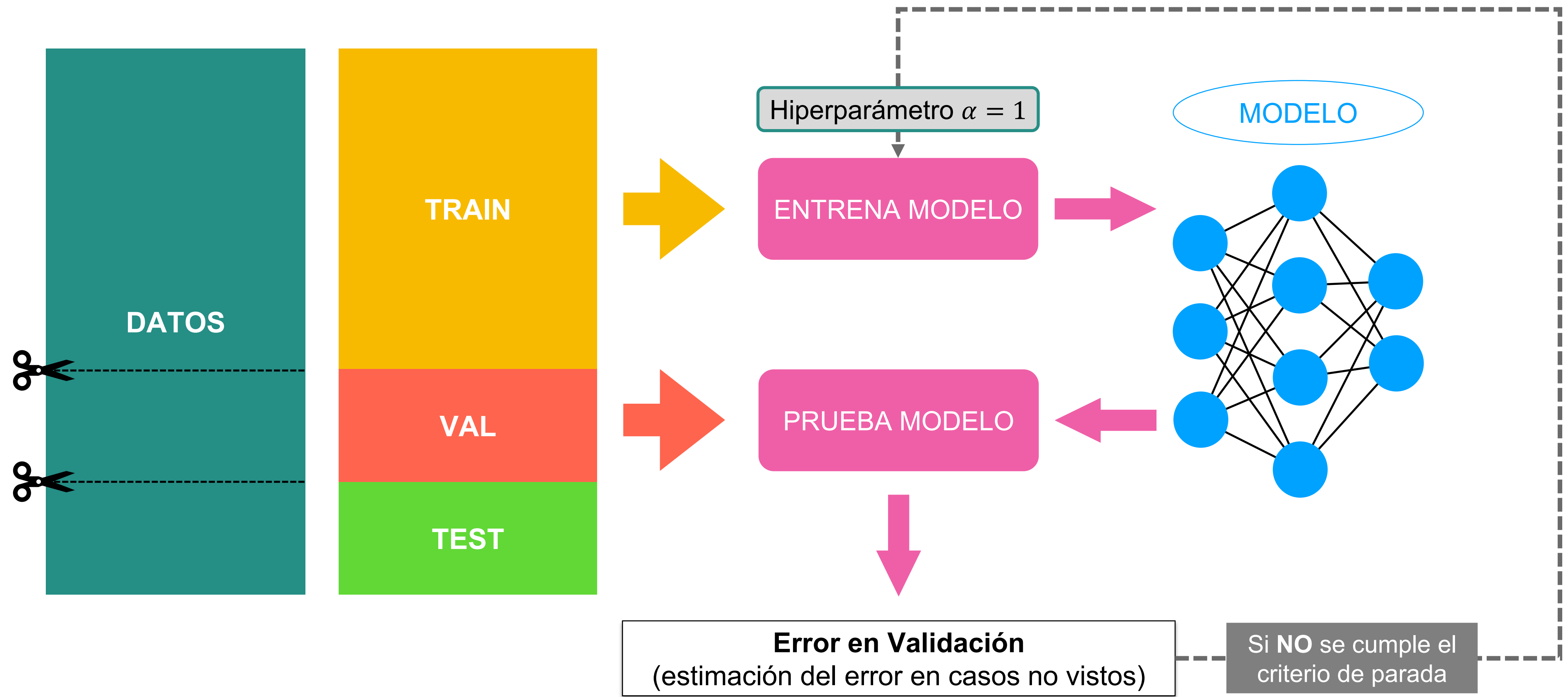
■ 3 - Parada temprana (early stopping)

- Averiguar hasta qué punto hay que seguir entrenando antes de caer en el sobreajuste.
- Funcionamiento:
 - Detenemos el entrenamiento antes de que la diferencia entre validación y entrenamiento varíe demasiado y se alejen.



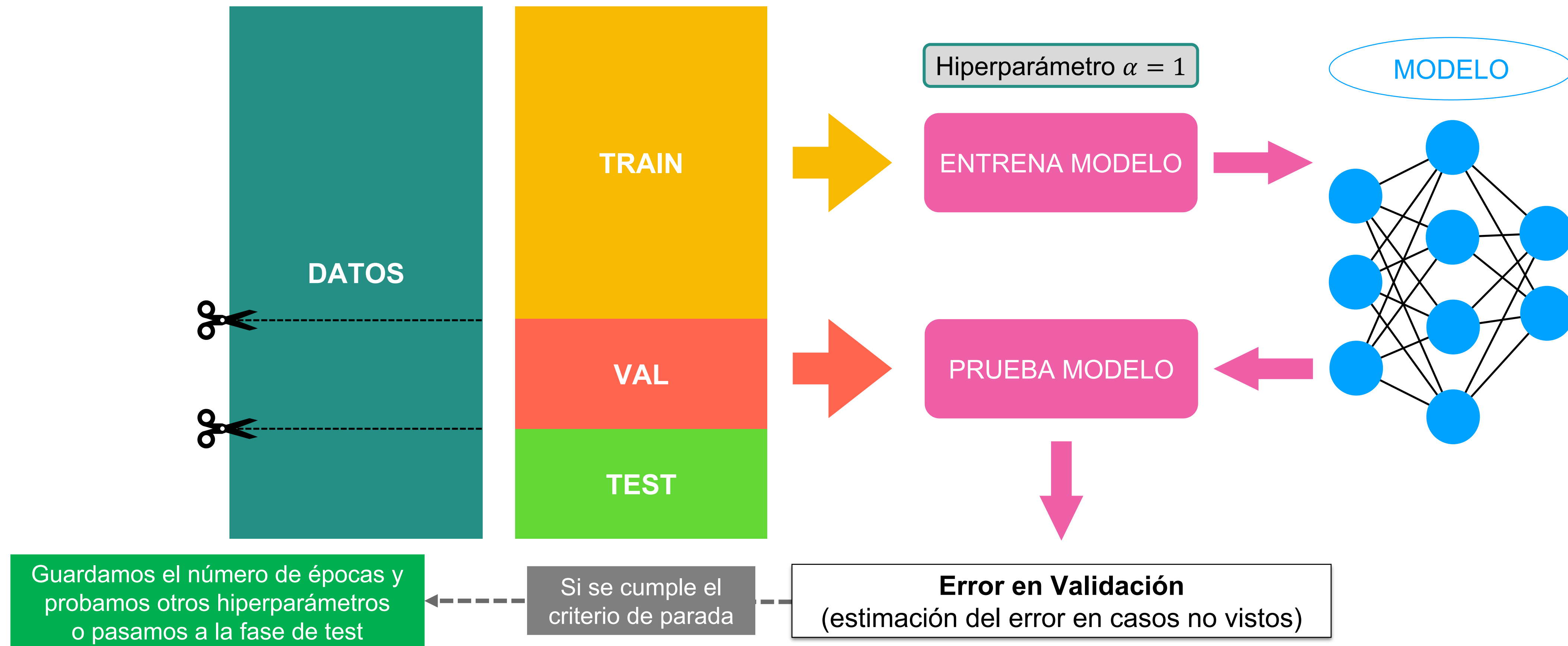
Evitar el sobreajuste

- 3 - Parada temprana (early stopping)



Evitar el sobreajuste

- 3 - Parada temprana (early stopping)



Contenido

1. Introducción

2. Arquitectura

3. Entrenamiento

4. Técnicas

- Evitar el sobreajuste
- Acelerar el entrenamiento

5. Resumen

Acelerar el entrenamiento

- **Técnicas:**
 1. Reducción de la ganancia de aprendizaje (learning rate).
 2. Estandarización de variables.
 3. Uso de mini-batches.
 4. Optimizador Adam.

Acelerar el entrenamiento

■ 1 - Reducción de la ganancia de aprendizaje (learning rate)

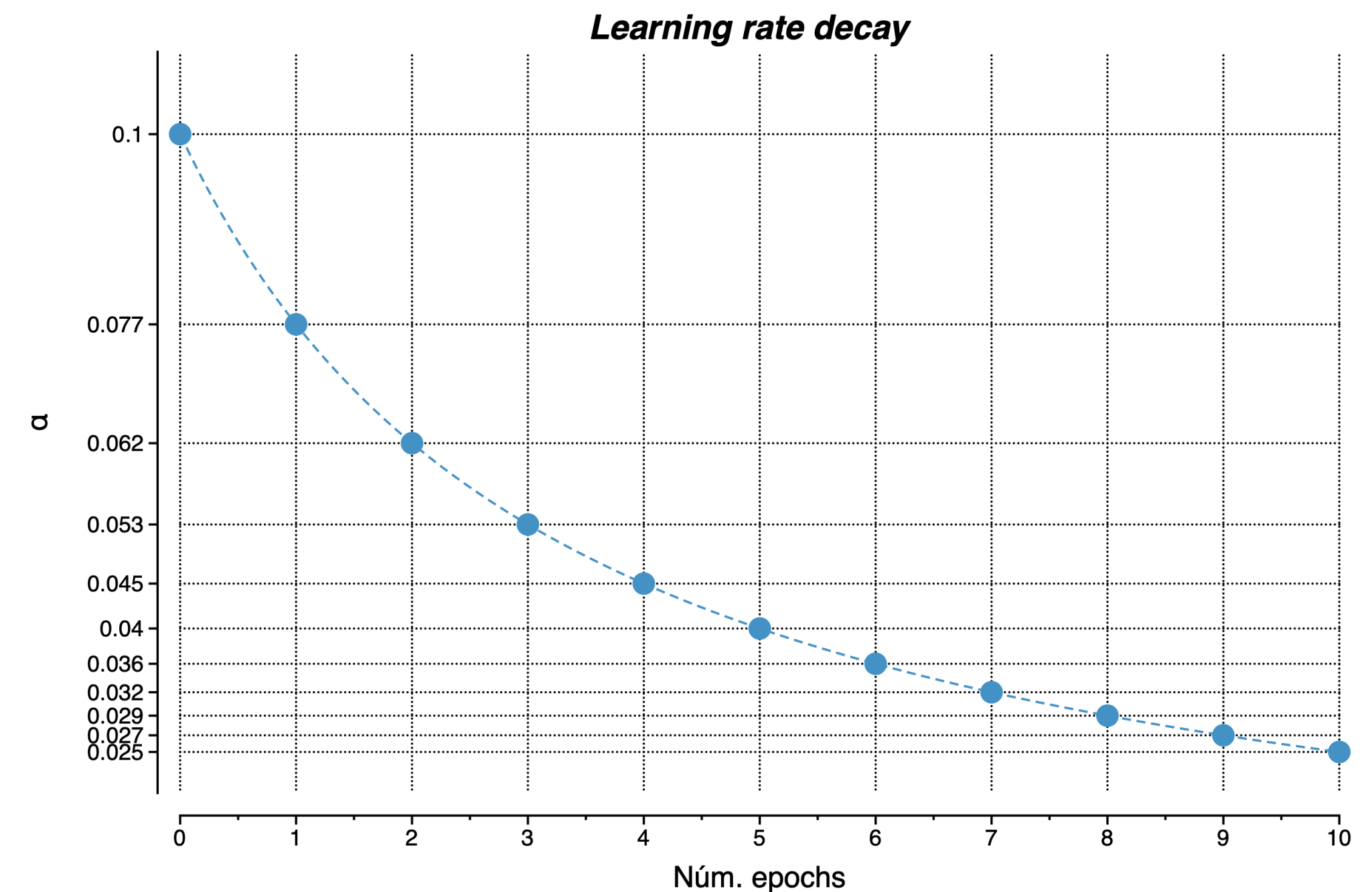
• Problema:

- A medida que el sistema converge, es necesario hacer modificaciones menos bruscas a los parámetros (pesos) de la red

• Solución:

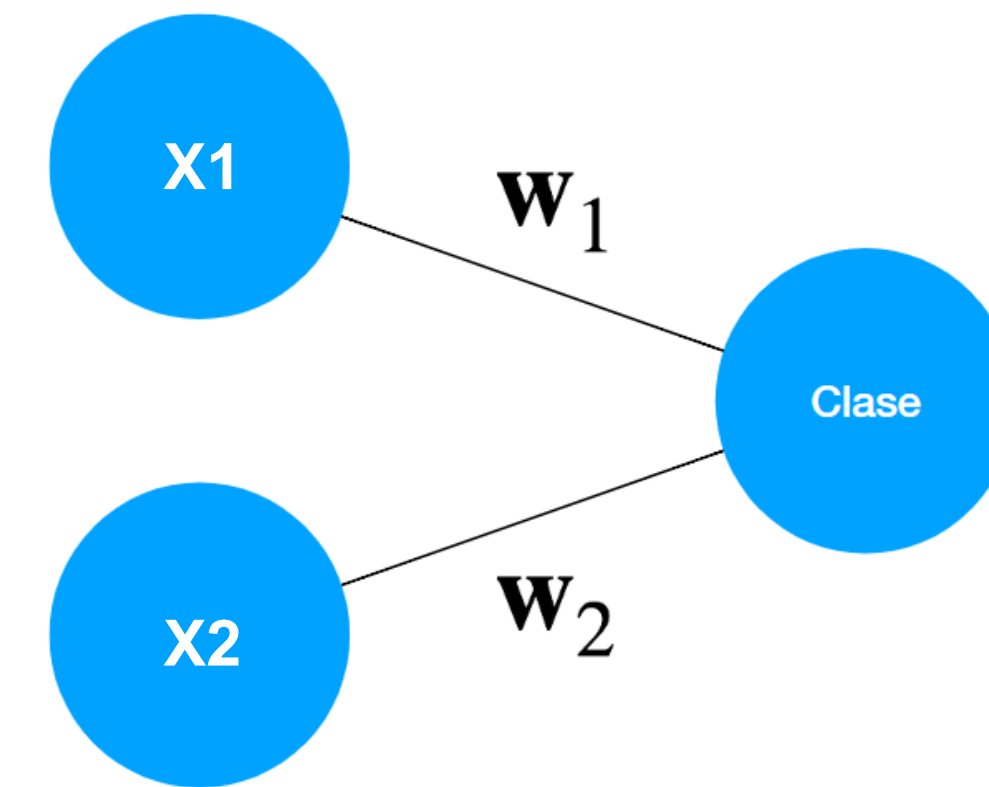
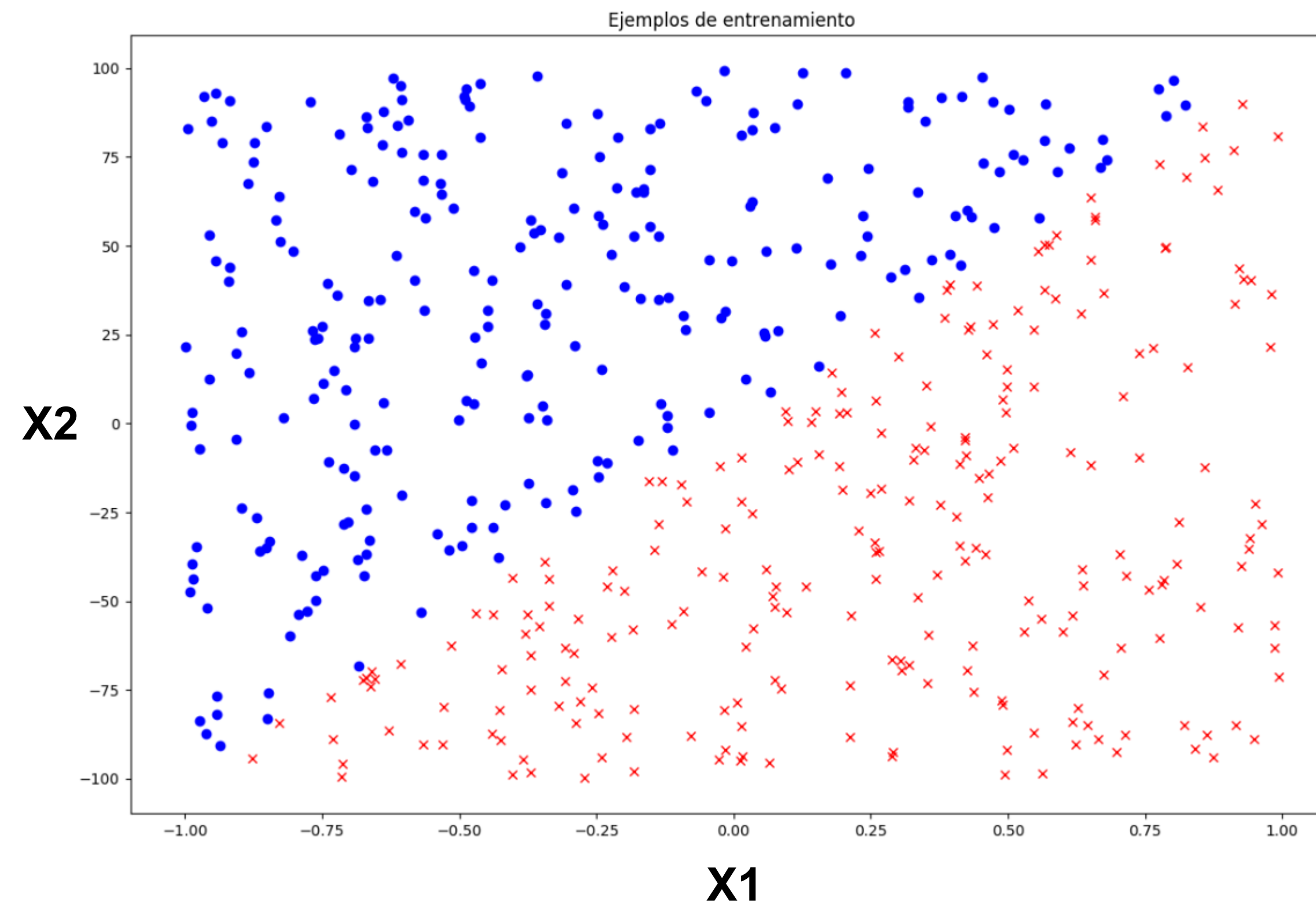
- Se puede utilizar una ganancia de aprendizaje variable, en función del número de iteraciones (epochs):

$$\alpha_i = \frac{1}{1 + \rho \alpha_0}$$



Acelerar el entrenamiento

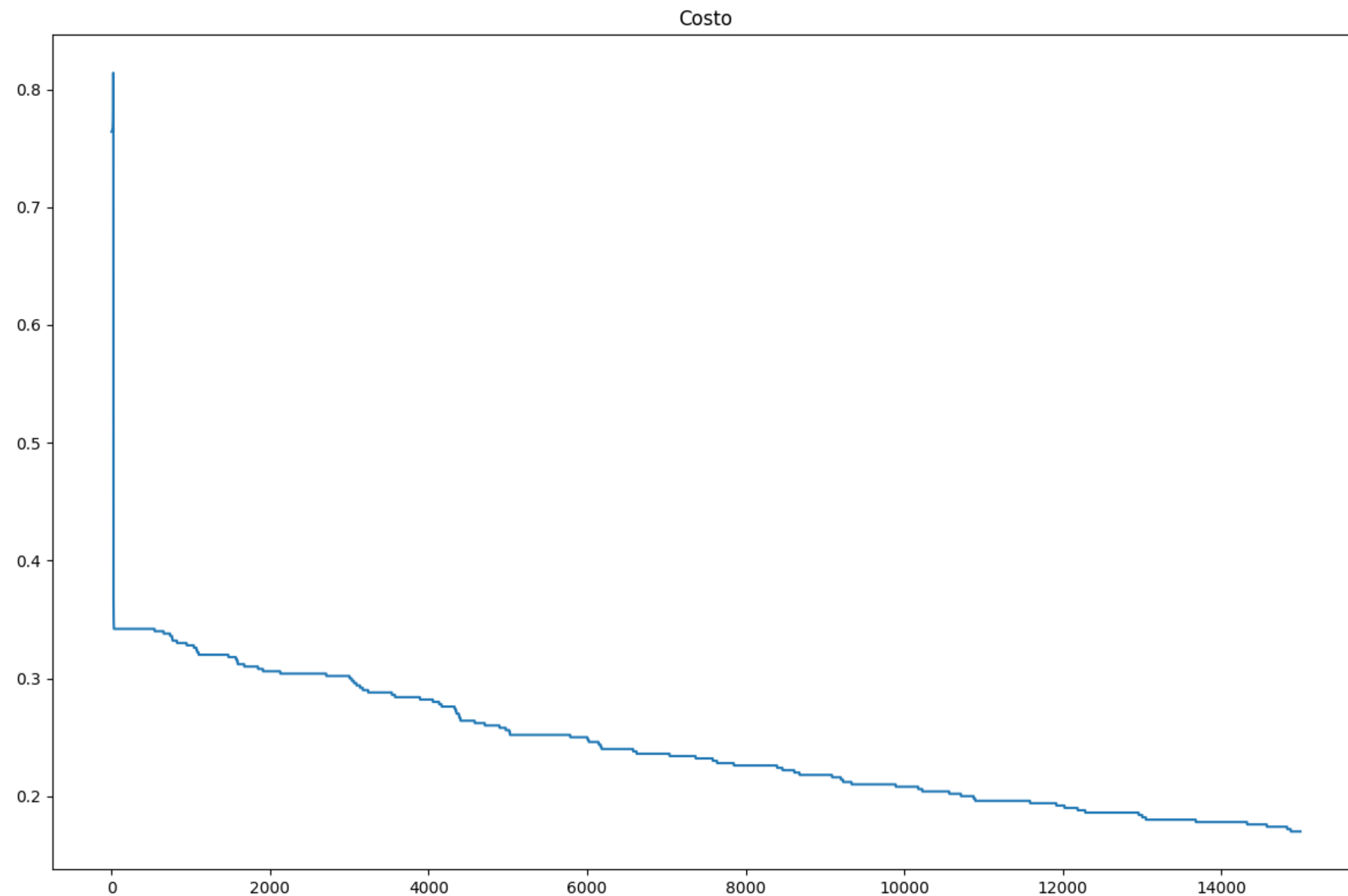
- 2 - Estandarización de variables.



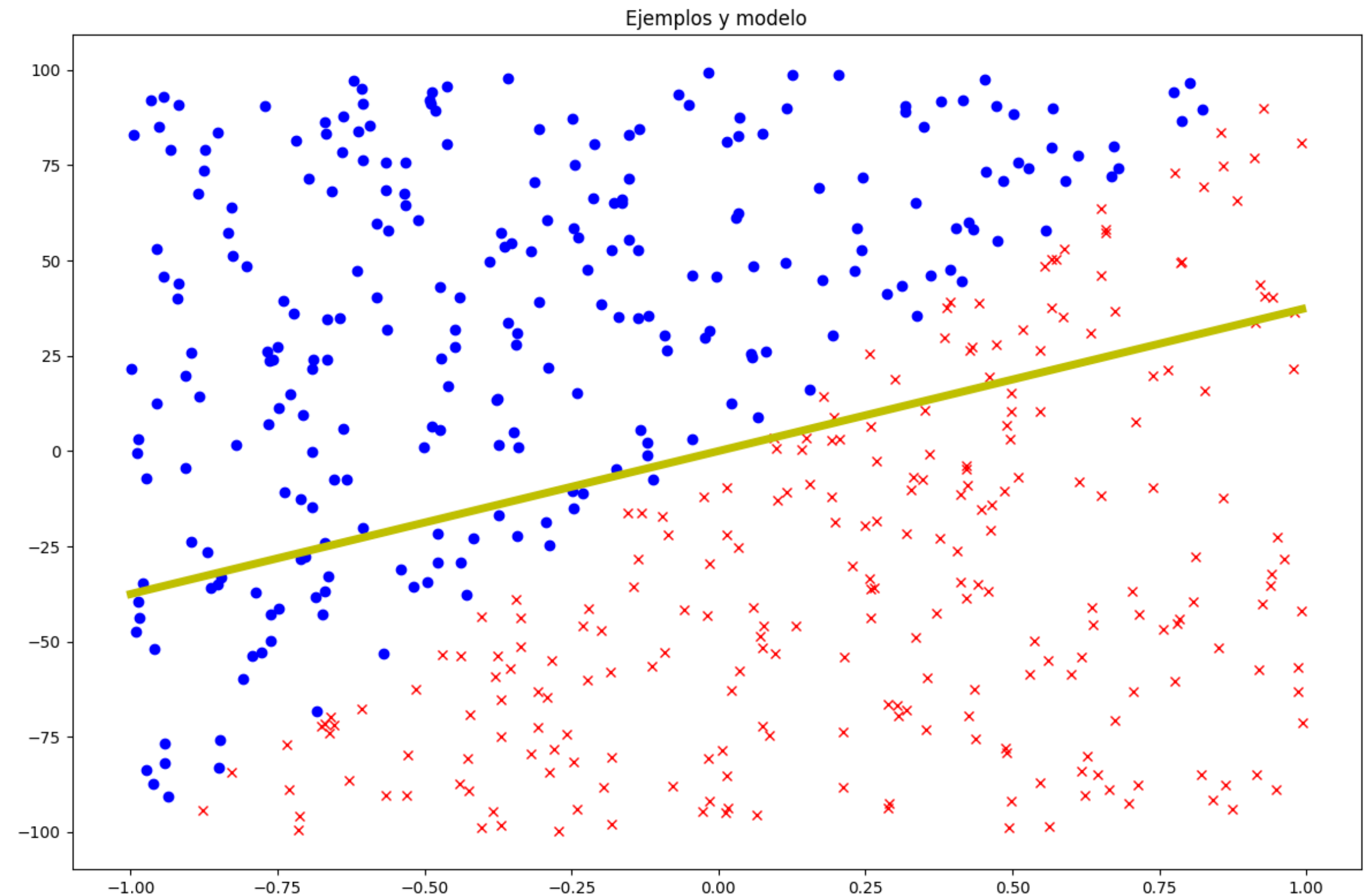
$$\Pr(c = \bullet \mid x_1, x_2) = \frac{1}{1 + e^{-f(x_1, x_2)}}$$
$$f(x_1, x_2) = w_1 x_1 + w_2 x_2$$

Acelerar el entrenamiento

- 2 - Estandarización de variables.



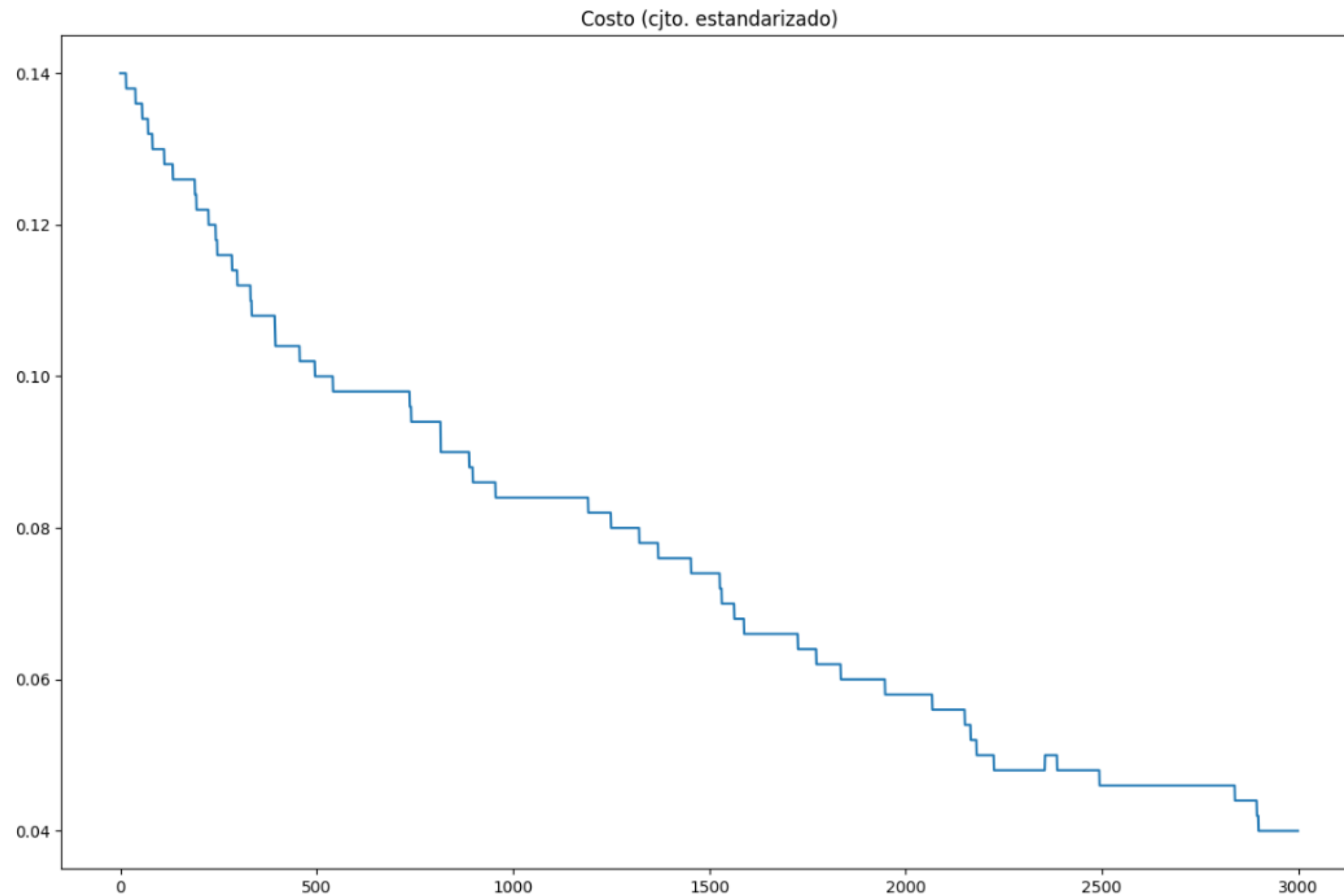
**Error de entrenamiento
sobre el conjunto original
(17%)**



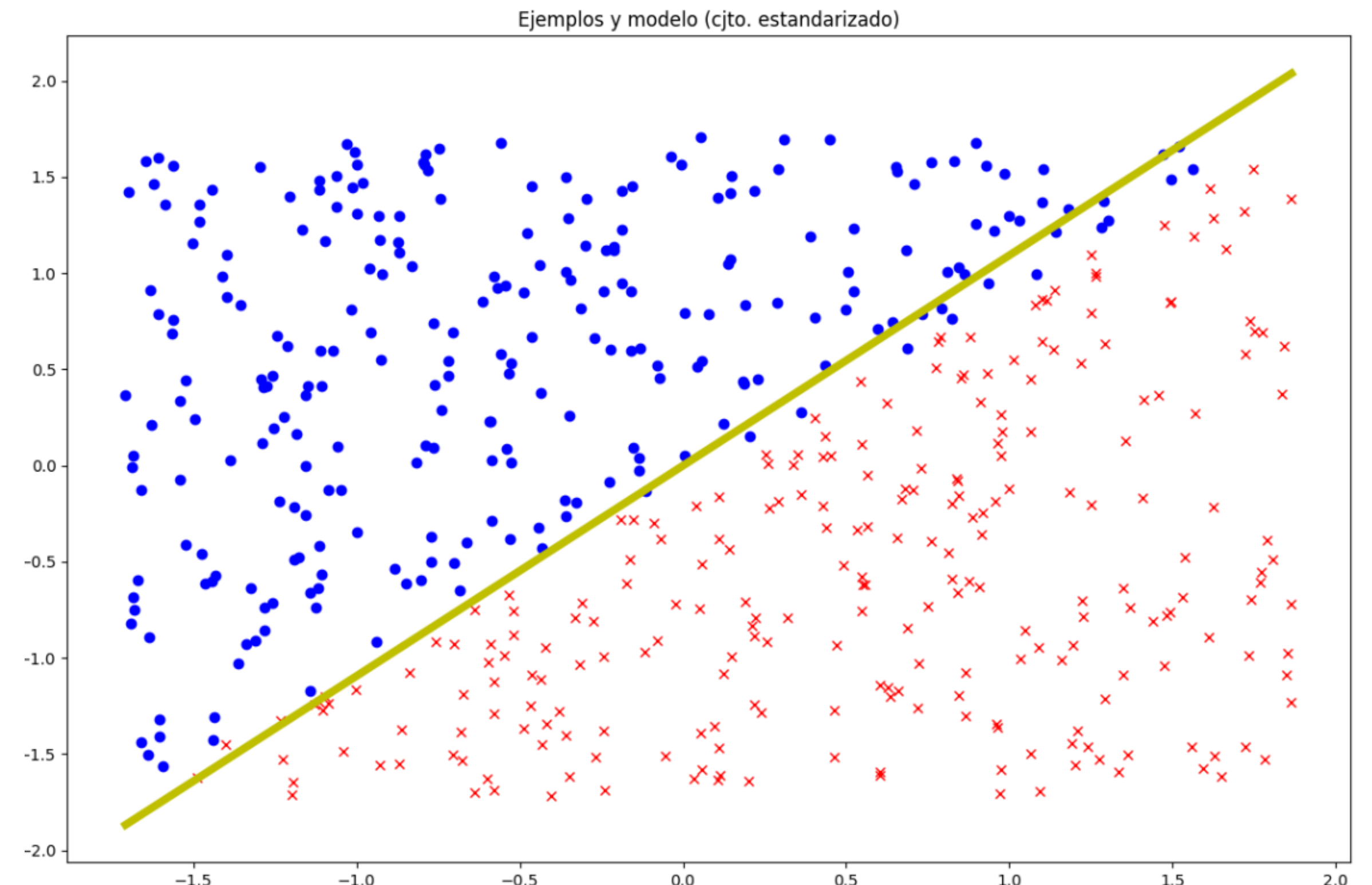
**Modelo obtenido
sobre el conjunto original
tras **15000** iteraciones**

Acelerar el entrenamiento

- 2 - Estandarización de variables.



**Error de entrenamiento
sobre el conjunto estandarizado
(4%)**



**Modelo obtenido
sobre el conjunto estandarizado
tras 3000 iteraciones**

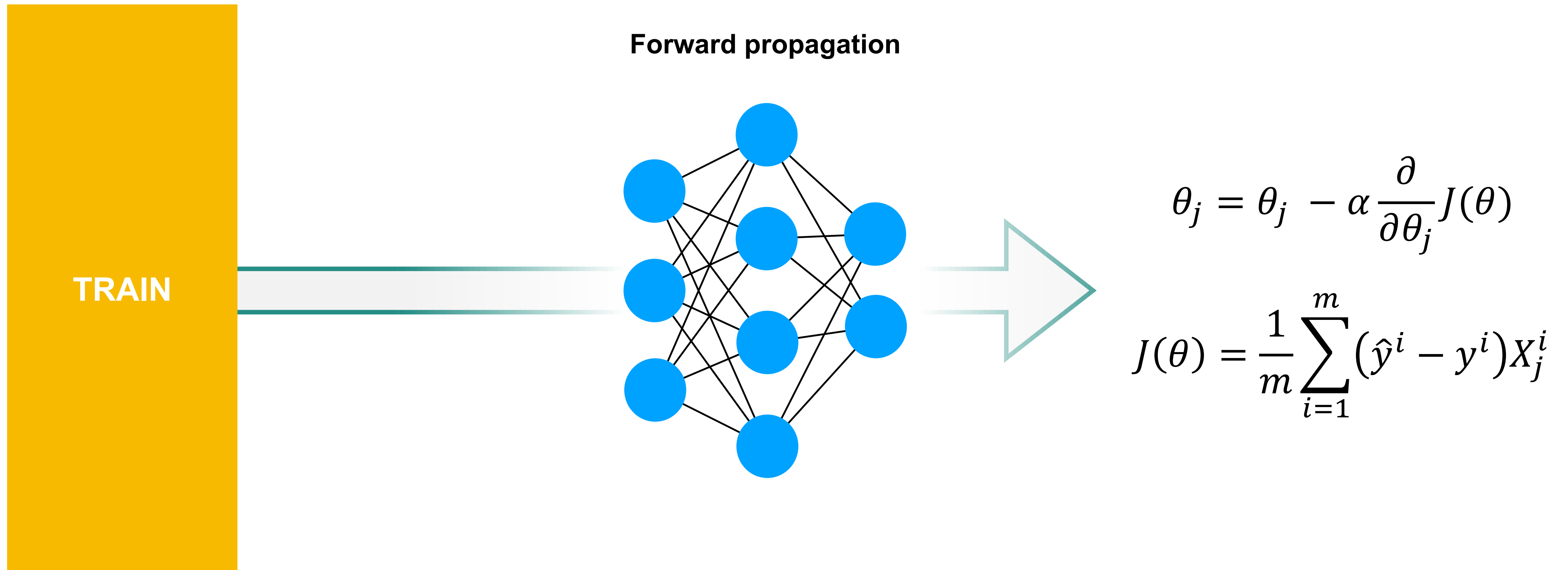
Acelerar el entrenamiento

- **3 - Uso de mini-batches**

- En contextos de Big Data el descenso de gradiente:
 - Ralentiza mucho el entrenamiento.
 - Requiere mucha memoria.
- Solución:
 - Partir el conjunto en mini-batches y aplicar el algoritmo de aprendizaje de forma iterativa sobre todos los mini-batches, repetidas veces.

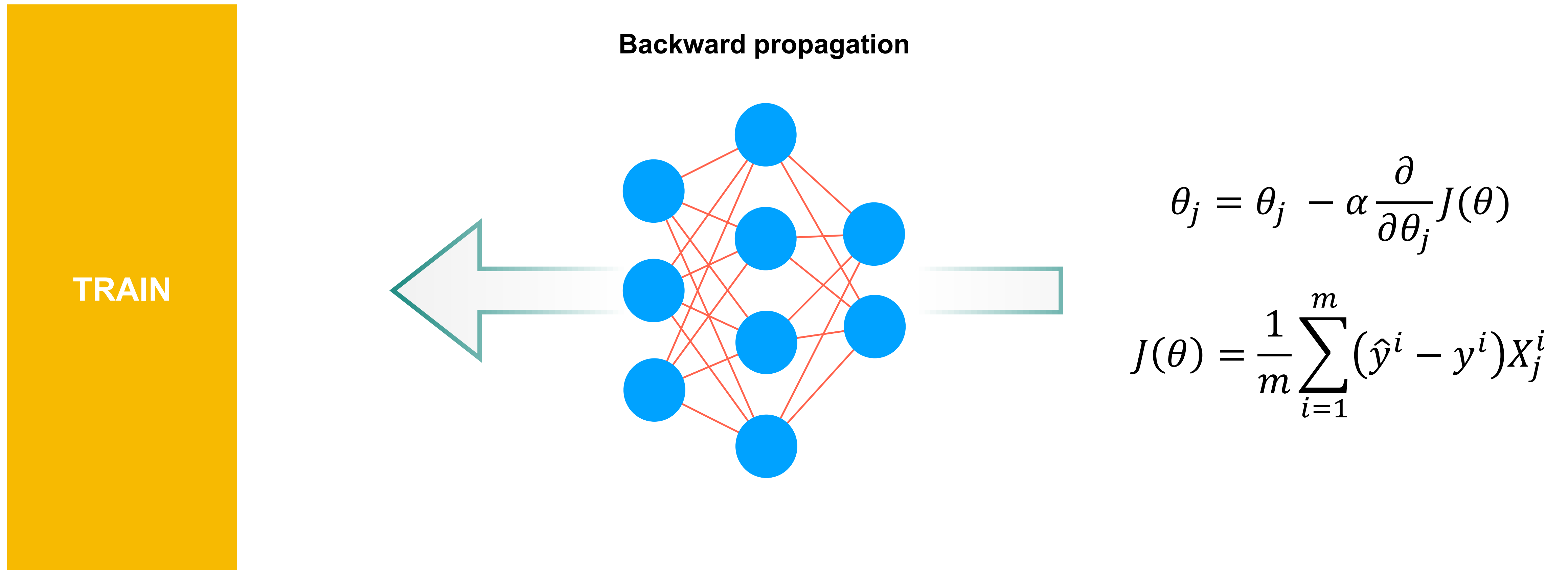
Acelerar el entrenamiento

- Gradient descent tradicional



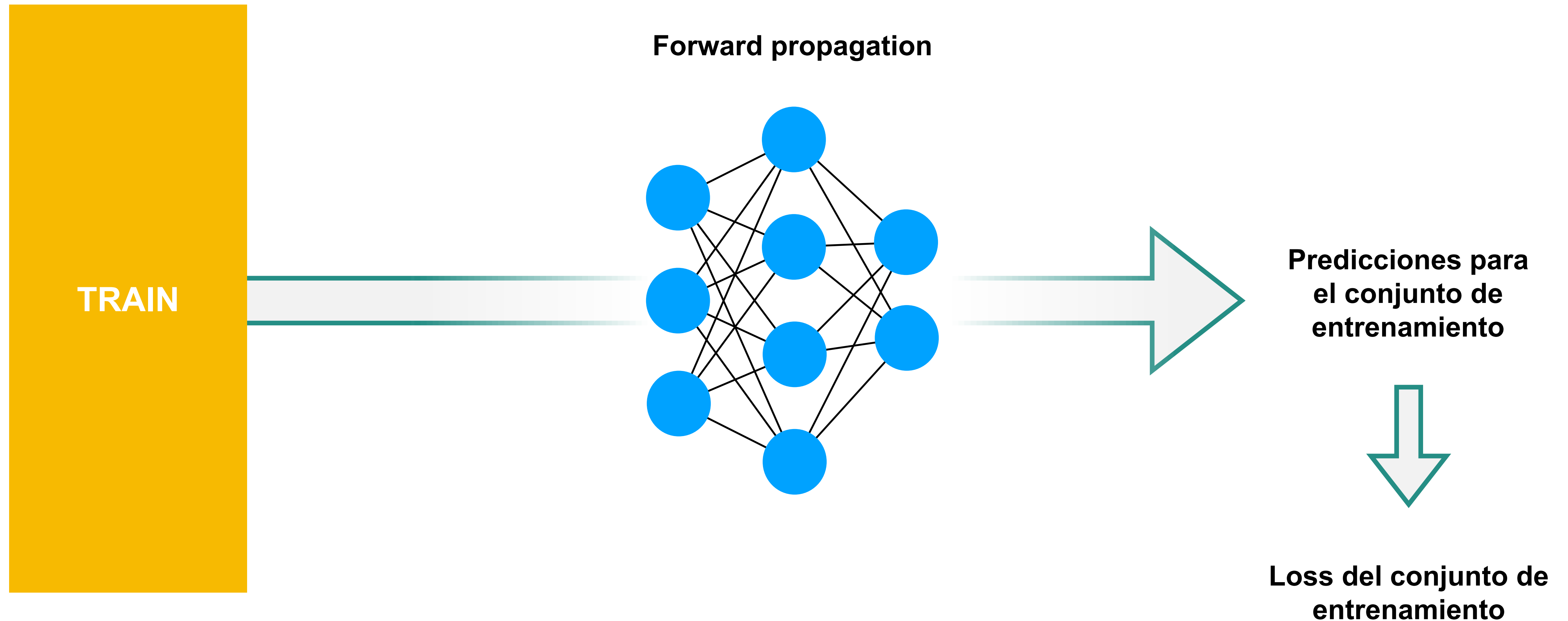
Acelerar el entrenamiento

- Gradient descent tradicional



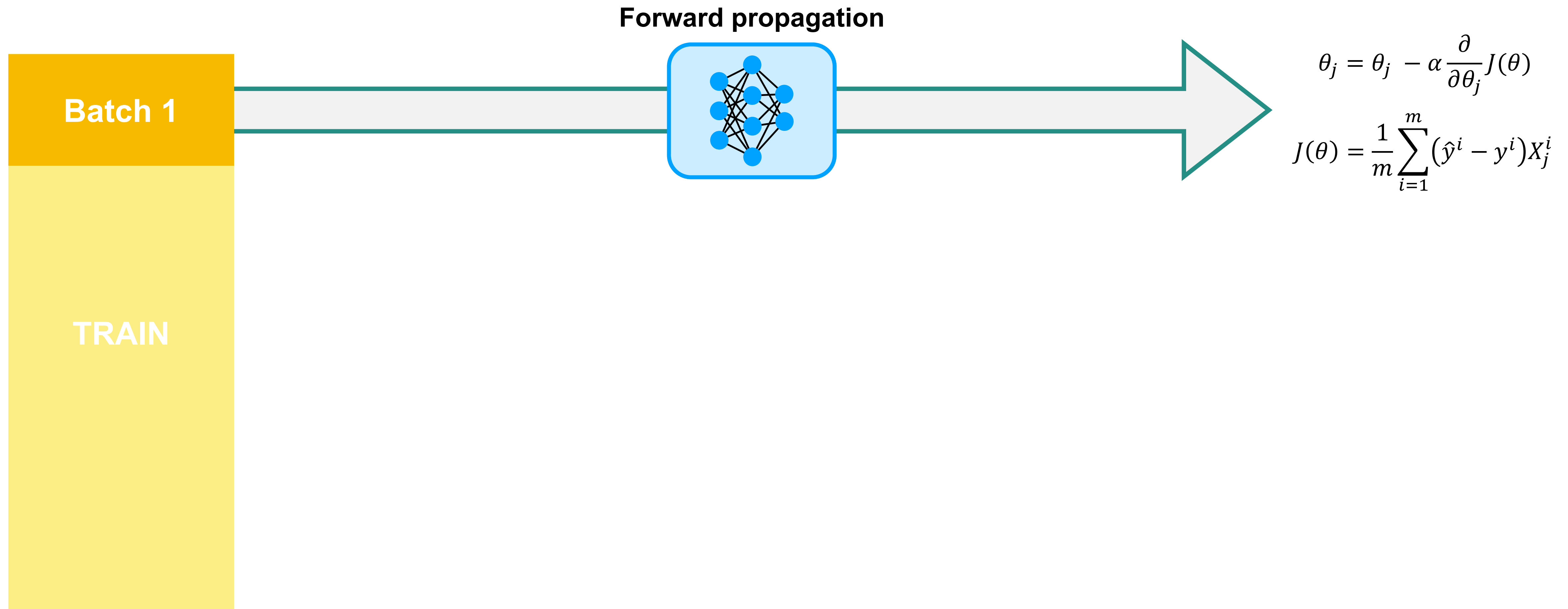
Acelerar el entrenamiento

- Gradient descent tradicional



Acelerar el entrenamiento

- Mini-batch Gradient descent

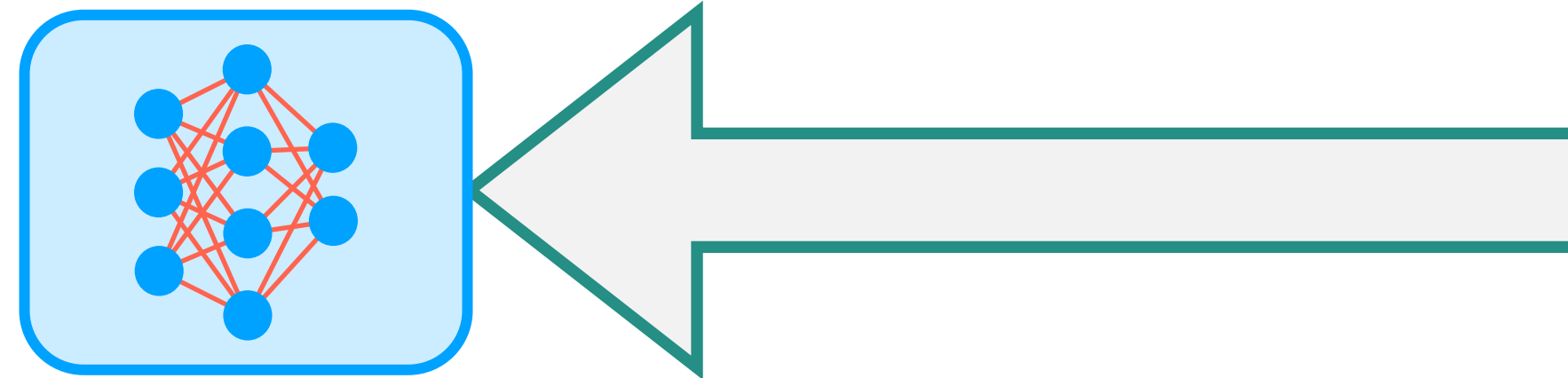


Acelerar el entrenamiento

- Mini-batch Gradient descent



Backward propagation

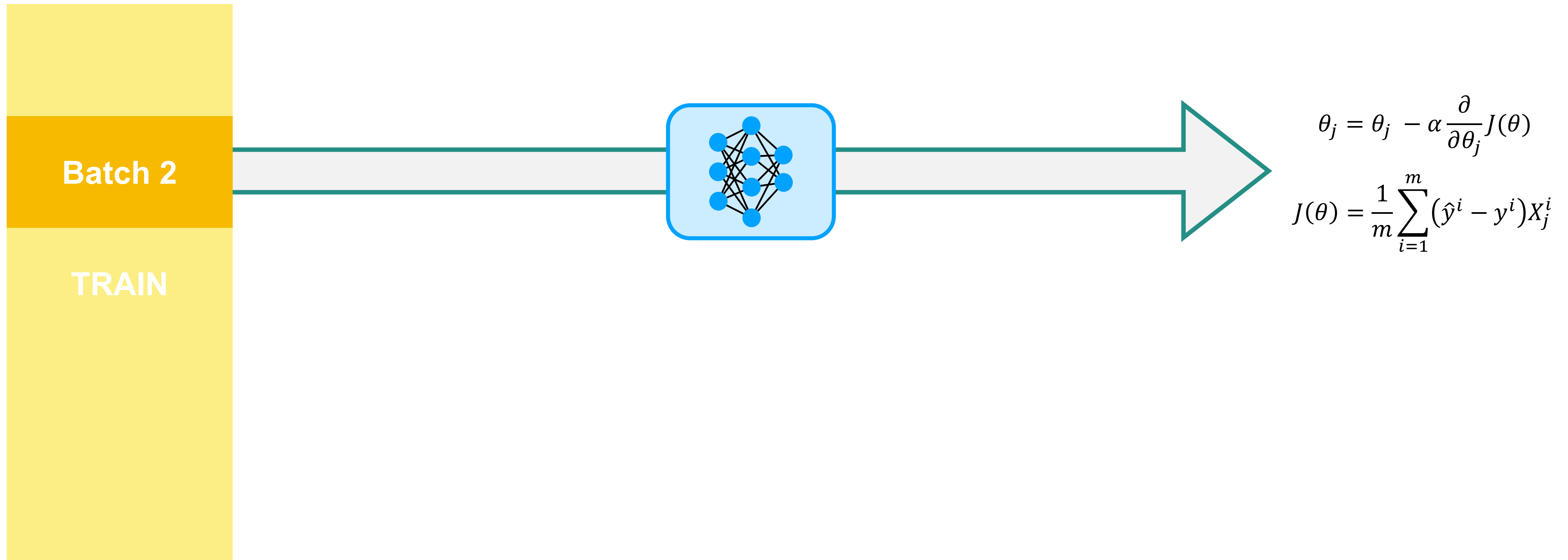


$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$
$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (\hat{y}^i - y^i) X_j^i$$

Acelerar el entrenamiento

- Mini-batch Gradient descent

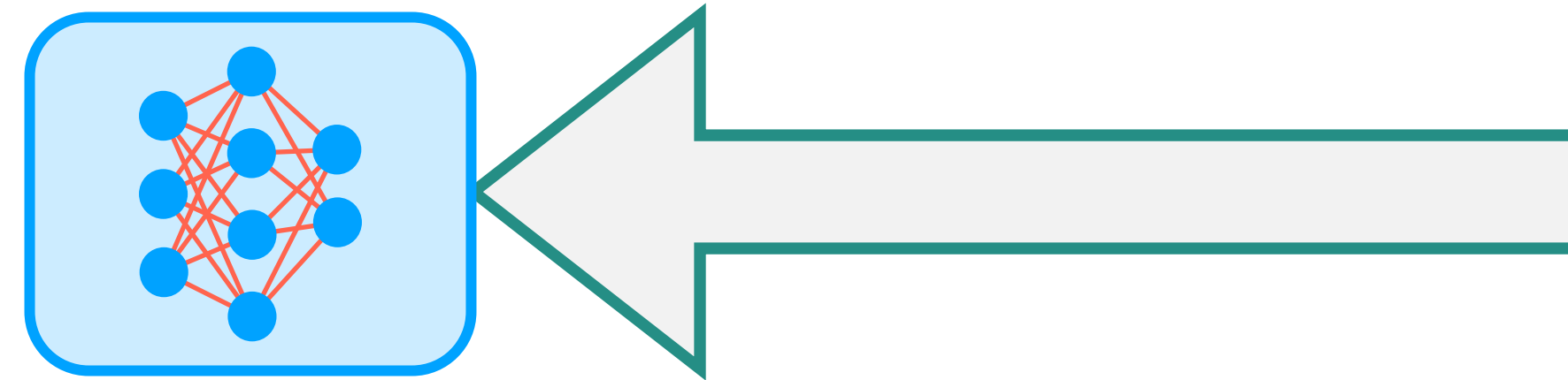
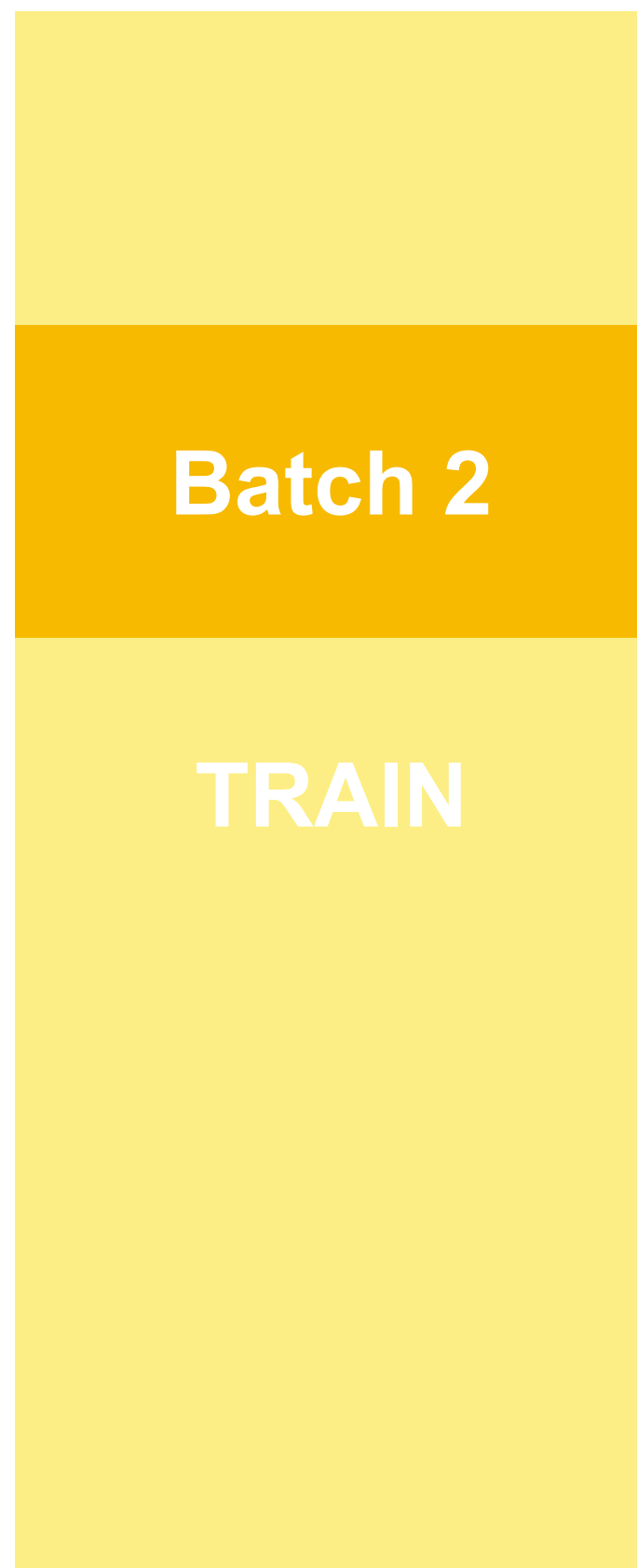
Forward propagation



Acelerar el entrenamiento

- Mini-batch Gradient descent

Backward propagation

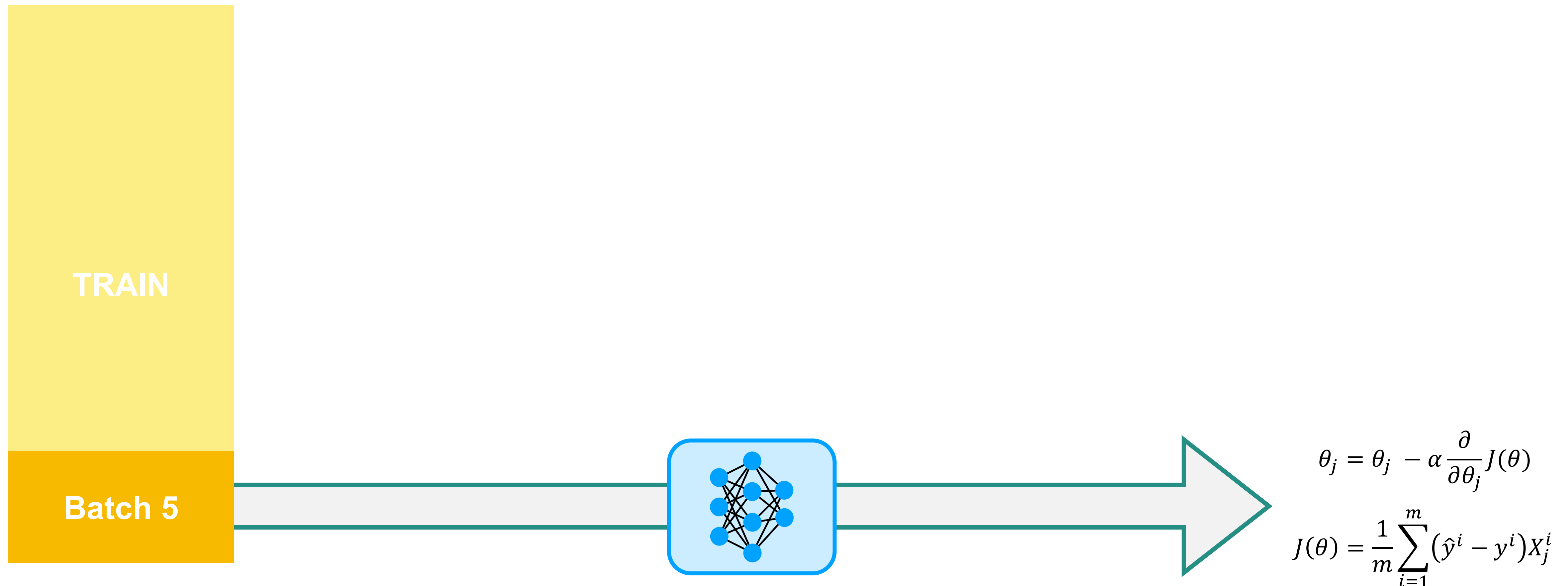


$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$
$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (\hat{y}^i - y^i) X_j^i$$

Acelerar el entrenamiento

- Mini-batch Gradient descent

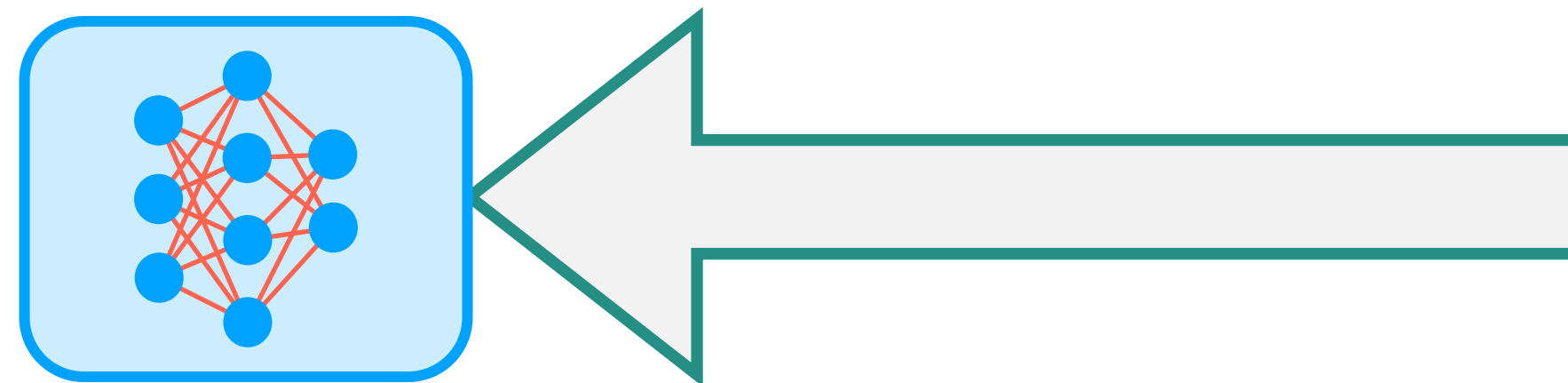
Forward propagation



Acelerar el entrenamiento

- Mini-batch Gradient descent

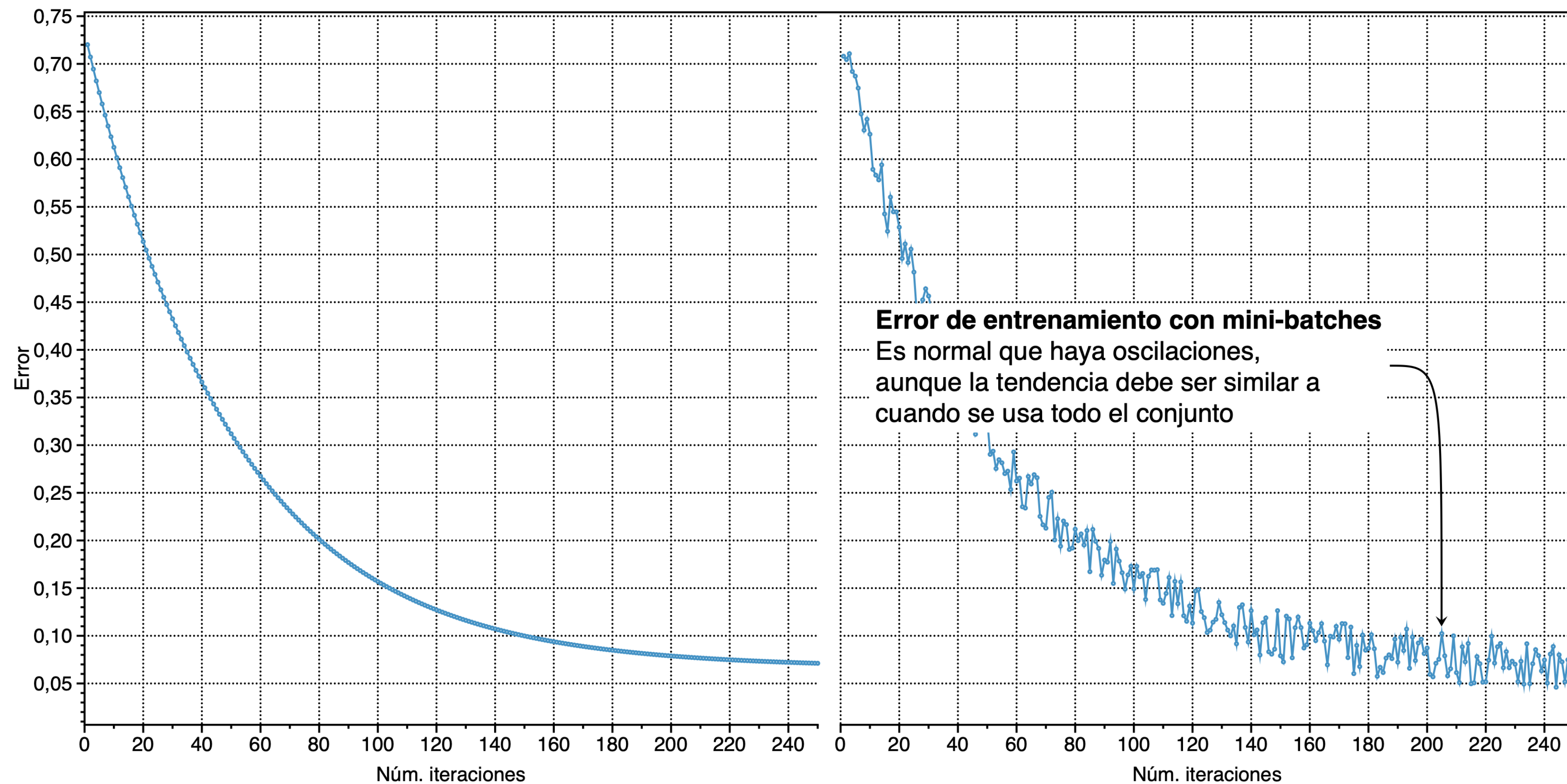
Backward propagation



$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$
$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (\hat{y}^i - y^i) X_j^i$$

Acelerar el entrenamiento

- Gradient descent vs Mini-batch Gradient descent



Acelerar el entrenamiento

- **Mini-batch Gradient descent**

- Dependiendo del tamaño del conjunto de entrenamiento, n , y del tamaño del mini-batch, m , estaremos aplicando:
 - $m = n \rightarrow$ Descenso de gradiente.
 - $m = 1 \rightarrow$ Descenso de gradiente estocástico.
 - $1 < m < n \rightarrow$ Descenso de gradiente con mini-batch.
- Suele ser una buena práctica que m sea potencia de 2:
 - 128, 256, etc.

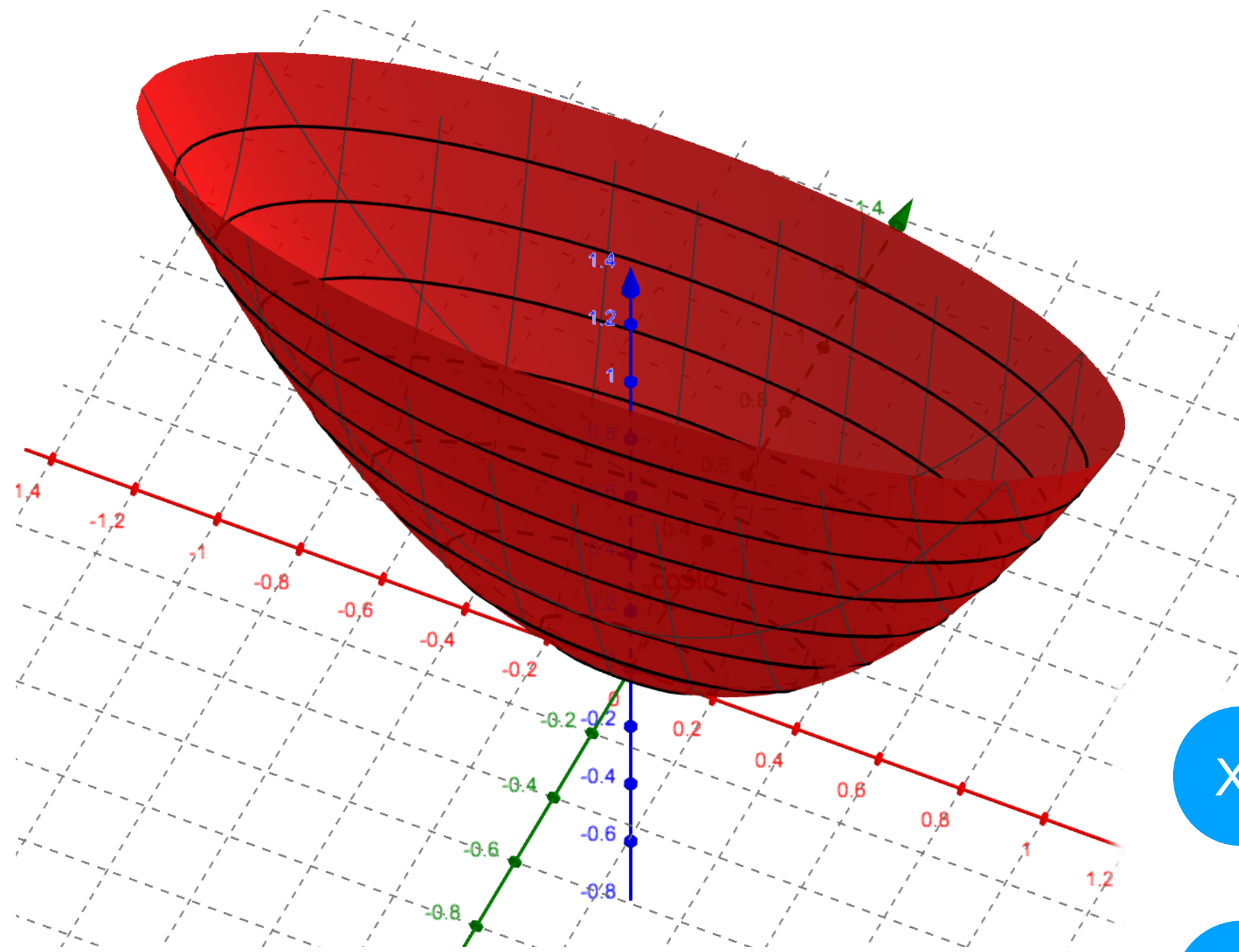
Acelerar el entrenamiento

- **4 - Optimizador Adam**

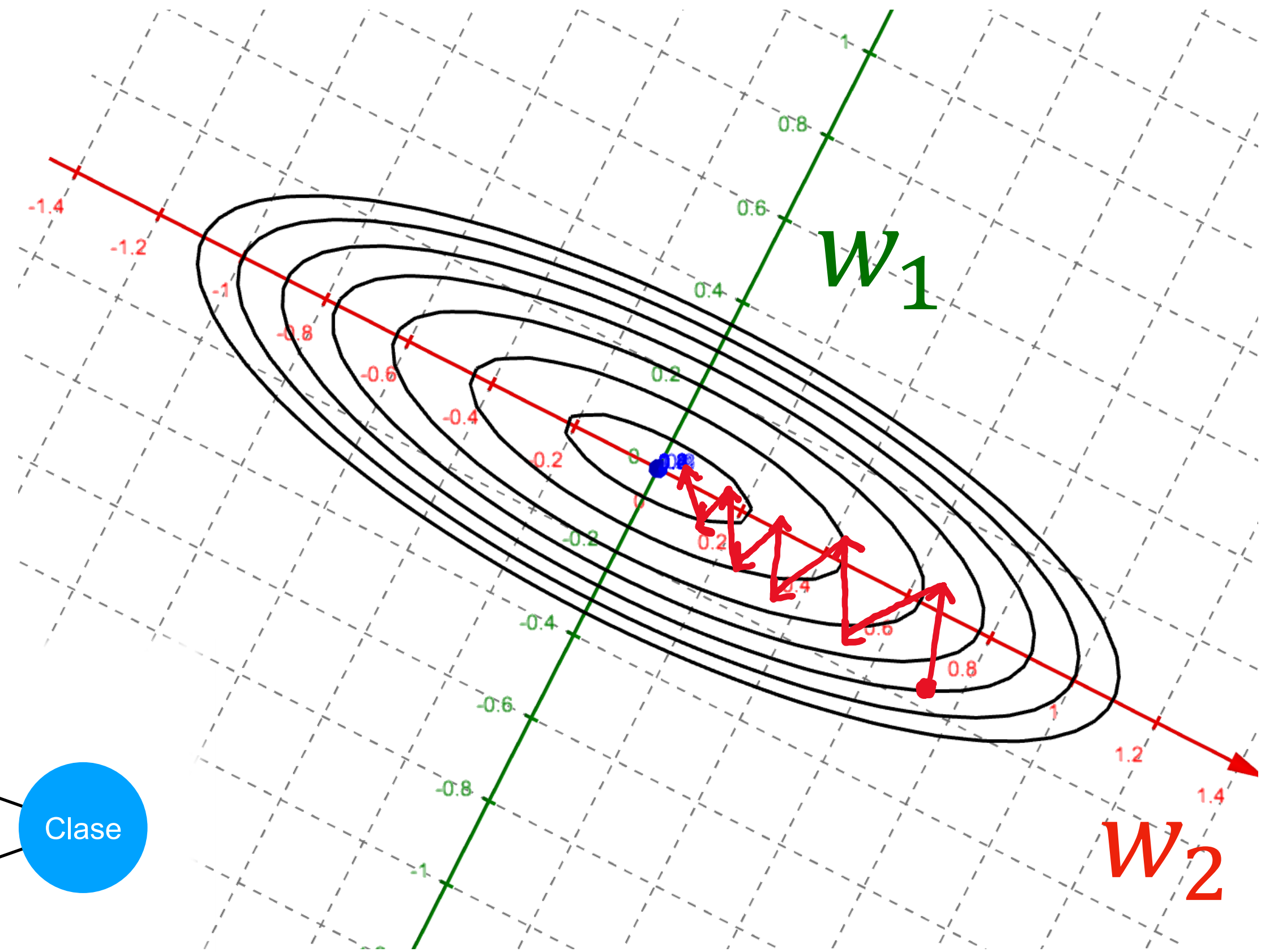
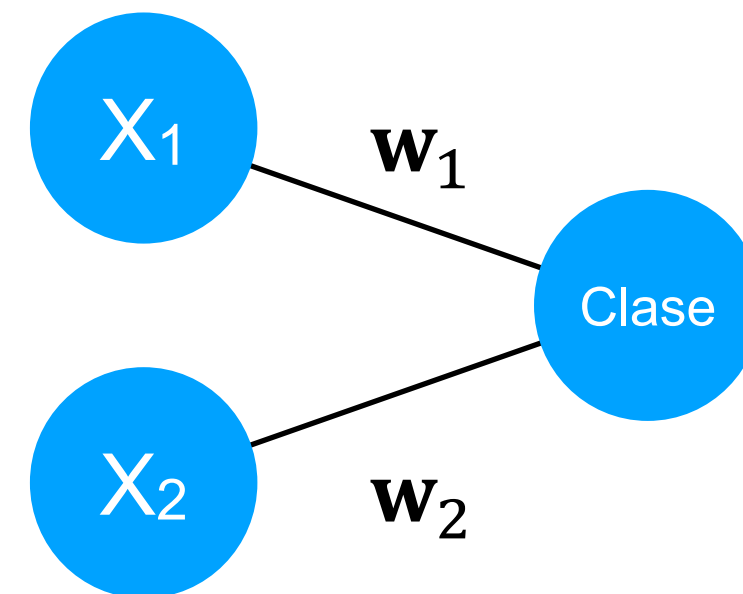
- Combina dos modificaciones al descenso de gradiente tradicional:
 - Descenso de gradiente con momento.
 - RMSprop.

Acelerar el entrenamiento

■ 4 - Optimizador Adam



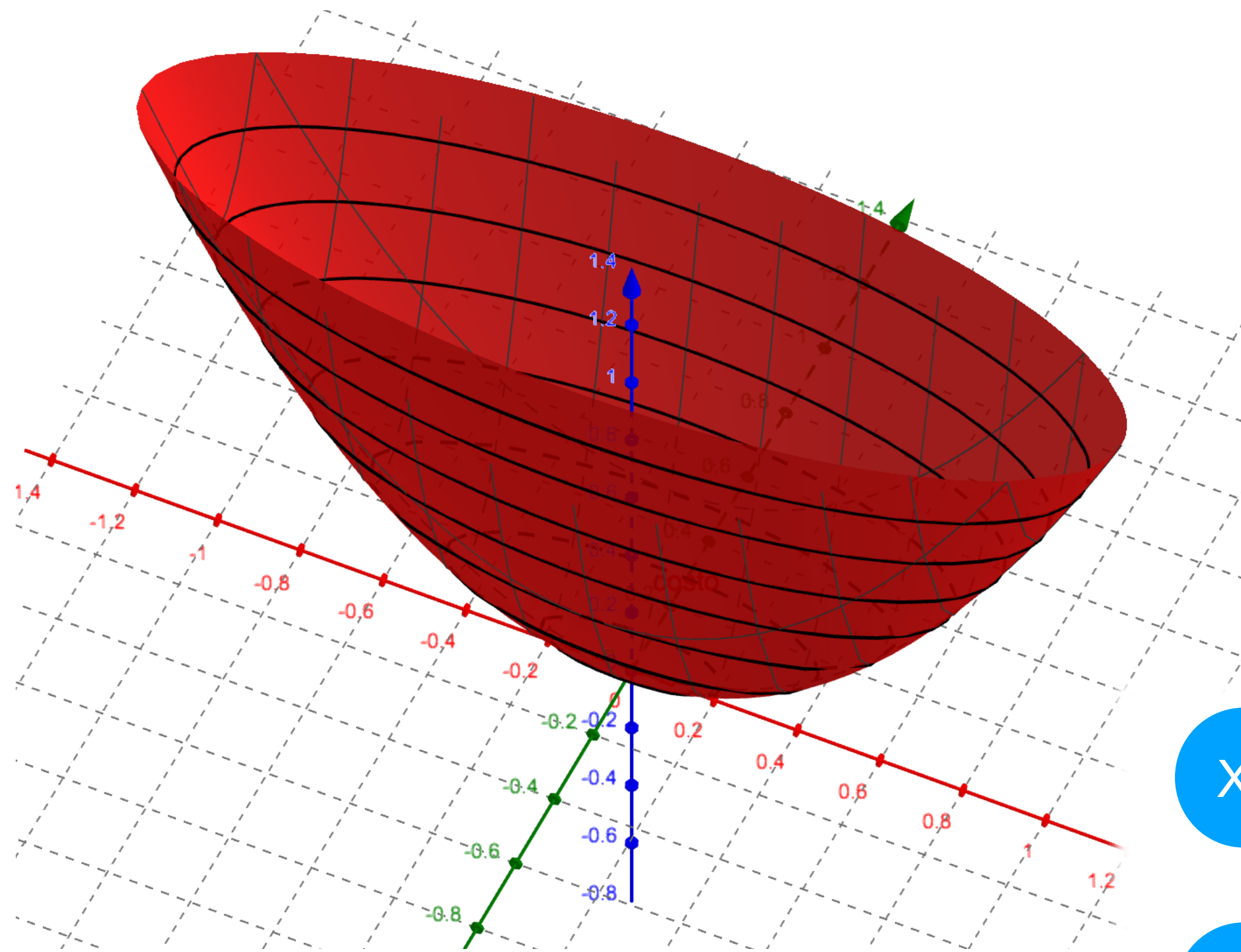
Hipotética función de costo



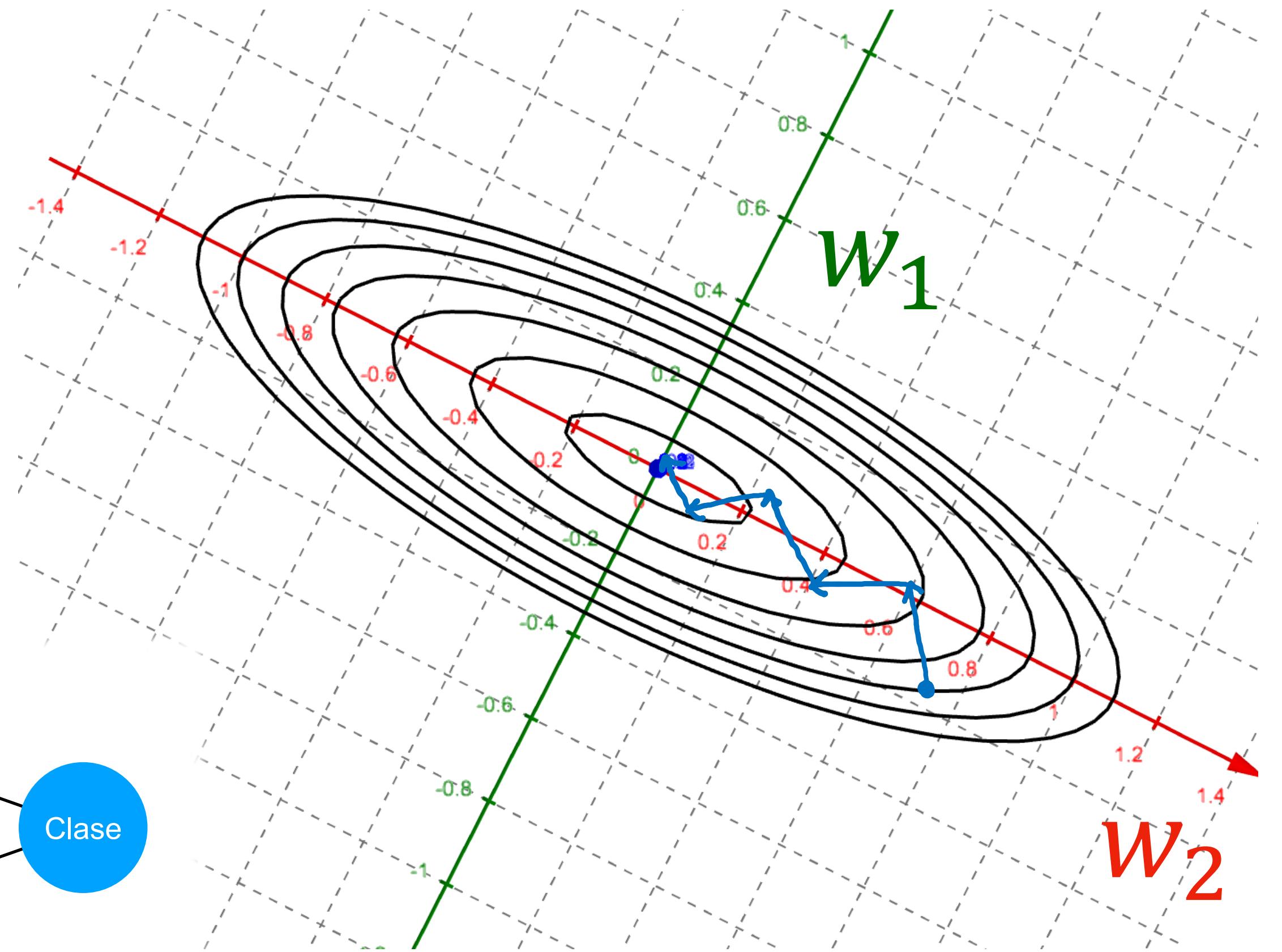
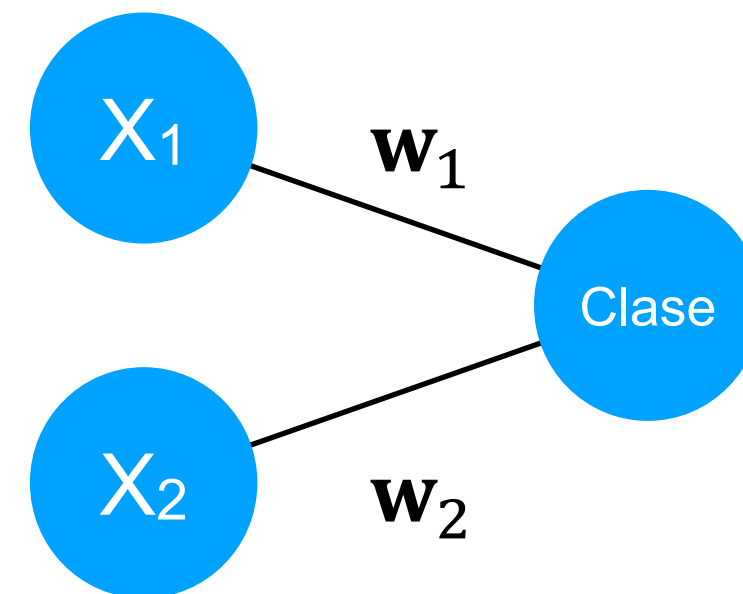
Contorno de la función de costo

Acelerar el entrenamiento

■ 4 - Optimizador Adam



Hipotética función de costo



Contorno de la función de costo

Contenido

1. Introducción
2. Arquitectura
3. Entrenamiento
4. Técnicas
 - Evitar el sobreajuste
 - Acelerar el entrenamiento
5. Resumen

Resumen

- **Redes neuronales**
 - Formadas por múltiples perceptrones combinados en capas fully connected.
 - Sirven para resolver todo tipo de problemas (regresión, clasificación, ...).
- **Cada capa fully connected tiene:**
 - **Parámetros:** Pesos y bias.
 - **Hiperparámetros:** Número de neuronas.
- **La loss se optimiza mediante:**
 - GD, Mini-batch GD, Adam, ...
- **Buscar equilibrio entre:**
 - **Overfitting:** DropOut, Data Augmentation, ...
 - **Underfitting:** Más épocas, modelo más complejo, ...